重 庆 邮 电 大 学

CHONGQING UNIVERSITY OF POSTS AND TELECOMMUNICATIONS

专业学位硕士学位论文

DOCTORAL DISSERTATION



论文题目 异构多核平台下的服务质量感知混合任务调度研究

专业学位类别	电子信息
学　　号	**S220132051**
作者姓名	何爽
指导教师	夏军　高级工程师
学　　院	通信与信息工程学院

学校代码 10617 UDC 004.2
分类号 TP301 密级 公开

学 位 论 文

异构多核平台下的服务质量感知混合任务调度研究

何 爽

指导教师 夏 军 高级工程师

申请学位级别 硕士 学科专业 电子信息
专业学位领域 通信工程（含宽带网络、移动通信等）
答辩委员会主席 柴蓉 教授 论文答辩日期 2025 年 5 月 18 日
学位授予单位和日期 重庆邮电大学 2025 年 6 月

Research on Scheduling for QoS-aware Hybrid Task on Heterogeneous Multicore Platforms

A Master Thesis Submitted to

Chongqing University of Posts and Telecommunications

Discipline	**Electronic and Information Engineering**
Student ID	**S220132051**
Author	**HE Shuang**
Supervisor	**SN.ENGR. XIA Jun**
School	**School of Communications and Information Engineering**

重庆邮电大学

学位论文独创性声明

本人郑重声明：所呈交的学位论文，是本人在导师指导下，独立进行研究工作所取得的成果。除文中已经注明引用的内容外，本论文中不包含其他个人或集体已经发表或撰写过的作品成果。对本文的研究做出重要贡献的个人和集体，均已在论文中以明确方式标明。本人完全知晓本声明的法律后果由本人承担。

学位论文作者签名：

日期：　　年　　月　　日

重庆邮电大学

学位论文使用授权书

本人同意学校保留并向国家有关部门或机构送交论文的复印件和电子版，允许论文被查阅和借阅。

本学位论文属于 ：

□**公开论文**

□**涉密论文**，保密____年，过保密期后适用本授权书。

（请在以上方框内选择打“√”）

作者签名：　　导师签名：

日期：　　年　　月　　日

摘 要

现有异构多核平台上任务调度问题的研究多集中于单一任务类型（精确计算任务或非精确计算任务），而对同时包含两类任务的混合任务集的研究相对较少。混合任务调度问题的关键在于在满足能耗约束与实时性要求的前提下实现系统服务质量的最大化。本文的研究目标是在异构多核平台上设计一种满足能耗约束及实时性约束的服务质量感知混合任务调度算法。

针对混合任务模型的研究不足，本文基于通用异构多核平台，综合考虑能耗、实时性及任务依赖等多重约束，构建了以最大化系统服务质量为目标的混合任务调度模型。为降低问题求解难度，本文通过线性化与 Benders 分解方法，将原问题分解为主问题和子问题。并基于分解后的问题框架，本文提出了一种基于服务质量感知的混合任务调度近似最优算法。该算法通过主问题与子问题的交替迭代求解，利用 YALMIP 和 CPLEX 求解器求得近似最优解。实验结果表明，该算法在非精确计算任务模型中得到的系统服务质量比其他优化算法高 39.09%；在混合任务模型中得到的系统服务质量比其他优化算法高 23.82%~31.48%。然而，该算法的能耗使用比其他优化算法高 14.81%。

针对近似最优算法在中大规模任务集下求解时间长、实现复杂度高的问题，本文又提出了一种基于服务质量感知的混合任务调度启发式算法。该算法采用多阶段策略，分为强制任务分配、空闲时间优化及可选任务分配三个阶段。实验结果表明，该算法在非精确计算任务模型中的系统服务质量相较于其他启发式算法提升了 7.22%~49.18%，在混合任务模型中的系统服务质量提升了 31.05%~64.22%，但是能耗使用提高了 6.25%~21.65%。同时，该算法相较于前文算法大幅缩减了求解时间，且能达到其 31.05%~67.28%的服务质量性能。

关键词：异构多核，混合任务，服务质量，任务调度

ABSTRACT

Current studies on task scheduling in heterogeneous multi-core environments predominantly focus on single task types (either precise or imprecise computation tasks), while research on hybrid-task sets containing both types remains limited. The critical challenge in hybrid-task scheduling lies in maximizing system quality of service (QoS) while satisfying energy constraints and real-time requirements. The research objective of this thesis is to design a QoS-aware hybrid task scheduling algorithm that satisfies energy consumption constraints and real-time requirements on heterogeneous multi-core platforms.

To address the limited research on hybrid task models, comprehensively considering multiple constraints including energy consumption, real-time requirements, and task dependencies on general heterogeneous multi-core platforms, a hybrid task scheduling model is constructed in this thesis to maximize the system QoS. For problem simplification, the original problem is decomposed into master and subproblems through linearization and Benders decomposition methods. Based on this decomposed framework, an approximate optimal QoS-aware hybrid task scheduling algorithm is proposed in this thesis. This algorithm employs alternating iterative solutions between master and subproblems using YALMIP and CPLEX solvers. Experimental results demonstrate that the proposed algorithm achieves 39.09% higher system QoS than existing optimization algorithms in imprecise computation task models and 23.82%–31.48% superiority in hybrid task models, albeit at the cost of 14.81% increased energy consumption.

To resolve the prolonged computation time and high implementation complexity of approximate optimal algorithms in medium-to-large scale task sets, a QoS-aware heuristic scheduling algorithm is further proposed in this thesis. This three-phase strategy includes mandatory task allocation, idle time optimization, and optional task allocation. Experimental results reveal that the heuristic algorithm enhances system service quality by 7.22%–49.18% in imprecise computation models and 31.05%–64.22% in hybrid task models compared to other heuristic approaches, albeit with 6.25%–21.65% higher energy consumption. Additionally, it significantly reduces

computation time while maintaining 31.05%–67.28% of the service quality achieved by the preceding near-optimal algorithm.

Keywords: Heterogeneous Multi-core, Hybrid Tasks, QoS, Task Scheduling

目 录

摘 要 ····· I
ABSTRACT ····· II
第 1 章 绪论 ····· 1
1.1 研究背景及意义 ····· 1
1.2 国内外研究现状 ····· 3
1.3 论文研究主要内容 ····· 6
1.4 论文组织结构 ····· 7
第 2 章 相关技术 ····· 9
2.1 本章引言 ····· 9
2.2 系统模型 ····· 9
2.2.1 实时系统模型 ····· 9
2.2.2 异构多核处理器 ····· 10
2.3 任务模型 ····· 12
2.3.1 任务模型分类 ····· 12
2.3.2 混合任务模型及服务质量 ····· 14
2.4 多核系统能耗管理技术 ····· 15
2.4.1 功耗模型 ····· 15
2.4.2 能耗管理及优化技术 ····· 16
2.5 服务质量感知任务调度算法 ····· 17
2.5.1 任务版本调度算法 ····· 17
2.5.2 能量自适应调度算法 ····· 18
2.5.3 引导搜索启发式算法 ····· 19
2.5.4 启发式近似感知调度算法 ····· 19
2.6 本章小结 ····· 20
第 3 章 基于服务质量感知混合任务调度近似最优算法 ····· 21
3.1 本章引言 ····· 21
3.2 平台模型 ····· 21
3.2.1 任务模型 ····· 21
3.2.2 处理器模型 ····· 21
3.2.3 功耗模型 ····· 23

3.3 问题定义 ····· 24
3.4 问题分析 ····· 24
3.4.1 任务分配约束 ····· 25
3.4.2 实时性约束 ····· 26
3.4.3 能耗约束 ····· 27
3.4.4 问题形式化定义 ····· 28
3.5 近似最优算法 ····· 28
3.5.1 问题线性化 ····· 28
3.5.2 问题分解 ····· 30
3.5.3 近似最优算法 ····· 33
3.6 仿真结果与分析 ····· 35
3.6.1 仿真参数设置 ····· 35
3.6.2 仿真结果分析 ····· 36
3.7 本章小结 ····· 42
第 4 章 基于服务质量感知混合任务调度启发式算法 ····· 44
4.1 本章引言 ····· 44
4.2 平台模型 ····· 44
4.2.1 任务模型 ····· 44
4.2.2 处理器模型与功耗模型 ····· 45
4.3 问题定义 ····· 45
4.4 问题分析 ····· 45
4.5 启发式算法 ····· 47
4.5.1 算法概述 ····· 47
4.5.2 任务强制部分分配 ····· 48
4.5.3 空闲时间优化 ····· 53
4.5.4 任务可选部分分配 ····· 56
4.6 仿真结果与分析 ····· 58
4.6.1 仿真参数设置 ····· 59
4.6.2 仿真结果分析 ····· 60
4.7 本章小结 ····· 66
第 5 章 总结与展望 ····· 67
5.1 主要工作与创新点 ····· 67
5.2 研究展望 ····· 68

参考文献 …… 69
作者简介 …… 75
1. 基本情况 ……75
2. 教育和工作经历 ……75
3. 攻读学位期间的研究成果 …… 75
3.1 申请（授权）专利 ……75
3.2 参与的科研项目及获奖 ……75
致 谢 ……76

第 1 章 绪论

1.1 研究背景及意义

随着物联网（Internet of Things, IoT）、工业 4.0 以及自动驾驶等技术的快速发展，人们对嵌入式系统的性能需求也在持续不断地增长[1]。嵌入式系统作为这些技术的核心支撑平台，其性能直接决定了上层应用的执行效率与可靠性。在嵌入式系统的核心架构中，作为核心部件的处理器对系统性能的提升起到了关键作用。然而，传统的单核处理器以及同构多核处理器由于其固有的架构限制和计算能力瓶颈，已经难以满足现代应用场景对处理器性能的需求[2]。例如，在自动驾驶系统中[3]，实时数据处理与决策、传感器数据融合以及路径规划等任务对计算能力的要求极高，传统的处理器架构在有限的时间内难以高效地处理这些复杂任务。在这一背景下，计算机技术和集成电路技术的进步推动了嵌入式系统处理器向集成化和异构化的方向演进，形成了异构多核处理器。

异构多核处理器与传统的单核及同构多核处理器相比，其独特之处在于其集成了多种功能各异的处理器核心，每种核心针对特定类型的计算任务进行了优化设计[4]。例如，基带处理器（Baseband Processing Unit, BPU）专为高效处理数据通信任务而设计，能够满足无线通信系统中对信号调制解调、信道编码解码等任务的高实时性要求；数字信号处理器（Digital Signal Processor, DSP）则专注于数字信号处理任务，如音频信号处理、图像滤波等，其高效的并行计算能力使其在实时信号处理领域具有显著优势；图像信号处理器（Image Signal Processor, ISP）专门用于处理相机摄像任务，能够高效完成图像去噪、色彩校正、自动曝光等复杂的图像处理操作；神经网络处理器（Neural-network Processing Unit, NPU）则针对机器学习算法进行了优化，能够高效执行深度学习模型的推理与训练任务，广泛应用于计算机视觉、自然语言处理等领域。这种多样化的处理器核心集成设计使得系统能够根据任务的计算特性以及任务对系统资源需求的差异性，灵活地选择合适的处理器核心来执行任务，实现计算资源的优化配置，进而推动系统性能的全面提升。得益于此优势，异构多核平台已在工业控制系统、自动驾驶系统、通信系统以及视频图像处理系统等领域得到广泛应用[5]。因此，异构多核平台的任务调度问题，已成为当下研究的重点课题。

尽管异构多核平台可以显著提升嵌入式系统的计算与处理能力，但这种复杂的系统架构同时也面临着能耗优化和时间管理这两个主要方面的挑战和冲突。一方面，嵌入式系统通常是部署在能源受限的计算平台上[6]，如移动设备与物联网终

端设备等，因此具有严格的能耗约束。这种能源限制不仅制约着系统的持续运行时间及可靠性水平，更直接影响着整体架构的能源利用效率与工程应用价值。在面向多核处理器架构的功耗优化研究中，动态电压与频率调节技术（Dynamic Voltage and Frequency Scaling, DVFS）作为一种核心能效控制机制，已成为实现系统级低功耗设计的关键技术[7]。研究[8]表明，通过动态调整处理单元的供电电压与时钟频率参数，可有效实现运算性能与能耗效率的动态平衡，从而满足嵌入式系统对功耗管理的严格要求。然而单一依赖基于硬件架构的功耗管理策略或软件层面的任务调度算法与资源分配策略，均存在显著的能效瓶颈问题。因此，将高效的任务调度算法与 DVFS 技术相结合的软硬件协同优化机制，具有重要的理论研究价值与实际应用意义。

另一方面，许多嵌入式应用要求系统具备实时响应能力[9]。在工业控制、无人驾驶、实时音视频传输等对实时性要求极高的场景中，系统必须能够在严格的时间限制内迅速处理数据，并做出相应的决策或执行动作。一旦应用程序执行时间超过其设定的截止期限，将可能对系统性能产生负面影响，导致任务执行失败的风险增加，甚至引发严重的后果。为了确保系统在能源受限的条件下仍能严格遵守实时性约束，文献[10]提出了采用非精确计算任务的方法，非精确计算任务从逻辑上可以分为强制部分和可选部分，其中任务强制部分必须在其截止期限前完成，任务可选部分则需根据系统可用资源选择性执行，以降低服务质量（Quality of Service, QoS）为代价确保系统能够在其截止期限前完成任务[11][12]。以 JPEG2000 编解码器为例[13]，在其编码器中，图像的质量损失主要来源于 Tier-2 速率的控制，而在解码器中，图像也可以通过逆离散小波变换（Inverse Discrete Wavelet Transform, IDWT）以不同分辨率形式输出，因此 JPEG2000 编解码器可以在资源有限的情况下得到低质量的图像。

然而在实际应用中，应用程序是由精确计算任务（或常规任务）和非精确计算任务组成的混合任务集。例如，图像处理中的图像特征的提取和分类需要使用精确计算，图像增强和去噪则可以使用非精确计算；深度学习中的关键层需要使用精确计算以确保预测结果的准确性，而中间层或辅助任务可以使用非精确计算来减少复杂度并加速推理。综上所述，针对此类混合任务集，如何高效地管理和调度多样化的处理器核和实现任务的合理分配与协同处理机制，以在能耗限制和实时性约束的情况下得到更高的服务质量是当前异构多核平台研究的关键问题。

基于上述研究背景，本文的核心研究聚焦于异构多核平台，旨在探索并实现一种基于服务质量感知的混合任务调度算法。该算法通过综合考虑能耗约束、实

时性要求以及任务特性，实现对异构多核资源的高效管理与任务优化调度，从而在满足系统约束的同时提升系统的服务质量。

1.2 国内外研究现状

由于异构多核处理器已成为多核处理器发展的主流方向，其实时系统任务调度问题也成为了计算机操作系统领域的研究热点。异构多核处理器系统中的任务调度问题已被证明是非确定性多项式（Non-deterministic Polynomial, NP）难问题[14]，这类问题在短时间内难以求解得到最优解。鉴于此，现有研究多采用启发式算法、引导搜索算法、线性规划或凸优化建模等方法，通过缩短求解时间获得次优解，这一结果在业界通常被认为是可接受的。从当前研究现状分析，可根据不同算法优化目标把各项研究分为以能耗为优化目标的调度算法和以提高服务质量为优化目标调度算法，下面将对其分别进行详细介绍。

（1）以能耗为优化目标

以能耗为优化目标的任务调度算法是在有限的系统资源约束下通过高效的任务分配和调度方法最大限度地减少能源消耗，以达到节能效果。

对于同构多核平台，CHEN 等人提出了一种结合 DVFS 和动态功耗管理（Dynamic Power Management, DPM）技术优化算法[15]，主要是通过其提出的基于执行窗口分析的细化技术确定哪些任务执行之间有可能构成空闲时间间隔，然后利用 DVFS 和 DPM 技术减少能耗，但其任务到处理器核的分配是固定的，没有考虑优化任务分配。Esmaili 等人则进一步考虑了任务分配的问题，引入了决策变量来表示任务到处理器的分配和任务在处理器上的顺序，将优化问题转化为混合整数线性规划，并基于此提出了一种两阶段启发式算法[16]。第一阶段使用列表调度算法确定任务到处理器的分配和任务在处理器上的顺序，第二阶段在固定这些变量后，求解简化后的问题，以获取任务的执行开始时间和其在不同频率下的工作量分布。

对于异构多核平台，任务调度问题增加了处理器核以及工作电压/频率等级的变量选择，这些变量通常以非线性方式相互耦合，这增加了解决问题的难度。LI 等人针对独立任务提出了简单取整算法（Relaxation-based Naive Rounding Algorithm, RNRA）和基于松弛的迭代取整算法（Relaxation-based Iterative Rounding Algorithm, RIRA）[17]。RIRA 首先将异构多核系统中的独立的基于帧的任务分配问题建模为二进制整数非线性规划问题，然后通过松弛处理将原问题转换为凸优化问题，并使用内点法求解。最后根据松弛后的最优解以及按照任务的平均执行周

期对任务进行排序的顺序表，优先分配对能耗影响最大的任务，更新优化问题并重复上述过程直到分配完所有任务。

LI 等人针对依赖任务提出了基于动态规划的加速搜索算法[18]，该算法包括三个阶段：最小能耗概率约束算法（Minimum Energy Under Probability Constraints, MEUPC）、叶分区算法（Leaf-Partition, LP）以及时间换能耗算法（Trading Energy For Time, TEFT）。该算法首先在 MEUPC 中计算任务最小和最大能耗以确定搜索范围，然后通过动态规划算法构建初始分配表，选择具有最少概率为 1.0 的任务的方案作为初始最优任务分配方案。然后在 LP 中根据初始最优任务分配方案和任务图中的优先关系，构建每个处理器核上的执行队列。最后在 TEFT 中遍历任务表，为每个任务找到能耗增量最小且时间更短的新方案，根据能耗差异对任务进行排序，逐步更新任务方案，直到满足时间约束。

上述以能耗为优化目标的调度算法主要是针对精确计算任务，其执行周期为固定值，导致系统服务质量的提升受到限制。由于精确计算任务的计算量不可调节，算法在优化过程中无法通过动态调整任务执行量来平衡能耗与服务质量，从而难以在满足能耗约束的同时进一步提升系统服务质量。

（2）以提高服务质量为优化目标

以提高服务质量为优化目标的任务调度算法是在能耗约束以及实时性约束下通过高效的任务分配和调度算法调整非精确计算任务中可选部分的执行量，以实现系统服务质量的最大化。

对于同构多核平台，MO 等人提出了一种最大化服务质量任务映射近似最优算法[19]，该算法将任务映射原始问题通过问题转化以及对偶分解，引入拉格朗日乘子并构建拉格朗日函数，将问题分解为关于任务分配和执行时间的独立子问题。

Kim 等人提出了一种基于奖励机制的启发式算法优化算法[20]，以最大化可选部分的执行奖励和降低处理器频率带来的能耗奖励。该算法分为两个阶段，第一阶段使用贪婪启发式方法，将任务分配到处理器核上，确保强制性部分满足实时性约束；第二阶段在每个处理器核上，使用整数线性规划模型进一步优化可选部分的执行和频率选择，以最大化奖励和降低能耗。

但以上任务调度方法，主要针对独立任务，对于依赖任务，Ravindran 等人提出一种离线-在线混合调度启发式算法[21]。该算法在离线阶段采用模拟退火算法将任务分配到处理器核上，确保每个任务的强制部分均满足其执行需求，并通过 DVFS 技术为任务分配合适的工作电压/频率，以优化系统能耗。在线阶段，若任务提前完成，其释放的执行时间将被回收，并根据任务的优先级将回收的资源重新分配给其他任务，从而进一步提升资源利用率与系统服务质量。

对于异构多核平台，WEI 等人研究了非精确计算任务在异构多核处理平台上的分配问题[22]，但是其所提出的调度方案均是按照固定的工作电压/频率来执行任务，且未考虑实时性这一约束条件。

ZHOU 等人提出了一种三阶段启发式算法[23]：1）确定性任务分配算法（Deterministic Task Allocation, DTA）；2）能量自适应任务分配算法（Adaptive Task Allocation, ATA）；3）服务质量驱动的任务调度算法（Task Selection for QoS Optimization, TSQO）。该启发式算法首先通过 DTA 按处理器功耗因子对处理器进行升序排列，以及按任务功耗因子对任务进行降序排列，根据排列顺序在满足截止期限条件下将任务分配到合适的处理器，以最小化能耗总和。然后通过 DTA 算法判断系统能量状态，并据此选择可选部分执行量。最后通过 TSQO 算法计算每个任务的可选部分每周期能耗，并按此升序排列任务，优先执行排序靠前的任务，遍历排序后的任务，尽可能最大化系统服务质量。但该启发式算法未考虑任务迁移以及工作电压/频率调整对任务分配和调度的影响。

YU 等人提出了一种服务质量驱动的动态调度方案[24]。该方案通过优化运行时的时间松弛分配最大化系统服务质量。针对多处理器核环境下时间松弛分布与能耗约束的复杂性，该研究提出了一种启发式引导搜索算法，动态选择各任务的处理器核的最优工作电压/频率；同时设计了一种基于图分解的松弛接收器选择策略，用于识别具有最大服务质量提升潜力的任务子集，从而进一步优化资源分配效率。此外，研究进一步扩展了框架以处理处理器核间通信引发的时延不确定性，提出通过 DVFS 进行本地缩放的方法补偿通信波动导致的质量损失。然而该研究同样未考虑任务迁移对系统性能的影响。

MO 等人提出了一种基于 Benders 分解的服务质量近似最优求解算法[25]。该算法将原问题划分为主问题和子问题，主问题用于求解任务到处理器核的分配以及任务的调度顺序，而子问题用于求解可选子任务执行量，通过主问题与子问题的交替迭代求解，逐步更新约束条件，最终逼近问题的最优解。然而，该算法在任务分配方案中假设处理器核是以固定的工作电压/频率来执行任务，并未考虑 DVFS 对系统能耗的影响。

LI 等人提出一种基于 ARM big. LITTLE 架构平台的非精确计算任务调度多阶段启发式算法[26]。该算法分为频率分配、任务分配和调度及可选周期调整三个阶段。首先，在频率分配阶段，算法基于 DVFS 技术为每个强制子任务确定最优执行频率，以最小化其执行所需能耗，同时满足实时性约束与能耗约束。其次，在任务分配与调度阶段，基于频率分配的结果，算法通过负载均衡策略将任务分配到处理器核上，并确定任务的开始时间，以确保系统资源的合理利用。最后，在

可选周期调整阶段，算法利用处理器的空闲时间间隔与剩余能量，动态调整可选子任务的执行量，以进一步提升系统服务质量。然而，该算法的设计基于 ARM big.LITTLE 平台，其架构特性限制了算法在通用异构多核处理器模型中的适用性。

现有文献关于服务质量感知的任务调度算法主要集中于单一任务模型，而少有研究同时考虑精确计算任务和非精确计算任务并存的混合任务模型。此外，现有算法大多未能综合考虑任务分配、任务调度及任务迁移的协同优化，且缺乏对 DVFS 技术的有效使用，尤其是在通用异构多核平台上的应用研究较为有限。因此，本文旨在提出一种适用于通用异构多核平台下的基于服务质量感知的混合任务调度算法。该算法充分考虑了任务分配、任务调度及任务迁移的优化策略，并结合 DVFS 技术，从近似最优算法和启发式算法两个维度解决混合任务调度问题。

1.3 论文研究主要内容

本文主要研究异构多核平台下基于服务质量感知的任务调度问题，以解决在系统资源约束与实时性约束下的服务质量优化问题。研究内容涵盖了异构多核系统模型的构建、混合任务模型的分析、功耗模型的建立以及服务质量的定义。通过系统性地探讨上述问题，本文提出一种近似最优算法和一种高效的启发式任务调度策略，以在满足多重约束条件的同时实现系统服务质量的最大化。具体研究内容如下：

（1）现有研究主要集中于常规任务或非精确计算任务的调度问题，少有研究同时考虑两类任务并存的混合任务模型。因此，本文基于通用异构多核平台，针对此类混合任务模型，综合考虑能耗约束、实时性约束以及任务依赖约束等多个约束条件，以最大化系统服务质量为目标，构建了相应的任务调度问题模型。为降低原问题的计算复杂度，本文根据原问题的数学形式，通过线性化方法与 Benders 分解方法将原问题分解为一个主问题和一个子问题。主问题负责优化整数变量，而子问题则以主问题的解为参数优化连续变量。

（2）基于分解后的问题结构，提出了一种基于服务质量感知的混合任务调度近似最优算法，该算法通过主问题与子问题的交替迭代求解，利用 YALMIP 和 CPLEX 求解器进行实现，逐步逼近原问题的最优解。该算法不仅有效降低了原问题的求解难度，还通过动态调整任务分配与资源调度策略，实现了系统服务质量的最大化，为后续启发式算法提供了任务调度的理论上界。

（3）考虑到前文提出的近似最优算法在中大规模任务集下求解时间较长且工程实现复杂度较高的情况，本文提出了一种基于服务质量感知的混合任务调度启发式算法。该算法采用多阶段启发式策略，分为任务强制部分分配、空闲时间优

化以及任务可选部分分配三阶段。第一阶段根据任务分层结构及强制子任务计算量（常规任务只有强制部分），确定强制子任务分配优先级，在不超过系统为强制部分预留能耗的前提下，通过优化分配策略最小化强制子任务的总能耗，从而生成初步的任务分配方案。第二阶段则是基于第一阶段的初步分配方案，通过 DVFS 技术充分利用强制部分预留的能耗。具体而言，通过调整处理器核的工作频率与电压，最大化处理器上可用的空闲时间，为可选子任务的分配提供更多可用资源。在第三阶段，每个可选子任务遍历其所有可用处理器核上空闲时间区间，寻找能够最大化其执行量的分配方案，该阶段的目标是在满足能耗与实时性约束的前提下，尽可能提升系统的服务质量。本文提出的启发式算法通过分阶段优化策略，降低了中大规模任务集的求解时间，同时提高了算法的工程可实现性。

1.4 论文组织结构

本文分为五个章节，研究如何使用混合任务调度算法的方式提升异构多核平台的服务质量。论文的安排结构如下：

第一章，绪论。本章首先阐述了异构多核平台上混合任务调度研究的背景及意义，分析了当前嵌入式系统在实时性、能效等方面面临的挑战。其次对任务调度算法的国内外研究现状进行了综述，随后重点介绍了本文的主要研究内容，并简要说明了本文的组织结构。

第二章，相关技术。本章围绕任务调度的关键技术从系统模型、任务模型、功耗模型及低功耗技术、任务调度算法四个方面展开详细探讨。具体而言，本章首先分析了任务模型的基本理论与分类，其次深入研究了功耗模型的构建方法及典型低功耗技术的应用策略。最后，着重讨论了本文研究相关的任务调度算法，为后续研究奠定理论基础并提供技术支持。

第三章，基于服务质量感知的混合任务调度近似最优算法。本章介绍了本文提出的异构多核平台下基于服务质量感知的混合任务近似最优调度算法。首先详细描述了该算法采用的任务模型、处理器模型以及功耗模型，并明确了优化目标。其次，综合考虑能耗约束、实时性约束、任务依赖约束等相关约束，构建了混合任务调度的优化问题。为降低问题的求解难度，通过线性化和 Benders 分解方法将原问题分解为一个主问题和一个子问题，并基于分解后的结构提出一种近似最优算法。最后进行仿真分析，验证所提算法性能。

第四章，基于服务质量感知的混合任务调度启发式算法。本章介绍了本文提出的异构多核平台下基于服务质量感知的混合任务调度启发式算法。首先对该算法采用的任务模型、处理器模型以及功耗模型进行了说明。其次，通过分析原问

题提出一种多阶段启发式算法，详细阐述了任务强制部分分配、空闲时间优化及任务可选部分分配三个阶段的具体流程。该算法通过分阶段优化策略，进一步降低了求解难度和计算时间。最后通过仿真分析，验证所提算法的性能表现。

第五章全面总结了本文的工作内容，同时指出了当前研究的局限性，并对未来的研究方向进行了展望。

第 2 章 相关技术

2.1 本章引言

本章主要介绍异构多核平台下任务调度研究中的相关技术概念，以及本文研究需要对比的服务质量感知的任务调度算法，为后续章节中所提出算法性能的验证做了铺垫。

本章结构如下：2.2 节从实时系统模型和异构多核处理器两方面介绍了系统模型的相关概念；2.3 节介绍了当前研究中常用的系统调度模型，以及混合任务模型中非精确计算任务的概念和服务质量的定义；2.4 节介绍了异构多核平台所采用的功耗模型和低功耗技术；2.5 节介绍了本文研究所提出算法需要对比的任务调度算法；2.6 节对本章内容进行总结。

2.2 系统模型

2.2.1 实时系统模型

实时系统是一类对任务执行时间具有严格约束的计算系统[27]，其任务执行的正确性不仅取决于计算结果的准确性，还依赖于任务是否能在预设的时间范围内完成[28]。嵌入式系统作为实时系统的重要实现载体，其硬件资源受限的特性与实时性要求形成了紧密的耦合关系，这种关系在硬实时系统中表现得尤为突出。实时系统通常根据任务执行过程中对时间约束的违反程度及其造成后果可以分为以下 3 类：

（1）硬实时系统：在此类系统中，所有任务的执行必须严格满足其截止期限要求，任何超出截止期限的行为都将导致系统功能失效或灾难性后果，甚至引发系统崩溃等不可逆的错误[29]。这类系统在安全关键型领域具有广泛应用，如自动驾驶系统中的紧急制动决策模块必须在毫秒级时间内完成感知-决策-执行的全过程；在航空航天领域，飞行控制系统和导弹制导系统等都需满足硬实时约束，以确保系统的可靠性和安全性。

（2）软实时系统：此类系统有一定的时间容错性，允许任务在有限范围内超出截止期限，偶尔的实时性违反不会对系统功能产生严重影响，系统的输出质量与任务完成时间呈负相关关系，即任务完成得越及时，其输出结果价值越高[30]。软实时系统广泛应用于日常生活场景中，包括车票预订、信息查询等，都建立在

软实时系统基础上。这类系统对数据传输延迟有一定容忍度，短暂的响应延迟虽不会导致功能失效，但会影响用户感受。

（3）准实时系统：此类系统处于硬实时和软实时系统之间，其任务的执行时间可以超过截止期限，任务执行在超过截止期限后虽然不会引发系统错误，但将完全丧失其原本的执行价值[32]。例如，在视频传输任务中，若某一帧未能在规定时间内完成传输，则该帧将被丢弃，可能导致视频播放的短暂卡顿，但不会中断整个传输过程。

本研究重点关注基于硬实时系统，这类系统对任务的时间约束具有零容忍特性，任何时间约束的违反都将导致系统功能异常或失效。

2.2.2 异构多核处理器

多核处理器是在一块单独的芯片上集成了多个处理器核心[31]，每个核心都可以独立执行指令，处理数据，类似于多个单核处理器并行工作。处理器核间通过集成总线进行通信，不仅能够传递数据，还能共同处理并行线程计算。根据芯片上集成的核心架构类型异同，多核处理器可以分为同构多核和异构多核两类[32]。

同构多核芯片的特征在于其集成了多个性能均衡且架构对称的处理器核心，又被命名为对称多核处理器，其设计原理相对简单，实现难度较低[33]。该架构的历史可追溯至 1996 年，当时美国斯坦福大学研究团队成功研制出 Hydra 处理器[34][35]，这款 4 核处理器是典型的同构多核处理器原型。如图 2-1 所示，其主要架构是由 4 个通用处理器核组成，4 个核心无论是在其内部结构、功能以及架构中的所在位置都是一致的。

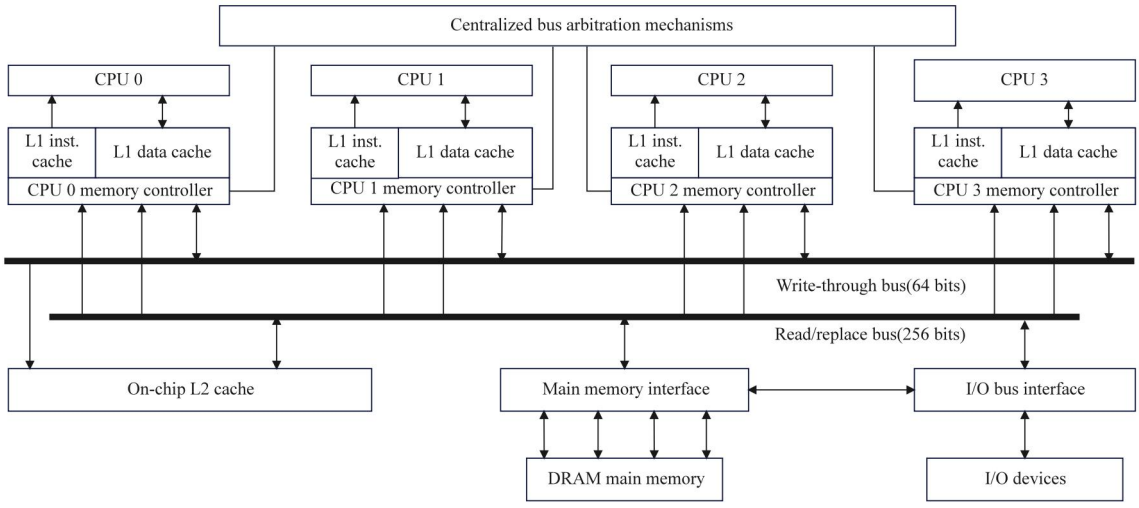


图 2-1 Hydra 处理器架构

Fig.2-1 Hydra processor architecture

异构多核处理器的特征在于其集成了多种在架构、指令集、性能特征等方面存在差异的处理器核心，这种设计又被称为非对称多核处理器。异构多核处理器通过结合不同类型的处理核心，能够为不同类型的任务提供优化处理，在提高处理效率的同时，实现能源效率的优化和功耗的降低。当前，以 CPU-GPU 为主要形式[36]的异构多核处理器架构，包括 ARM big. LITTLE 架构[37]、CELL 处理器以及三星 Samsung Exynos 9 系列处理器等。其中最典型的代表处理器是由 SONY、Toshiba 和 IBM 联合开发的 CELL 处理器[38]，该处理器在异构多核技术发展史上具有开创性意义，其是首款成功实现商业化的异构多核处理器。如图 2-2 所示，CELL 处理器主要架构由 8 个 32 位的协同处理器元件（Synergistic Processor Element, SPE）和一个 64 位的电源处理器元件（Power Processor Element, PPE）构成，这些组件通过高速总线（Element Interconnect Bus, EIB）在 CPU 内部进行数据交互与通信, 实现高效互联。其中作为主处理器核心的 PPE 采用双通道线程架构与发射端设计，其指令执行过程遵循顺序处理原则。该核心具备双结果并行获取能力，可同时处理四条指令，承担操作系统全局调度及 SPE 协调功能，相当于整个系统架构的中枢。相比之下，SPE 作为协处理核心，在架构特征、功能定位及系统层级等方面均与主处理核心存在显著差异，其主要承担基础浮点运算任务以及确保程序稳定执行，从而使 CELL 处理器在高效能与低功耗方面达到优异平衡。

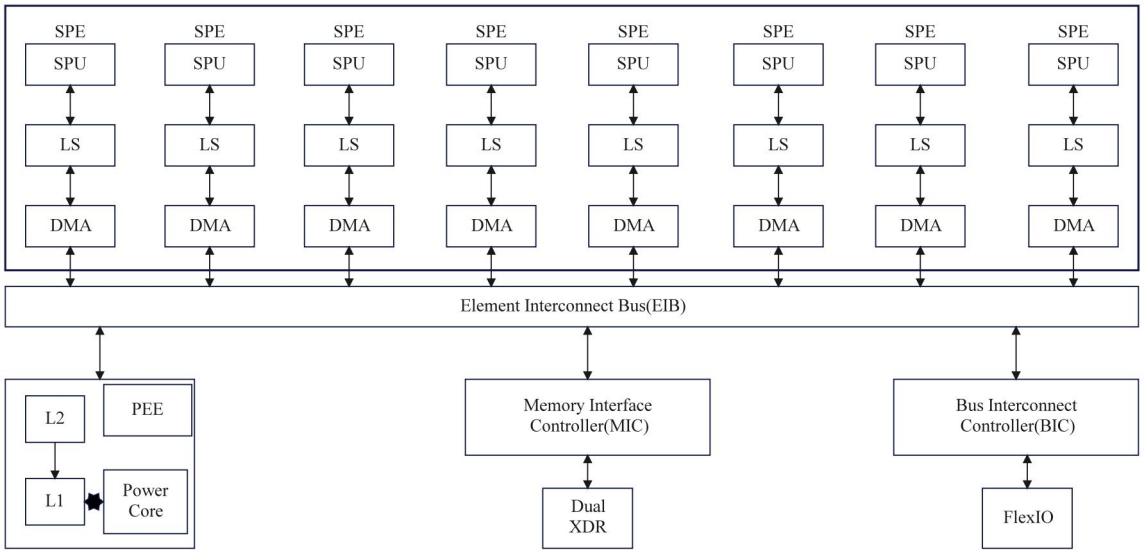


图 2-2 CELL 处理器架构

Fig.2-2 CELL processor architecture

根据前文所述，异构多核处理器由功能各异的核心组成，每个核心均具备特定的功能定位。CELL 处理器就是采用主从架构设计，主核与从核在功能、地位及结构上均存在显著差异。通过任务类型的合理分配，可充分发挥各个核心的功能，

实现核心性能得到最大释放，从而提升系统整体的工作效率并优化其功耗表现。然而异构多核处理器的复杂结构、开发难度以及负载均衡问题限制了系统性能的进一步优化。因此，除了处理器本身的高性能特性外，高效的任务调度算法是实现性能突破的关键，它不仅能优化任务分配、缩短调度时间，更能充分发挥处理器的并行计算能力。本文的研究内容是通过优化任务调度策略，提升任务执行过程中的并行性，同时减少系统资源浪费，从而提升异构多核平台整体性能。

2.3 任务模型

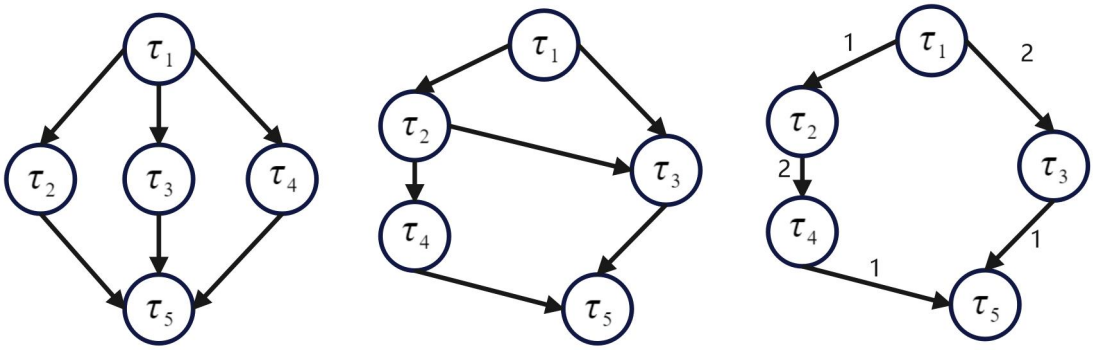
2.3.1 任务模型分类

在任务调度研究中，常用的任务模型主要包括以下几种类型[39]：

任务交互图（Task Interaction Graph, TIG）[40][41]是一种用于描述并行计算环境中任务间关联关系的模型。如图 2-3(a)所示，TIG 模型以节点表示任务，边表示任务间的交互关系。在并行计算领域，任务间的交互模式对系统性能与效率起着关键作用，而 TIG 模型为分析和优化此类交互提供了有效手段。通过构建此类交互图，能够直观呈现任务间的依赖关系及数据传输路径，从而为提升系统并行性能的研究提供重要支持。

任务优先图（Task Priority Graph, TPG）[42]是一种用于描述任务优先级关系的模型。如图 2-3(b)所示，TPG 模型以节点表示任务，有向边则用于表示任务间的优先级依赖关系。通过构建 TPG 模型，可以清晰地定义任务的执行顺序，确保高优先级任务先得到处理，从而提升系统响应速度与运行效率。研究表明，在任务调度及系统资源分配中，任务的执行顺序及优先级分配对系统性能具有显著影响，而 TPG 模型为此类问题的分析与优化提供了有力支撑[43]。

时态任务交互图（Temporal Task Interaction Graph, TTIG）[44]是一种用于描述任务间时间关联关系的模型，属于 TPG 模型的改进模型。如图 2-3(c)所示，该模型不仅描述了任务间的依赖关系与通信需求，还引入了时间属性，包括任务的执行时间、开始时间、截止时间以及任务间的时间约束。在实际系统中，任务的执行顺序及其时间关系对系统性能具有显著影响，而 TTIG 模型为分析与优化任务间的时间交互带来了帮助。通过构建 TTIG 模型，能够明确定义任务的执行时间顺序，确保任务按时执行，从而提升系统的时效性与运行效率。



(a) TIG (b) TPG (c) TTIG

图 2-3 任务模型

Fig.2-3 Task model

有向无环图（Directed Acyclic Graph, DAG）[45][46]是目前异构多核平台下研究任务调度中使用最为广泛的一个模型，本文针对异构多核平台的任务调度研究的方案设计和实施也均基于 DAG 模型实现。如图 2-4 所示，DAG 模型同样包含节点与有向边，分别用于表示任务与任务间的依赖关系。DAG 模型可表示为 $G=<N,C,Q>$，其中 N 表示图中节点数量，即任务数量，使用 τ_i 表示第 i 个任务。C 表示有向边上的权值的集合，即任务间的数据通信时间开销集合，如图 2-4 任务 τ_1 和任务 τ_6 之间的有向边的权值 $c_{1,6}=14$ 表示这两个任务间的数据通信开销。

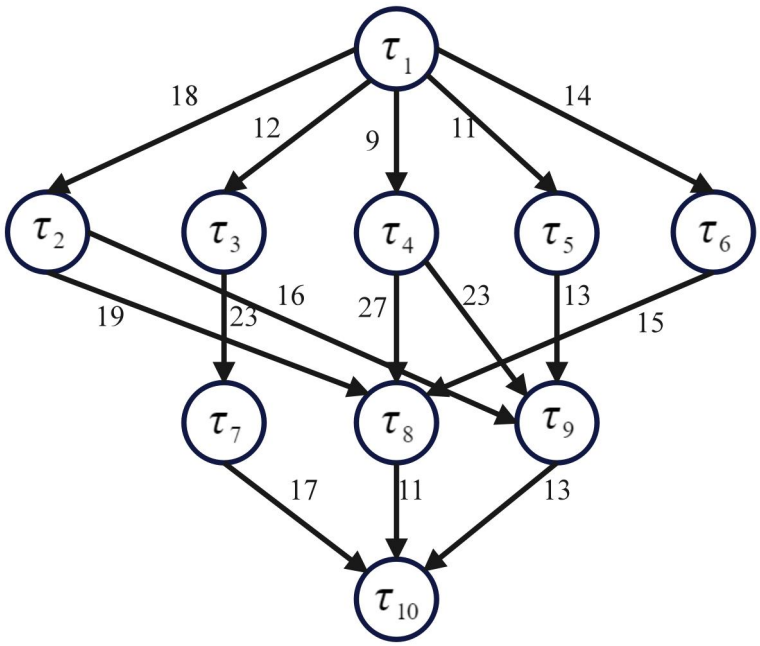


图 2-4 有向无环图示例

Fig.2-4 Example of DAG

Q表示有向边的集合，如图 2-4 任务τ_1和任务τ_2之间的有向边可以表示为$q_{1,2}=1$，任务τ_5和任务τ_9之间的有向边可以表示为$q_{5,9}=1$。DAG 模型通常使用集合$pred(\tau_i)$表示任务τ_i的直接前驱任务组，如图 2-4 中任务τ_8的直接前驱任务组$pred(\tau_8)=\{\tau_2,\tau_4,\tau_6\}$；相应也会使用集合$succ(\tau_i)$表示任务$\tau_i$的直接后继任务组，如图 2-4 中任务$\tau_1$的直接后继任务组$succ(\tau_1)=\{\tau_2,\tau_3,\tau_4,\tau_5,\tau_6\}$。当$pred(\tau_i)$为空时，即任务$\tau_i$没有直接前驱任务，则称此类任务为入口任务，使用$\tau_{entry}$表示，如图 2-4 中任务$\tau_1$为该任务集的入口任务；当$succ(\tau_i)$为空时，即任务$\tau_i$没有直接后继任务，则称此类任务为出口任务，使用$\tau_{exit}$表示如图 2-4 中任务$\tau_{10}$为该任务集的出口任务。

2.3.2 混合任务模型及服务质量

现有的任务调度算法的研究主要集中于两种类型：一类是基于能耗感知的精确计算任务调度，另一类是基于服务质量感知的非精确计算任务调度。然而少有研究考虑两类任务并存的混合任务模型。

在异构多核平台上，非精确计算任务是一种旨在平衡系统实时性、资源利用率与服务质量的任务模型。在该模型中，每个任务τ_i由强制部分M_i和可选部分O_i两个部分组成[47][48]。其中，强制部分M_i是指任务中必须在截止期限之前执行完成的关键计算部分，其完成是任务成功执行的必要条件，任何延迟都可能导致任务失败或系统错误。相比之下，任务的可选部分O_i则属于任务中可选择执行的部分，通常具有一定的灵活性，其执行主要用于提升任务的服务质量，但并非任务执行成功的必要条件。根据系统的可用资源状况，可选部分可能会被部分执行或完全跳过，以节省系统资源和执行时间。尽管这种处理方式会在一定程度上降低任务的服务质量，但通过这种方式，系统可以确保在有限的资源和时间约束下，优先完成强制部分，从而最大程度地保证任务在截止期限内的完成。通过这种任务分层机制，系统能够在资源紧张或负载较高的情况下动态调整任务执行策略，权衡性能与服务质量，以确保任务的及时完成和系统的稳定性。

对于混合任务模型中的任务τ_i可以使用$\Gamma=<M_i,O_i,\alpha_i>$来表示，其中$M_i$是任务$\tau_i$的强制部分，$O_i$是任务$\tau_i$的可选部分（精确计算任务可选部分$O_i$需全部执行），$\alpha_i$是任务$\tau_i$在所有处理器上的执行效率的集合，由于异构多核平台的特性，任务τ_i在不同处理器上的执行效率也会不同[49][50]。若τ_i为非精确计算任务，其可分为两个具有依赖关系子任务：强制子任务$\tau_{i,M}$和可选子任务$\tau_{i,o}$，因此，混合任务模型使用 DAG 模型可表示为如图 2-5 所示，其中τ_1、τ_5和τ_7为精确计算任务，而τ_2、τ_3、τ_4和τ_6为非精确计算任务。

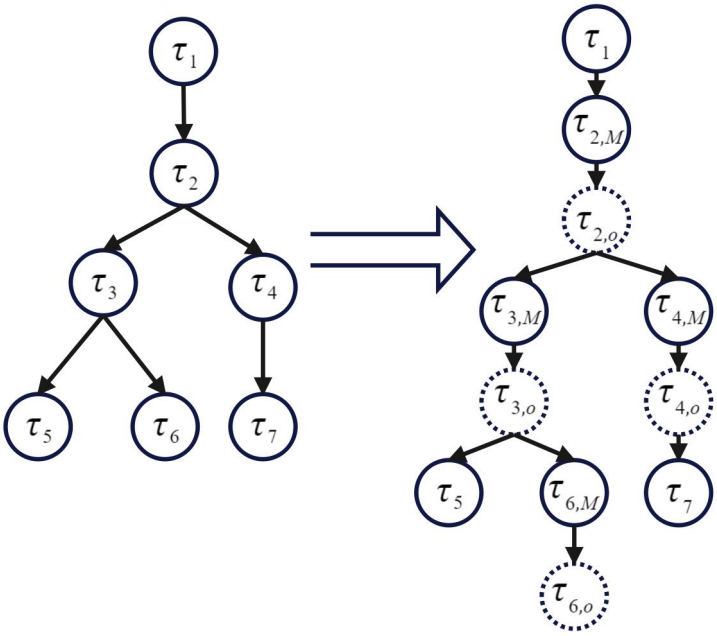


图 2-5 混合任务模型示例

Fig.2-5 Example of hybrid task model

由于系统资源和负载情况直接影响任务的可选部分执行长度，所以其实际执行长度可根据系统负载动态调整。因此，引入一个变量o_i来代表任务τ_i可选部分的实际执行长度，o_i的取值范围为$0 \le o_i \le O_i$，其中，O_i为任务τ_i的可选部分的最大执行长度。基于此，任务的实际执行长度可表示为：

$$L_i = M_i + o_i \tag{2-1}$$

系统服务质量与任务可选部分密切相关，系统应该尽可能多地执行任务的可选部分以实现系统服务质量的最大化。系统服务质量与任务可选部分之间的关系可以表示为线性函数[51]，其能充分模拟许多应用领域的行为，例如图像和语音处理、控制工程和自动目标识别。因此，本文将系统服务质量定义为系统中所有非精确计算任务的可选部分的实际执行长度之和的函数，具体表示为：

$$Q(o) = \sum_{i=1}^{N} o_i \tag{2-2}$$

2.4 多核系统能耗管理技术

2.4.1 功耗模型

在嵌入式系统功耗研究中，处理器的工作模式可以被分为忙（busy）模式、空闲（idle）模式和睡眠（sleep）模式 3 个工作模式[52]：busy 模式，也被称为激活（active）

模式或运行模式，表示系统正在处理任务；idle 模式，表示系统已启动，但没有执行任何任务；sleep 模式，也被称为关闭（shutdown）模式或休眠（dormant）模式，表示系统当前没有任务要运行，处于一个功率极低的模式，当需要执行新任务时系统会切换至 busy 模式。具体的功耗模型采用已通过 SPICE 仿真验证的功耗通用模型[53]，在该模型中，系统的功耗由动态功耗 P_{dyn}、静态功耗 P_{sta} 和固有功耗 P_{on} 三个部分构成[54]：

（1）动态功耗 P_{dyn} 主要源自于时钟网络，由器件上电情况下的电路反转引起，并受供电电压及时钟频率影响，动态功耗 P_{dyn} 在给定电压/频率的前提下可表示为：

$$P_{dyn} = C_{eff} V_{dd}^{\ 2} f \tag{2-3}$$

上式(2-3)中 C_{eff} 表示有效切换电容，V_{dd} 表示处理器核的供电电压，f 表示处理器核的时钟频率，其可进一步表示为：

$$f = (V_{dd} - V_{th})^{\alpha} / L_d K_6 \tag{2-4}$$

上式(2-4)中，α 和 K_6 表示技术参数常量，L_d 是由处理器的最长逻辑深度决定，V_{th} 表示门限电压[2]。门限电压 V_{th} 可进一步表示为：

$$V_{th} = V_{th1} - K_1 V_{dd} - K_2 V_{bs} \tag{2-5}$$

上式(2-5)中，V_{th1}、K_1、K_2 表示工艺技术参数常数，V_{bs} 表示体偏压，均与芯片工艺相关[2]。

（2）静态功耗 P_{sta}，也被称为漏电功耗，源于漏电流的产生，只要电路中有能量流动，漏电流就会存在，因此，静态功耗必然存在。静态功耗 P_{sta} 可表示为：

$$P_{sta} = L_g (V_{dd} I_{sub} + |V_{bs}| I_j) \tag{2-6}$$

上式(2-6)中，I_{sub} 表示漏电流，I_j 表示反偏节电流，L_g 表示芯片中存在的晶体管数量。

（3）固有功耗 P_{on} 是处理器在开启状态下产生的基础能耗，主要由芯片 I/O 系统和锁相环 PLL 等芯片外设组件引起。相较于动态功耗 P_{dyn} 和静态功耗 P_{sta}，固有功耗 P_{on} 的数值通常较低，但其在处理器处于 idle 模式时仍持续存在。由于不同技术架构和制造工艺的差异性，固有功耗的数值也是不同的，因此部分功耗管理研究通常将其设置为常量以简化分析[55]。

2.4.2 能耗管理及优化技术

在异构多核平台上，随着系统运行时间和任务状态的变化，系统的工作负载会发生变化。因此为了节能，可以根据系统的负载情况以及所执行任务状态的变

化将处理器切换到 idle/sleep 工作模式，或通过调节处理器的工作电压和频率来实现。鉴于此，异构多核功耗管理技术可分为两类：DPM 和 DVFS 技术[2]。

DPM 技术，即动态功耗管理技术，是一种通过动态调整系统或处理器的工作状态来降低功耗的方法。其核心思想是根据系统当前的负载情况，实时调整系统或处理器的工作模式，以降低静态功耗。由 2.4.1 节可知，处理器有三种工作模式：busy、idle、sleep。若系统没有任务执行或任务完成执行后，系统将会切换到 idle 或者 sleep 模式；若系统正在处理任务时，系统将会切换到 busy 模式，此时为全功率模式。由于 Idle 状态和 sleep 状态的切换有一定的时间开销和能量消耗[2]，DPM 技术将会在系统处于长时间 idle 模式时切换至 sleep 模式，此时为低功率模式，从而实现节能的效果。为保证 DPM 技术下这种模式的切换是有效的，切换的时机需要满足 1）系统切换模式带来的延迟不会超过任务的截止期限；2）系统切换模式带来的额外能耗小于系统在 sleep 模式下节约的能耗。

DVFS 技术，即动态电压频率调节技术，是一种通过动态调整处理器的工作电压和频率来降低功耗的方法[56]。其核心思想是根据系统当前的负载需求，实时调整处理器的运行频率和电压，从而在满足性能要求的同时减少能量消耗。由公式(2-3)可知，处理器的功耗与时钟频率和电压的平方呈正相关，因此系统可以通过降低处理器时钟频率，以延长任务的执行时间为代价换来更低的功耗。时钟频率降低的幅度取决于任务的实时性约束条件，即任务的执行时间不得超过其截止期限。

通过对 DPM 和 DVFS 的技术原理进行对比分析，可以看出 DPM 与 DVFS 在应用场景和技术实现上存在显著差异。DPM 技术是通过将系统或部分硬件模块切换到 sleep 模式来减少静态功耗，更适合负载较低或空闲的场景；DVFS 技术则通过实时调节芯片的电压和频率来减少动态功耗，适用于负载频繁变化的场景。

2.5 服务质量感知任务调度算法

2.5.1 任务版本调度算法

任务版本调度算法（Task Version Scheduling Algorithm, TVS）是一种面向异构多核平台的实时非精确计算任务调度算法[57]，其目标是在满足系统总功率限制、任务依赖关系及截止时间的约束下，通过整数线性规划最大化系统服务质量。该算法采用 DAG 模型构建依赖任务集，其中非精确计算任务根据可选部分执行量分为多个版本。如表 2-1 所示，任务 τ_1 只有 1 个可执行版本；任务 τ_2 有 3 个可执行

版本，每个版本强制部分周期相同，可选部分周期分别为 5、7 及 10；任务 τ_3 有 3 个可执行版本，每个版本强制部分周期相同，可选部分周期分别为 5、8 及 10。

表 2-1 任务版本

Table 2-1 Task Version

任务	M_i	O_i
τ_1^1	4	2
τ_2^1	20	5
τ_2^2	20	7
τ_2^3	20	10
τ_3^1	20	5
τ_3^2	20	8
τ_3^3	20	10

该算法定义二进制变量 $Z_{iklt\eta}$，若 $Z_{iklt\eta}=1$ 则表示任务 τ_i 的第 k 个版本在处理器核 θ_η 上以工作频率 f_l 在时间 t 开始执行；否则为 0。同时引入以下约束：1）唯一执行时间约束：每个任务必须唯一确定其开始时间、任务版本、工作电压/频率级别及处理器；2）资源约束：每个处理器在任何时刻最多执行一个任务；3）任务依赖约束：任务 τ_i 必须在所有前驱任务完成后开始；截止时间约束：整个任务图必须在共同截止时间 D 内完成；4）功率约束：系统总功率不得超过预算。

通过建立目标函数最大化系统服务质量，即所有任务的实际执行可选部分的总和，构造整数线性规划问题，并使用 CPLEX 求解器进行求解，生成调度表，包含任务到处理器核的映射、起始时间、工作电压/频率等级及任务版本。

2.5.2 能量自适应调度算法

能量自适应调度算法（Energy Adaptive Scheduling Algorithm, EAS）[23]是在异构多核平台下针对非精确计算任务的分配与调度算法，其大致可分为三个阶段：1）首先将处理器核 θ_j 按能耗贡献因子 $A_j=C_{eff}V_j^2$ 升序排列，将任务 τ_i 按能耗贡献因子 $k_i=\mu_i L_i$ 降序排列，其中 μ_i 表示任务的执行效率，基于贪心算法将高 k_i 分配至低 A_j 处理器核，确保时间约束；2）计算任务能耗特性 $EC_i=P(\theta_j,\tau_i)/f_j$，其中 $P(\theta_j,\tau_i)$ 表示任务 τ_i 在处理器核 θ_j 的执行功率，然后按 EC_i 升序排列任务；3）计算剩余能量，基于贪心算法优先执行 EC_i 最小的任务可选部分，并更新剩余能量，直到确定所有任务可选部分执行长度，以实现系统服务质量的最大化。

2.5.3 引导搜索启发式算法

启发式引导搜索算法（Guided-Search Heuristic Algorithm, GHS）[24]是一种面向异构多核平台的实时非精确计算任务调度算法，其目标是在满足系统能耗限制及实时性约束下，通过低复杂度的启发式算法最大化系统服务质量。该算法采用 DAG 模型构建依赖任务集，初始将所有任务工作频率设置为最高频率，然后根据以下两种情况进行预调整：1）能量不足：若总能耗超过约束，逐步降低任务的工作频率，直到满足能量约束；2）时间不足：若时间不足，则提高频率以缩短执行时间。最后进行循环优化：首先确定任务服务质量提升潜力，选择服务质量梯度最大且频率调整能耗差最小的任务；然后进行频率调整，降低频率以节省能量，或提高频率以释放时间，直到任务达到最大周期限制，或者所有频率调整完毕。

2.5.4 启发式近似感知调度算法

启发式近似感知调度算法（Approximate-Aware Heuristic Scheduling Algorithm, AHS）[26]是一种面向异构多核平台的实时非精确计算任务调度算法。该算法主要针对 ARM big. LITTLE 架构平台，其平台架构如图 2-6 所示，其由两种不同性能的集群组成，分别为 big 集群和 LITTLE 集群，但是同一集群中的处理器是同构的。

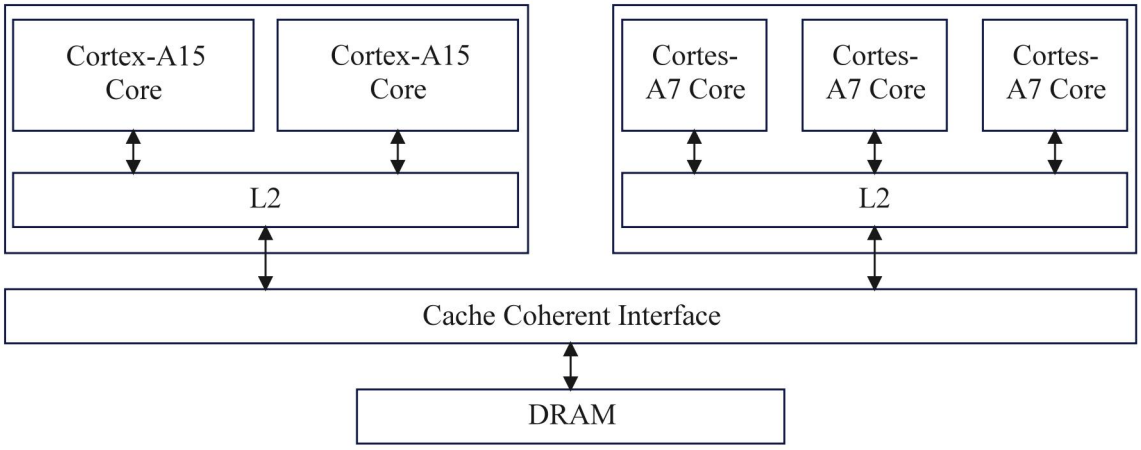


图 2-6 ARM big. LITTLE 处理器架构

Fig.2-6 ARM big. LITTLE processor architecture

该算法的任务模型采用 DAG 模型构建依赖任务集，其核心目标是在同时满足实时性约束与能耗约束，通过将任务分配至最优的处理器集群及其对应的工作电压/频率组合，以实现系统服务质量的最大化。以下为该算法的具体实现步骤：

（1）任务频率分配，此阶段的核心目标是通过优化执行频率的选择，以实现所有任务强制部分执行能耗总和的最小化。该过程采用贪婪算法策略，其具体实

现分为两层循环结构：外层循环依次遍历每个任务τ_i，内层循环则遍历所有可用的工作电压/频率等级。对于每个任务-频率组合，算法分别计算任务强制部分的执行时间与能耗。若计算结果同时满足预设的能耗约束与实时性约束，则更新当前最小能量值及其对应的最优频率。该过程通过迭代优化，最终确定每个任务的最优执行频率配置及所处集群位置。

（2）任务分配和调度，此阶段的优化目标是通过分配和调度任务的强制部分，实现处理器负载的均衡化，从而最大化每个处理器的空闲时间，为后续任务可选部分的执行预留更多的系统资源。该过程可分解为以下三个步骤：1）任务层序号确定：根据任务间的依赖关系，确定每个任务的层序号，以此决定任务分配与调度的优先级顺序；2）任务分配与调度：基于任务层序号及频率分配结果，将任务分配至合适的处理器，并确定其开始执行时间，以确保处理器空闲时间的最大化；3）实时性约束调整：若任务分配结果违反实时性约束，则动态调整当前任务及其前驱与后继任务的执行频率，以满足系统实时性要求，最后输出任务分配与调度的优化结果。

（3）可选周期调整，此阶段基于前两个阶段的优化结果，包括任务强制部分的分配方案、系统剩余能量以及各个处理器的空闲时间分布。该阶段的核心目标是通过充分利用处理器的空闲时间窗口和系统剩余能量，对任务可选部分的执行周期进行优化调整，以实现系统服务质量的最大化。具体而言，算法通过动态分配空闲时间与剩余能量资源，重新规划任务可选部分的执行策略，从而在满足系统约束条件的前提下，进一步提升整体性能指标。

2.6 本章小结

本章介绍了异构多核平台下任务调度研究中的相关技术概念，以及本文研究用于对比的服务质量感知的任务调度算法。首先，详细介绍了系统模型中实时系统模型的基本知识及异构多核处理器的核心特性。然后，介绍了任务调度算法中常用的任务模型，并重点介绍了本文研究中用于构建依赖任务集的 DAG 模型，包括该模型的基本定义、属性及其在任务调度中的应用，并介绍了混合任务模型的基本概念及其与服务质量的关联性。其次，分析了多核系统的功耗模型，并介绍了任务调度研究中常用的功耗管理及优化技术。最后，对本文研究中用于对比的算法进行逐一介绍，为后续实验分析提供了理论基础。

第 3 章 基于服务质量感知混合任务调度近似最优算法

3.1 本章引言

针对异构多核平台，现有任务调度算法存在对通用异构多核平台的适配性不足，以及缺乏对精确计算任务和非精确计算任务并存的混合任务模型的研究，本章提出了一种适用于通用异构多核平台下的基于服务质量感知的混合任务调度近似最优算法。

本章结构如下：3.2 节从任务模型、处理器模型及功耗模型 3 个方面介绍了本章算法采用的平台模型；3.3 节根据提出的模型进行问题的定义；3.4 节根据问题的定义将问题形式化表达，整合约束条件和优化目标构建问题；3.5 节根据原问题进行线性化和分解处理，并基于此提出了基于服务质量感知混合任务调度近似最优算法，且详细给出了算法的设计思路和实现；3.6 节通过设置多组仿真实验对提出的算法性能进行验证分析；3.7 节对本章内容进行总结。

3.2 平台模型

本章研究针对通用异构多核平台，提出一种基于服务质量感知的混合任务调度近似最优算法，算法使用的平台模型包括任务模型、处理器模型以及功耗模型。本节将对采用的模型进行逐一介绍。

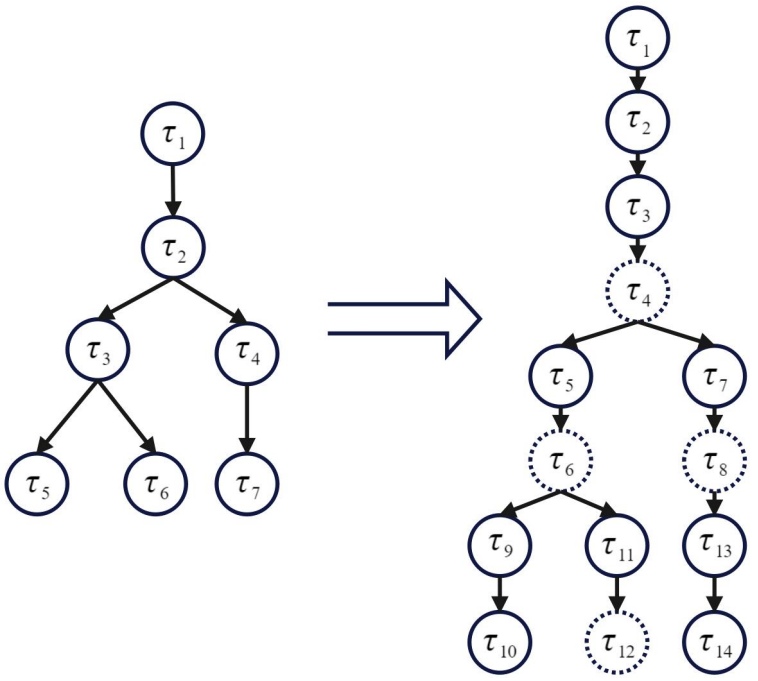
3.2.1 任务模型

任务模型的具体形式已在如 2.3.2 节说明，本章采用 DAG 模型来描述任务间的依赖关系。如图 3-1(a)所示，这种依赖关系可以使用 $G(\Gamma,E)$ 的任务图结构简化表示。其中，顶点集 $\Gamma=\{\tau_1,\tau_2,...,\tau_N\}$ 表示待执行的任务集，边集 E 表示任务间的数据依赖关系。本章重点研究具有依赖关系的混合任务模型，为区分混合模型中任务的不同类别，引入二进制变量 a_i：当 $a_i=1$ 时，任务 τ_i 为非精确计算任务；反之，任务 τ_i 为精确计算任务。

非精确计算任务可选部分的执行程度与系统服务质量呈正相关。对于每个任务 τ_i，M_i 和 o_i 均以最坏情况执行周期（the Worst Case Execution Cycles, WCEC）进行度量。其中，强制部分 M_i 是固定的，而可选周期 o_i 是可调的，并且 o_i 存在上限 O_i，即 $0\le o_i\le O_i$。由于任务通常执行周期数达到十万量级，单个周期的影响可以忽略不计[58]，因此本章将非精确计算任务可选部分执行周期 o_i 近似为连续变量进行处理。任务在时间 0 时刻释放，每个任务 τ_i 都有其对应的截止时间 D_i。任务 τ_i 在其

所有前驱任务的输入数据到达之前无法开始执行，同时，其输出数据仅在任务执行完成后才可供所有后继任务同时使用。因此，混合任务模型中的任务τ_i可以表示为$\tau_i =< M_i, a_i, o_i, D_i >$，若任务$\tau_i$为精确计算任务，则其不存在可选部分。=

对于非精确计算任务τ_i，其可在逻辑上分解为强制子任务$\tau_{i,M}$和可选子任务$\tau_{i,o}$，分别对应M_i个周期和o_i个周期的执行。强制子任务必须在任务截止期限前完成，以确保系统满足实时性约束；而可选子任务则在强制部分完成后执行，并根据系统中剩余可用资源（如处理器空闲时间与剩余能量）在截止期限前有选择地完成。由于本章研究考虑非精确计算任务的任务迁移，混合任务模型中的每个任务τ_i被分解为两个子任务τ_{2i-1}和τ_{2i}，分别表示强制子任务和可选子任务。若τ_i为精确计算任务，则其子任务τ_{2i}的执行周期为0。混合任务模型的DAG表示如图3-1(b)所示。



(a) 初始 DAG　　(b) 考虑任务迁移的 DAG

图 3-1 任务模型示例

Fig.3-1 Example of task model

3.2.2 处理器模型

本章研究算法主要基于通用异构多核平台实现，考虑一个具有M个处理器核$\Theta = \{\theta_1, \theta_2, ..., \theta_M\}$的处理器模型，每个处理器$\theta_j$都由给定的工作电压/频率$v_k / f_k$来表示，其模型如图3-2所示。

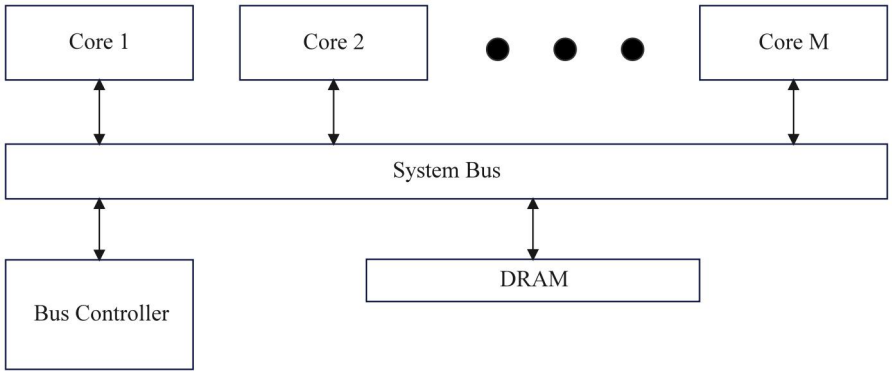


图 3-2 处理器模型

Fig.3-2 Processor model

本章使用的处理器模型基于以下几个重要前提：

（1）所有处理器核均可以使用任务级的 DVFS 技术，且处理器核之间相互独立，互不干扰，即处理器核可以单独调节自身执行单个任务时的工作电压/频率，每个处理器核均支持一组离散的工作电压/频率集合 $\{v_1/f_1, v_2/f_2, ..., v_L/f_L\}$；

（2）由于处理器模型的异构性，每个任务在不同的处理器核上的执行效率不同，任务 τ_i 在处理器核 θ_j 上的执行效率用 $\alpha_{i,j}$ 表示，其取值范围为[0.4,1][59]，例如数字信号处理相关的任务在 DSP 上的执行效率为 1；

（3）本章研究的任务调度是非抢占式调度，处理器核之间的通信和数据交互通过系统高速数据总线进行，不存在总线竞争，处理器核计算模块和通信模块同时进行。因此，与任务执行时间和能源相比，处理器核之间的通信成本可以忽略不计[60]，并且任务执行过程中将任务从一个处理器核迁移到另一个处理器核也是可行的[61]。

3.2.3 功耗模型

根据 2.4.1 节的功耗模型，处理器的功耗主要包含三个部分：动态功耗 P_{dyn}、静态功耗 P_{sta} 及固有功耗 P_{on}，其可表示为：

$$P_{total} = P_{dyn} + P_{sta} + P_{on} \tag{3-1}$$

在处理器运行过程中，静态功耗与固有功耗是维持系统正常运行的必要开销。即便处于待机模式，只要未完全切断电源，这部分功耗就会持续产生。然而完全关闭系统后再重启系统会带来更大的开销，因此本章假设系统始终处于运行状态。

由于任务分配算法和处理器时钟频率调整都无法改变静态功耗与固有功耗，且这些功耗与动态功耗相互独立，同时也不受 DVFS 技术调控的影响，因此为简化本章问题模型，在处理器激活功耗模型中对静态功耗与固有功耗不予考虑。虽

然这些功耗客观存在，但在本章的能耗计算中，系统总功耗仅由各处理器核的动态功耗决定。

3.3 问题定义

本章考虑了由M（$M \geq 2$）个异构核$\{\theta_1,\theta_2,...,\theta_M\}$组成的系统，满足 3.2.1 节中处理器模型的全部要求，在系统上执行一个任务数量为N的任务集Γ，任务集$\Gamma=\{\tau_1,\tau_2,...,\tau_N\}$，处理器核数量$M$小于任务数量$N$，根据 3.2.1 节中对平台模型的介绍可知，本章采用的任务可以使用一个四元组进行表示为$\tau_i =< M_i,a_i,o_i,D_i >$，具体含义可见 3.2.1 节。

由于平台处理器核的异构特性，任务在不同处理器核上的执行效率不同。引入$\alpha_{i,j}$表示任务在不同的处理器核上的执行效率，其中$i \in [1,N]$，$j \in [1,M]$，分析可知系统中每个任务都对应着一个M维的执行效率向量，将这些参数向量组合成矩阵可表示为：

$$\boldsymbol{\alpha}_{N\times M}=\begin{bmatrix} \alpha_{1,1} & \alpha_{1,2} & \cdots & \alpha_{1,M} \\ \alpha_{2,1} & \alpha_{2,2} & \cdots & \alpha_{2,M} \\ \vdots & \vdots & \ddots & \vdots \\ \alpha_{N,1} & \alpha_{N,2} & \cdots & \alpha_{N,M} \end{bmatrix} \tag{3-2}$$

上式(3-2)中矩阵$\boldsymbol{\alpha}_{N\times M}$的行向量$[\alpha_{i,1},\alpha_{i,2},\ldots,\alpha_{i,M}]$对应任务$\tau_i$在$M$个处理器核上的执行效率分布，其中每个元素$\alpha_{i,j}$的取值范围为[0.4,1]。

综上所述，在通用异构多核平台上，综合考虑处理器的异构性、DVFS 技术、任务执行的实时性以及平台能耗有限的情况，需要设计一种算法满足以下条件：

（1）确定分配方案，将任务分配到各个处理器核上；

（2）确定任务在处理器核上执行的工作电压/频率；

（3）确定任务的开始执行时间；

（4）任务调度结果符合任务依赖关系，不存在任务抢占；

（5）任务调度结果满足实时性约束；

（6）任务调度结果满足能耗约束。

3.4 问题分析

根据 3.2.1 节的论述，本章研究所采用的任务调度模型为 DAG 模型。在该模型中，每个任务τ_i最终的调度结果需明确其分配的处理器核以及执行时所采用的工

作电压/频率配置。此外，调度过程必须严格遵循任务间的依赖关系约束，以确保任务执行的正确性和时序一致性。

3.4.1 任务分配约束

（1）处理器核分配约束：引入二进制变量 $\lambda_{i,j}$ 表示任务 τ_i 处理器核的分配情况，如 $\lambda_{2,3}=1$ 表示任务 τ_2 被分配到处理器核 θ_3。由于本章任务调度研究考虑非精确计算任务的任务迁移，即非精确计算任务 τ_i 强制部分 τ_{2i-1} 和可选部分 τ_{2i} 可在不同处理器核上执行，因此子任务 τ_i 的处理器核分配约束可用下式表示：

$$\sum_{j=1}^{M}\lambda_{i,j}=1 \quad \forall i\in N' \tag{3-3}$$

上式(3-3)中，N' 表示任务分解后的数量集合，即 $N'=\{1,...,2N\}$。通过该方式，可以确保每个子任务都分配到一个处理器核。

（2）工作电压/频率分配约束：本章研究考虑任务级 DVFS，即每个处理器核 θ_j 使用一个工作电压/频率级别来执行分配的任务 τ_i。因此，引入二进制变量 $l_{i,l}$ 表示任务 τ_i 使用的工作电压/频率等级分配情况，如 $l_{2,1}=1$ 表示任务 τ_2 将以 v_1/f_1 执行。任务 τ_i 的工作电压/频率分配约束可用下式表示：

$$\sum_{l=1}^{L}l_{i,l}=1 \quad \forall i\in N' \tag{3-4}$$

上式(3-4)中，L 表示处理器核工作电压/频率可调整等级数量。通过该方式，可以确保每个任务都选择了一个工作电压/频率去执行。

（3）任务依赖性约束：

DAG 模型中，所有任务间的依赖关系可由矩阵表示为：

$$\boldsymbol{Q}_{N\times N}=\begin{bmatrix} q_{1,1} & q_{1,2} & \cdots & q_{1,N} \\ q_{2,1} & q_{2,2} & \cdots & q_{2,N} \\ \vdots & \vdots & \ddots & \vdots \\ q_{N,1} & q_{N,2} & \cdots & q_{N,N} \end{bmatrix} \tag{3-5}$$

任取依赖关系矩阵 $\boldsymbol{Q}_{N\times N}$ 中的一个行向量 $[q_{i,1},q_{i,2},\ldots,q_{i,N}]$ 表示任务 τ_i 与其他任务的依赖关系，如 $q_{i,j}=1$，则表示任务 τ_i 是任务 τ_j 的直接前驱任务。为确保任务依赖关系得到满足，避免因执行顺序错误导致数据不一致，根据任务的依赖关系引入任务的依赖性约束：

$$ts_j+(1-q_{i,j})D\geq te_i \tag{3-6}$$

上式(3-6)中，D表示任务共同截至期限。若任务τ_i是任务τ_j的直接前驱任务，即$q_{i,j}=1$，那么$ts_j \geq te_i$；相反$q_{i,j}=0$时，由于D远大于可能的任务开始时间差，因此该约束始终满足。

（4）任务非抢占调度约束：

同时，为确保处理器核在同一时间仅执行一个子任务，避免资源冲突，根据任务的依赖关系引入任务的非抢占调度约束：

$$te_i \leq ts_j + (2-\lambda_{i,k}-\lambda_{j,k})D + (1-q_{i,j})D \quad \forall i \neq j \in N', \forall k \in M \tag{3-7}$$

$$te_j \leq ts_i + (2-\lambda_{i,k}-\lambda_{j,k})D + q_{i,j}D \quad \forall i \neq j \in N', \forall k \in M \tag{3-8}$$

上式(3-7)和(3-8)中，若任务τ_i与任务τ_j分配在同一处理器核θ_j上执行，即$\lambda_{i,k}=\lambda_{j,k}=1$，则它们的执行时间不能重叠，因此在这个条件下，若任务τ_i是任务τ_j的直接前驱任务，即$q_{i,j}=1$，那么式(3-7)转换为$te_i \leq ts_j$，式(3-8)转换为$te_j \leq ts_i + D$，可以确保其执行时间不会重叠；若任务τ_i并非任务τ_j的直接前驱任务，即$q_{i,j}=0$，那么式(3-7)转换为$te_i \leq ts_j + D$，式(3-8)转换为$te_j \leq ts_i$，可以确保其执行时间不会重叠。

3.4.2 实时性约束

当任务τ_i被分配至处理器核θ_j，并以v_k/f_k执行时，其执行时间t_i可表示为：

$$t_i = \frac{M_i+o_i}{\alpha_{i,j}f_k} \tag{3-9}$$

由于本章研究考虑了非精确计算任务的任务迁移，即任务τ_i强制部分τ_{2i-1}和可选部分τ_{2i}可以在不同处理器核上执行，因此其完成时间te_{2i-1}和te_{2i}可分别表示为：

$$te_{2i-1} = ts_{2i-1} + \sum_{j=1}^{M}\sum_{l=1}^{L}\frac{\lambda_{2i-1,j}l_{2i,l}M_i}{\alpha_{i,j}f_l} \tag{3-10}$$

$$te_{2i} = ts_{2i} + \sum_{j=1}^{M}\sum_{l=1}^{L}\frac{\lambda_{2i,j}l_{2i,l}o_i}{\alpha_{i,j}f_l} \tag{3-11}$$

因此，为确保任务的实时性要求，避免任务因超时失效，引入实时性约束条件：

$$te_i \leq D_i \quad \forall i \in N' \tag{3-12}$$

上式(3-12)中，D_i表示任务的相对截止期限，其值由任务τ_i所需最小执行时间$t_{i,\min}$和最大执行时间$t_{i,max}$以及任务开始执行时间共同决定，具体表达式为：

$$D_i = ts_{2i-1} + \varphi(t_{i,max} - t_{i,min}) + t_{i,min} \tag{3-13}$$

上式(3-13)中，φ 为时间影响因子，其取值范围为 $0 \le \varphi \le 1$，$t_{i,max}$ 和 $t_{i,min}$ 分别表示任务 τ_i 在所有处理器核中的最大和最小执行时间，其数学表达式分别为：

$$t_{i,max} = \max_{j \in M, l \in L} \left[\frac{\lambda_{i,j} l_{2i,l} (M_i + O_i)}{\alpha_{i,j} f_l} \right] \tag{3-14}$$

$$t_{i,min} = \min_{j \in M, l \in L} \left[\frac{\lambda_{i,j} l_{2i,l} (M_i + O_i)}{\alpha_{i,j} f_l} \right] \tag{3-15}$$

通过上述实时性约束条件，系统能够在满足任务截止期限的前提下，优化任务的分配与执行。

3.4.3 能耗约束

若任务强制子任务和可选子任务完成分配，各自执行所需能耗 $E_{i,M}$ 和 $E_{i,o}$ 可分别表示为：

$$E_{i,M} = \frac{M_i}{\alpha_{i,j} f_l} P_{dyn} \tag{3-16}$$

$$E_{i,o} = \frac{o_i}{\alpha_{i,j} f_l} P_{dyn} \tag{3-17}$$

基于上式(3-16)及式(3-17)，系统执行所有任务所需总能耗可表示为：

$$E_{total} = \sum_{i=1}^{N} \sum_{j=1}^{M} \sum_{l=1}^{L} \left[\frac{\lambda_{2i-1,j} l_{i,l} M_i}{\alpha_{i,j} f_l} + \frac{\lambda_{2i,j} l_{i,l} o_i}{\alpha_{i,j} f_l} \right] P_{dyn} \tag{3-18}$$

为控制系统的总能耗，使其不能超过能量预算，引入任务的能耗约束：

$$E_{total} \le E_{sup} \tag{3-19}$$

上式(3-19)中，E_{sup} 表示平台的能耗预算，其取值范围由任务集Γ所有任务执行所需最小能耗 E_{min} 和最大能耗 E_{max} 表示：

$$E_{sup} = (E_{max} - E_{min}) \beta + E_{min} \tag{3-20}$$

上式(3-20)中，β 为能耗影响因子，其取值范围为 $0 \le \beta \le 1$，E_{max} 和 E_{min} 分别表示任务集在处理器核上执行的最大能耗和最小能耗，其数学表达式分别为：

$$E_{max} = \max_{j \in M, l \in L} \sum_{i=1}^{N} \left[\frac{\lambda_{i,k} l_{i,l} (M_i + O_i)}{\alpha_{i,j} f_l} \right] P_{dyn} \tag{3-21}$$

$$E_{max} = \min_{j \in M, l \in L} \sum_{i=1}^{N} \left[\frac{\lambda_{i,k} l_{i,l} (M_i + O_i)}{\alpha_{i,j} f_l} \right] P_{dyn} \tag{3-22}$$

通过上述能耗约束条件，系统能够在满足能量预算的前提下，优化任务的分配与执行。

3.4.4 问题形式化定义

本章研究算法是在通用异构多核平台上，以 DAG 模型为任务模型，在能耗约束和实时性约束下实现的，因此原问题可表示为：

$$\max_{\lambda,l,o,ts} Q(o)$$

$$s.t.\begin{cases} \sum_{j=1}^{M}\lambda_{i,j}=1, \sum_{i=1}^{L}l_{i,l}=1 \quad \forall i\in N' \\ ts_j+(1-q_{i,j})D\geq te_i \quad \forall i\in N' \\ te_i\leq ts_j+(2-\lambda_{i,k}-\lambda_{j,k})D+(1-q_{i,j})D \quad \forall i\neq j\in N',\forall k\in M \\ te_j\leq ts_i+(2-\lambda_{i,k}-\lambda_{j,k})D+q_{i,j}D \quad \forall i\neq j\in N',\forall k\in M \\ te_i\leq D_i \quad \forall i\in N' \\ E_{total}\leq E_{sup} \end{cases} \tag{3-23}$$

由于 $\lambda_{i,j}$、$l_{i,l}$ 是二进制变量，o_i 是线性变量，乘积 $\lambda_{i,j}l_{i,l}o_i$ 是非线性项，因此原优化问题是一个混合整数非线性规划（Mixed-Integer Nonlinear Programming，MINLP）问题。由于其兼具整数规划的离散特性与非线性规划的复杂特性，MINLP 问题在计算复杂性上被归类为 NP 难问题[25]。

3.5 近似最优算法

根据 3.4.4 节的分析，原问题可归类为 MINLP 问题，该问题属于 NP 难问题范畴。这一性质表明，随着问题规模的扩大，求解 MINLP 问题的计算复杂度将呈指数级上升。鉴于这一计算复杂性，本小节首先通过线性化方法将原始问题转化为混合整数线性规划（Mixed Integer Linear Programming, MILP）问题，从而降低问题的求解难度。进一步地，基于 Benders 分解方法，将 MILP 问题分解为一个整数线性规划（Integer Linear Programming, ILP）主问题和一个线性规划（Linear Programming, LP）子问题。通过这种分解，主问题负责优化二进制变量，而子问题则以主问题的解为参数优化和求解连续变量，从而在迭代过程中逐步逼近原问题的近似最优解，这一方法可以有效降低求解复杂度。

3.5.1 问题线性化

由于原问题同时包含连续变量、整数变量以及非线性目标函数和约束条件，因此直接解决此类非线性问题是很困难的。为求解原问题，本小节采用引理 1 所

描述的方法对原始问题进行线性化处理，从而将其转化为 MILP 问题。这一转换过程不仅保持了问题的本质特征，同时降低了求解复杂度，为后续的优化计算提供了理论基础。

引理 1：给定常数 s_1 和 s_2，其值均大于 0，定义两个约束空间：

$$S_1=\left\{[h,a,b,x]\middle|h=abx,-s_1\le x\le s_2,a\in\{0,1\},b\in\{0,1\}\right\} \tag{3-24}$$

$$S_2=\left\{[h,a,b,x]\middle|\begin{matrix}-abs_1\le h\le abs_2,h+abs_1-x-s_1\le 0,\\ h-abs_2-x+s_2\ge 0,a\in\{0,1\},b\in\{0,1\}\end{matrix}\right\} \tag{3-25}$$

则 $S_1\leftrightarrow S_2$，即两者描述的可行域等价。

证明：

1）当 $a=1$，$b=1$：

S_1 的约束为 $h=x$，且 $-s_1\le x\le s_2$；S_2 的约束为 $-s_1\le h\le s_2$，且 $h-x\le 0$，$h-x\ge 0$，即 $h=x$，与 S_1 一致。

2）当 $a=0$，$b=1$ 或 $a=1$，$b=0$ 或 $a=0$，$b=0$：

S_1 的约束为 $h=0$，且 x 的范围为 $-s_1\le x\le s_2$；S_2 的约束为 $h=0$，且 $h-x-s_1\le 0$，$h-x+s_2\ge 0$，即 $h=0$，且 x 的范围为 $-s_1\le x\le s_2$，S_1 和 S_2 等价性依然成立。综上所述，S_1 和 S_2 在所有情况下均等价，证毕■。

由于 $\lambda_{i,j}\in\{0,1\}$、$l_{i,l}\in\{0,1\}$ 和 $0\le o_i\le O_i$，根据引理 1，引入辅助变量 $h_{2i,j,l}$，将非线性项 $\lambda_{2i,j}l_{2i,l}o_i$ 转换为线性约束，可以确保原问题与线性化问题的等价性。因此，式(3-11)和(3-18)可被线性化为：

$$te_{2i}=ts_{2i}+\sum_{j=1}^{M}\sum_{l=1}^{L}\frac{h_{2i,j,l}}{\alpha_{i,j}f_l} \tag{3-26}$$

$$E_{total}=\sum_{i=1}^{N}\sum_{j=1}^{M}\sum_{l=1}^{L}\left[\frac{\lambda_{2i-1,j}l_{i,l}M_i}{\alpha_{i,j}f_l}+\frac{h_{2i,j,l}o_i}{\alpha_{i,j}f_l}\right]P_{dyn} \tag{3-27}$$

同时，针对辅助变量 $h_{i,j}$ 引入以下约束以保证变量替换后问题的等价性：

$$h_{2i,j,l}\le\lambda_{2i,j}l_{i,l}O_i \tag{3-28}$$

$$h_{2i,j,l}\le o_i \tag{3-29}$$

$$h_{2i,j,l}+(1-\lambda_{2i,j}l_{2i,l})O_i\ge o_i \tag{3-30}$$

上式中，约束条件式(3-26)用于确保 $h_{2i,j,l}$ 的取值不会超过可选子任务的最大执行量 O_i，并且该约束仅在子任务被分配到处理器 θ_j 并以 v_l/f_l 执行时（即 $\lambda_{2i,j}l_{2i,l}=1$）生效；约束条件式(3-27)进一步限制 $h_{2i,j,l}$ 的取值不会超过实际执行的可选子任务量 o_i。

约束条件式(3-28)规定，若可选子任务未被分配到处理器核（即 $\lambda_{2i,j}=0$ ）或无可选 v_l/f_l（即 $l_{2i,l}=0$ ），则强制 $h_{2i,j,l}\geq o_i$；然而，由于约束条件式(3-27)同时要求 $h_{2i,j,l}\leq o_i$，两者共同作用使得 $h_{2i,j,l}=o_i$，且此时式(3-26)要求 $h_{2i,j,l}\leq 0$，因此 $h_{2i,j,l}=o_i=0$，从而确保在子任务未分配时不会执行任何可选子任务。这一设计有效避免了资源分配中的逻辑冲突，保证了模型的合理性与可行性。

综上所述，原问题可转换为：

$$\max_{\lambda,l,o,ts,h} Q(o)$$

$$s.t.\begin{cases}\sum_{j=1}^{M}\lambda_{i,j}=1 \quad \forall i\in N'\\ \sum_{l=1}^{L}l_{i,l}=1 \quad \forall i\in N'\\ ts_j+(1-q_{i,j})D\geq te_i \quad \forall i\in N'\\ te_i\leq ts_j+(2-\lambda_{i,k}-\lambda_{j,k})D+(1-q_{i,j})D \quad \forall i\neq j\in N',\forall k\in M\\ te_j\leq ts_i+(2-\lambda_{i,k}-\lambda_{j,k})D+q_{i,j}D \quad \forall i\neq j\in N',\forall k\in M\\ te_i\leq D_i \quad \forall i\in N'\\ E_{total}\leq E_{sup}\\ h_{2i,j,l}\leq \lambda_{2i,j}l_{i,l}O_i\\ h_{2i,j,l}\leq o_i\\ h_{2i,j,l}+(1-\lambda_{2i,j}l_{i,l})O_i\geq o_i\end{cases} \tag{3-31}$$

3.5.2 问题分解

为进一步降低问题的维度，简化解决问题的难度，本小节采用 Benders 分解方法[62]对 3.5.1 小节 MILP 问题进行分解，其核心思想是将 MILP 问题分解为一个松弛主问题和一个子问题，子问题不断返回可行割和不可行割，然后把这些割添加到松弛主问题中去，逐步优化主问题的解，最终得到原问题的近似最优解。基于 Benders 分解方法，主问题为整数线性规划（Integer Linear Programming, ILP）问题，其任务是优化二进制变量；子问题则为线性规划（Linear Programming, LP）问题，其以主问题的解作为输入参数，优化连续变量。通过这种分解方式，主问题与子问题在迭代求解过程中相互协作，逐步趋近原问题的最优解。为便于使用 Benders 分解方法，本小节首先将优化目标函数由 $\max Q(o)$ 转化为 $-\min Q(o)$。以下引理及其证明阐述了通过上述分解方法，可以确定原问题最优解的上下界，从而为后续迭代求解近似最优解提供理论依据：

引理 2：主问题的解 Ψ_M 是原问题目标函数的下界，子问题的解 Ψ_S 是原问题的目标函数上界，即 $\Psi_M\leq\Psi_{min}\leq\Psi_S$，其中 Ψ_{min} 表示原问题的最优解。

证明：

1）主问题通过松弛连续变量 $\boldsymbol{y}$，仅优化二进制变量 $\boldsymbol{x}$，其解 Ψ_M 必然不超过原问题的最优值 Ψ_{min}，即 $\Psi_M \le \Psi_{min}$；

2）子问题在固定二进制变量 $\boldsymbol{x}$ 后优化连续变量 $\boldsymbol{y}$，其解 Ψ_S 是原问题在特定 $\boldsymbol{x}$ 条件下的可行解，其解 Ψ_S 必然不小于原问题的最优值 Ψ_{min}，即 $\Psi_{min} \le \Psi_S$。综上所述，原问题目标函数下界是主问题的解 Ψ_M，上界是子问题的解 Ψ_S，证毕■。

基于 Benders 分解方法开始对转换后的 MILP 问题进行分解：

（1）变量分解：

在对原问题进行分解之前，需对其涉及变量进行系统化的拆分与重组，具体规则如下：1）引入二进制变量向量 $\boldsymbol{x}=[\lambda,l]^T$，其包含任务处理器核分配变量 λ 和任务工作电压/频率等级分配变量 l；2）引入连续变量向量 $\boldsymbol{y}=[o,h,ts]^T$，其包含可选子任务执行量 o、辅助变量 h 和子任务开始时间 ts。通过这种变量拆分与重组方式，能够将原问题中的复杂耦合关系解耦，从而为后续的分解与求解提供清晰的数学表达和计算框架。

（2）约束分解：

进一步地，对约束进行分类处理：

1）将仅涉及二进制变量的约束统一表示为如下形式：

$$\boldsymbol{A}_1\boldsymbol{x} \bullet \boldsymbol{c}_1 \tag{3-32}$$

上式(3-30)中，$\boldsymbol{A}_1$ 为系数矩阵，$\boldsymbol{c}_1$ 为右端常数向量。

2）将同时涉及二进制变量和连续变量的约束统一表示为如下形式：

$$\boldsymbol{A}_2\boldsymbol{x}+\boldsymbol{B}\boldsymbol{y} \bullet \boldsymbol{c}_2 \tag{3-33}$$

上式(3-31)中，$\boldsymbol{A}_2$ 和 $\boldsymbol{B}$ 分别是为二进制变量和连续变量的系数矩阵，$\boldsymbol{c}_2$ 为右端常数向量。以能耗约束式(3-19)为例，其可转换为以下形式：

$$\sum_{i=1}^{N}\sum_{j=1}^{M}\sum_{l=1}^{L}\left[\frac{\lambda_{2i-1,j}l_{i,l}M_i}{\alpha_{i,j}f_l}\right]P_{dyn}+\sum_{i=1}^{N}\sum_{j=1}^{M}\sum_{l=1}^{L}\left[\frac{h_{2i,j,l}}{\alpha_{i,j}f_l}\right]P_{dyn} \le E_{sup} \tag{3-34}$$

相应的，由于原目标函数仅涉及连续变量 o，因此原目标函数可表示为 $\Psi=\boldsymbol{f}^T\boldsymbol{y}$，其中 $\boldsymbol{f}$ 是目标函数系数的向量。

（3）问题分解

基于 Benders 分解方法，原问题可被分解为一个主问题和一个子问题。其中主问题为 ILP 问题，其核心任务是优化二进制变量 $\boldsymbol{x}=[\lambda,l]^T$，同时引入辅助变量 Ψ' 作

为主问题目标函数代理变量。子问题则为线性规划 LP 问题，其以主问题的解 $\boldsymbol{x}$ 作为输入参数，优化连续变量 $\boldsymbol{y}$，其数学形式可表示为：

$$\begin{aligned} &\min_{y\geq 0} \Psi \\ &s.t. \boldsymbol{A}_2\boldsymbol{x}+\boldsymbol{B}\boldsymbol{y} \bullet \boldsymbol{c}_2 \end{aligned} \tag{3-35}$$

由于需要通过迭代优化求解，子问题需要生成约束以优化主问题。为此，对子问题引入拉格朗日乘子 γ（$\gamma \geq 0$），构造对偶问题：

$$\begin{aligned} &\max_{\boldsymbol{\gamma}\geq 0} \boldsymbol{\gamma}^T(\boldsymbol{A}_2\boldsymbol{x}-\boldsymbol{c}_2) \\ &s.t. \boldsymbol{f}+\boldsymbol{B}'\boldsymbol{\gamma} \geq 0 \end{aligned} \tag{3-36}$$

根据子问题的求解结果，生成 Benders 割平面，这些割平面以约束的形式添加到主问题中，以逐步逼近最优解。具体而言，根据子问题是否存在最优解 $\boldsymbol{y}$，可引入以下两类约束：

1）最优性约束：

若子问题存在最优解，则添加约束以更新辅助变量 Ψ'，从而优化主问题的目标函数：

$$\Psi' \geq \boldsymbol{\gamma}^T(\boldsymbol{A}_2\boldsymbol{x}-\boldsymbol{c}_2) \tag{3-37}$$

其确保主问题的目标值 Ψ' 不高于当前子问题的最优值。

2）可行性约束：

若子问题不存在最优解，则添加以下约束以排除当前主问题不可行的解 $\boldsymbol{x}$：

$$\boldsymbol{\gamma}^T(\boldsymbol{A}_2\boldsymbol{x}-\boldsymbol{c}_2) \leq 0 \tag{3-38}$$

以下引理及其证明阐述了通过引入上述约束，主问题的迭代过程能够逐步排除非最优解及不可行解，从而确保算法的收敛性与最优性：

引理 3：通过添加最优性约束和可行性约束，主问题在迭代过程能够逐步排除非最优解和不可行解。

证明：

1）最优性约束：

假设在第 u 次迭代，对偶子问题返回有界解 $\boldsymbol{\gamma}(u)^T$，若当前候选解 $\boldsymbol{x}(u)$ 非最优，即满足 $\Psi(\mathrm{k}) < \boldsymbol{\gamma}(u)^T(\boldsymbol{A}_2\boldsymbol{x}(u)-\boldsymbol{c}_2)$，该约束将排除 $\boldsymbol{x}(u)$，因为后续迭代要求变量 Ψ' 满足 $\Psi' \geq \boldsymbol{\gamma}(u)^T(\boldsymbol{A}_2\boldsymbol{x}(u)-\boldsymbol{c}_2)$，从而确保优化方向。

2）可行性约束：

若对偶子问题无界，即$\boldsymbol{\gamma}(u)^T(\boldsymbol{A}_2\boldsymbol{x}(u)-\boldsymbol{c}_2)=+\infty$，则表明此时原问题在$\boldsymbol{x}(u)$下无可行解，该约束将直接排除$\boldsymbol{x}(u)$，因为后续迭代要求$\boldsymbol{\gamma}(u)^T(\boldsymbol{A}_2\boldsymbol{x}(u)-\boldsymbol{c}_2)\le 0$，从而确保解的可行性。

通过数学归纳法可知，每次迭代至少排除一个非优解或不可行解。由于二进制变量$\boldsymbol{x}$的组合有限，算法最终将收敛至唯一最优解，证毕■。

通过添加子问题产生的割约束，主问题可表示为如下形式：

$$\min_{\boldsymbol{x},\Psi}\Psi' \quad s.t.\begin{cases}\boldsymbol{A}_1\boldsymbol{x}\le\boldsymbol{c}_1\\ \Psi'\ge\boldsymbol{\gamma}^T(\boldsymbol{A}_2\boldsymbol{x}-\boldsymbol{c}_2)\\ \boldsymbol{\gamma}^T(\boldsymbol{A}_2\boldsymbol{x}-\boldsymbol{c}_2)\le 0\end{cases} \tag{3-39}$$

3.5.3 近似最优算法

基于 Benders 通用分解方法的核心思路，本章提出一种基于服务质量感知近似最优调度算法（QoS-Aware Approximate Optimal Scheduling Algorithm, QAOS）。该算法通过对分解后的主问题和子问题进行迭代求解，以实现近似最优解的获取。为详细说明其求解过程，算法利用 YALMIP 和 CPLEX 求解器进行实现，其具体步骤如算法 3-1 的伪代码所示。

算法 3-1 近似最优算法

输入：$\Theta=\{\theta_1,\theta_2,...,\theta_m\},\Gamma,\boldsymbol{Q}_{N\times N},\boldsymbol{\alpha}_{N\times M}$

输出：任务分配和调度方案及Ψ_{min}

1: $x(0),\Psi_M(0)=0,\Psi_S(0)=\text{MAX_INT32},u=0$;

2: 定义二进制变量λ和l;

3: 添加初始约束，见式(3-3)、(3-4)、(3-6)、(3-7)、(3-8);

4: 初始化 Benders 割列表为空;

5: **while** $\Psi_S(u)-\Psi_M(u)\le\varepsilon$ **do**

6: $u=u+1$;//迭代次数

7: 使用 CPLEX 求解器求解主问题;

8: 使用 value 函数获取主问题最优解;

9: 将主问题解集代入子问题;

10: 使用 sdpvar 定义连续变量o,h,ts;

11: 添加约束，见式(3-12)、(3-19)、(3-28)、(3-29)、(3-30);

12: 使用 CPLEX 求解器求解子问题;

算法 3-1 近似最优算法（续）

13: **if** 子问题无解 **then**

14: 终止迭代;

15: **else**

16: 更新上下界;

17: 提取对偶变量;

19: 生成可行性割，见式(3-38);

18: 生成最优性割，见式(3-37);

20: 添加上述割至 Benders 割列表;

21: **end if**

22: **end while**

23: 输出任务分配和调度方案及Ψ_{min}

算法 3-1 的求解过程分为初始化阶段和迭代求解阶段。在初始化阶段（步骤 1），设定二进制变量向量$\boldsymbol{x}$的初始解集为$\boldsymbol{x}(0)$，目标函数最优值Ψ_{min}的上界初始化为无符号整数的最大值，下界初始化为 0，迭代次数u初始化为 0。步骤 2 定义二进制变量λ和l，其维度分别为$N \times M$维和$N \times L$维。步骤 3 为主问题添加初始约束条件，步骤 4 初始化 Benders 割列表为空集，即主问题在初次求解时不包含任何割平面约束。

步骤 5-19 为迭代求解阶段，其阶段的核心是通过主问题与子问题的交替求解逐步逼近最优解。首先，在步骤 5 中统计迭代次数$u = u + 1$。步骤 6 使用 CPLEX 求解器求解主问题，获得第u次迭代的候选解$\boldsymbol{x}(u)$，并通过 YALMIP 工具箱中的 value 函数获取当前主问题目标函数最优值$\Psi_M(u)$。步骤 7 将主问题的候选解$\boldsymbol{x}(u)$作为参数输入至子问题。步骤 8 使用 YALMIP 工具箱中的 sdpvar 函数定义连续变量o、h及ts，其维度分别为N维、$N \times M$维及N维。步骤 9 为子问题添加与连续变量相关的约束条件，步骤 10 使用 CPLEX 求解器求解子问题。

步骤 13 根据子问题求解结果执行分支操作：若子问题无可行解，则终止迭代过程；若子问题存在可行解，则更新当前迭代的上下界，提取对偶变量，并根据式(3-37)与式(3-38)生成最优性割和可行性割，将其添加至 Benders 割列表中，随后返回步骤 5 开始下一轮迭代。在每次迭代中，主问题将在增加 Benders 割列表中约束的情况下重新求解，以剔除上一轮迭代中的次优解及不可行解。迭代过程持续进行，直至满足收敛条件$\Psi_S(u) - \Psi_M(u) \le \varepsilon$（其中$\varepsilon$为预设的收敛容差）。若满足收敛条件，则判定算法收敛，输出最优解Ψ_{min}；否则，继续迭代直至收敛。

每次迭代过程中，主问题通过添加最优性约束和可行性约束后，其解作为原问题的下界是非递减的，而子问题的解作为原问题的上界是非递增的。由于原问

题最优解与上下界满足$\Psi_M \le \Psi_{min} \le \Psi_S$，且每次迭代会逐步缩小$\Psi_S(u)-\Psi_M(u)$，当该差值小于预设的收敛容差$\varepsilon$时，算法终止。同时，主问题的二进制变量$\boldsymbol{x}$的组合数为$2^{M\times N\times L}$，其值为有限值。尽管在最坏情况下算法可能需要遍历所有可能的$\boldsymbol{x}$组合，但在实际应用中，最优性约束和可行性约束会大幅减少所需的迭代次数。因此该算法能够在有限时间内收敛至近似最优解，从而避免陷入无限循环。

3.6 仿真结果与分析

为全面评估本章提出的 QAOS 算法的性能，本节设计并展开了多组对比实验。鉴于现有算法（如 TVS、AHS、GHS 及 EAS 等）主要针对非精确计算任务进行任务调度优化，而未考虑精确计算任务的存在，本节实验通过引入混合任务模型对上述算法进行扩展，分别得到 TVS-HT、AHS-HT、GHS-HT 及 EAS-HT 算法，并将其与 QAOS 算法进行性能对比分析。

本节所有仿真实验均在 MATLAB R2023a 仿真平台上完成。实验环境配置如下：操作系统为 Windows10 专业版，硬件平台配备 16G 内存及 Intel Core i7-11800H@2.3GHz 处理器（八核十六线程）。

为量化评估 QAOS 算法的性能，本节实验采用标准化实现的服务质量指标作为评价标准，具体定义如下：

标准化实现的服务质量（Normalized Achieved QoS, NAQ）：该指标表示各算法在实时性约束和能耗约束下能够实现的最大系统服务质量与系统所有非精确计算任务可选部分上限总和的比值，其数学表达式为：

$$NAQ = \frac{\max\left(Q(o)\right)}{\sum_{i=1}^{N} a_i O_i} \tag{3-40}$$

上式中，$\max\left(Q(o)\right)$表示算法在满足实时性约束和能耗约束条件下能够实现的最大系统服务质量值；$\Sigma_{i=1}^{N} a_i O_i$表示系统中所有非精确计算任务可选部分上限的总和。

3.6.1 仿真参数设置

本节仿真实验基于通用异构多核平台，旨在调度非固定数量的 DAG 任务。实验中的仿真变量参数取值范围详见表 3-1 所示。为模拟多样化任务场景，采用随机生成的方法为仿真实验生成不同规模的 DAG 任务集，其规模大小的取值范围参考文献[25]，[26]。假设所有处理器核支持 10 个离散频率执行点，其频率值分别为{0.1,0.2,…,1.0}。为确保实验数据的准确性并降低随机性影响，在仿真实验过程中，

针对每组实验的每种情况均进行了 1000 次数据输入，汇总所有结果后剔除最高值与最低值，最终对剩余数据取平均值。

表 3-1 仿真参数范围设置

Table3-1 Simulation parameter range setting

参数名	参数取值范围
任务执行周期 M_i、O_i	$[4\times10^7,6\times10^8]$
执行效率 α	[0.4,1]
处理器核数量 M	8
小规模任务集	15-35
中等以上规模任务集	50-100

实验一：本实验旨在评估不同能耗影响因子 β 和时间影响因子 φ 对本章算法性能的影响。实验设计中，时间影响因子 φ 的取值范围设定为[0.2:0.2:1]，能耗影响因子 β 取值设定为{0.2,0.4,0.6,0.8}，任务数分别设定为 5、15、30、50。其他参数均参照表 3-1 中的变量范围进行设置。

实验二：本实验旨在评估不同任务数量下各算法的服务质量性能表现以及能耗使用情况。实验设计中，时间影响因子 φ 固定为 0.5（或 0.2），能耗影响因子 β 固定为 0.5（或 0.2），任务数的取值范围设定为[5:5:100]。其他参数均参照表 3-1 中的变量范围进行设置。

实验三：本实验旨在评估任务模型中精确计算任务的不同占比对各算法的服务质量表现的影响。实验设计中，时间影响因子 φ 固定为 0.5，能耗影响因子 β 固定为 0.5，任务数分别设定为 15 和 50，精确计算任务数量的取值范围分别设定为[0:3:9]和[0:10:40]，其他参数均参照表 3-1 中的变量范围进行设置。

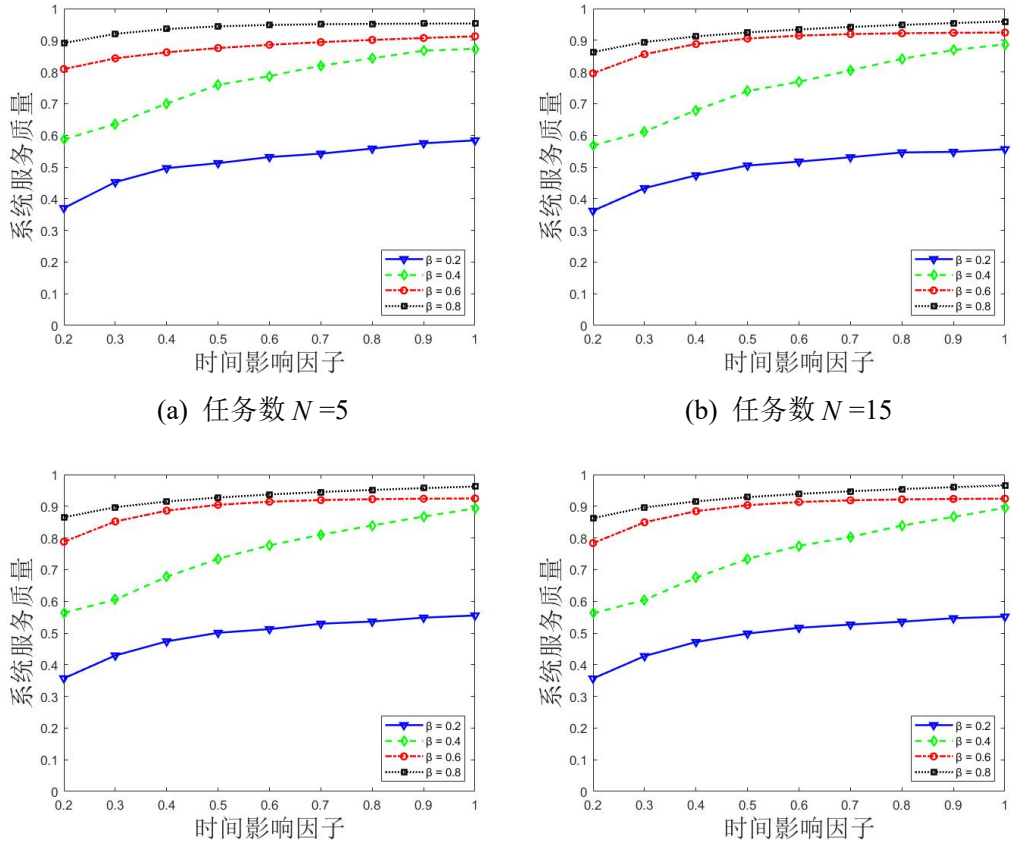
实验四：本实验旨在评估不同任务数量下的各算法的计算时间。在实验设计中，时间影响因子 φ 固定为 0.5，能耗影响因子 β 固定为 0.5，任务数的取值范围为[5:5:100]。其他参数均参照表 3-1 中的变量范围进行设置。

3.6.2 仿真结果分析

（1）实验一的仿真结果如图 3-3 及图 3-4 所示，本实验对比分析了本章提出的 QAOS 算法在不同任务数量及不同的能耗影响因子 β 和时间影响因子 φ 条件下能够实现的系统服务质量。

图 3-3(a-d)分别展示了任务数量为 5、15、30 及 50 时，QAOS 算法的系统服务质量随 β 和 φ 的变化的趋势。图 3-4(a-b)分别展示了系统提供能耗预算随能耗影

响因子变化以及任务截止期限随时间影响因子变化的情况。实验结果表明，在小型、中型及大型的任务规模范围内，系统服务质量均随 β 和 φ 的增加而显著提升。这一现象可归因于以下机制：如图 3-4(a)所示，能耗影响因子 β 与系统预设能耗呈正相关关系，随着 β 值的增大，系统能提供的能耗预算相应增加，从而使得可选子任务执行的周期数更多。同时如图 3-4(b)所示，时间影响因子 φ 与任务相对截止期限亦呈正相关关系，较高的 φ 值会使任务的截止期限更宽松，因此系统能够在满足实时性约束的条件下，为可选子任务分配更多的执行周期数。然而，在任务数量固定的情况下，随着 β 和 φ 的进一步增加，系统服务质量增长速率逐渐减缓并趋于平稳。这是由于非精确计算任务 τ_i 的可选子任务 $\tau_{i,o}$ 执行周期 o_i 存在上限，即满足 $0 \le o_i \le O_i$，从而导致系统服务质量的提升空间受到限制。

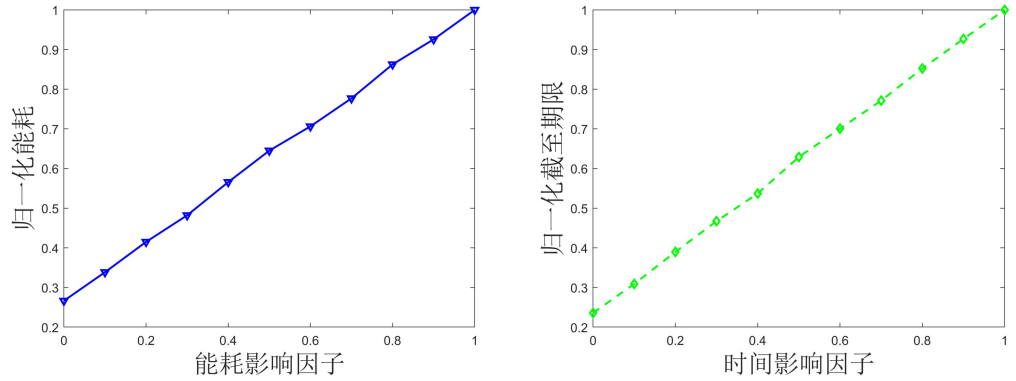


(a) 任务数 N =5　　(b) 任务数 N =15

(c) 任务数 N =30　　(d) 任务数 N =50

图 3-3 QAOS 算法在不同 N 、β 及 φ 条件下的系统服务质量变化趋势

Fig.3-3 Trends in system QoS achieved by the QAOS with N , β and φ varying



(a) 能耗变化　　(b) 截止期限变化

图 3-4 能耗与截止期限变化

Fig.3-4 Energy consumption and deadline changes

（2）实验二的仿真结果如图 3-5、图 3-6、图 3-7 所示，本实验对比分析了本章提出的 QAOS 算法和 TVS-HT、AHS-HT、GHS-HT 及 EAS-HT 算法在不同时间影响因子 φ 以及不同能耗影响因子 β 条件下，任务集（不包含精确计算任务）中不同任务数量对系统服务质量性能和能耗使用的影响。

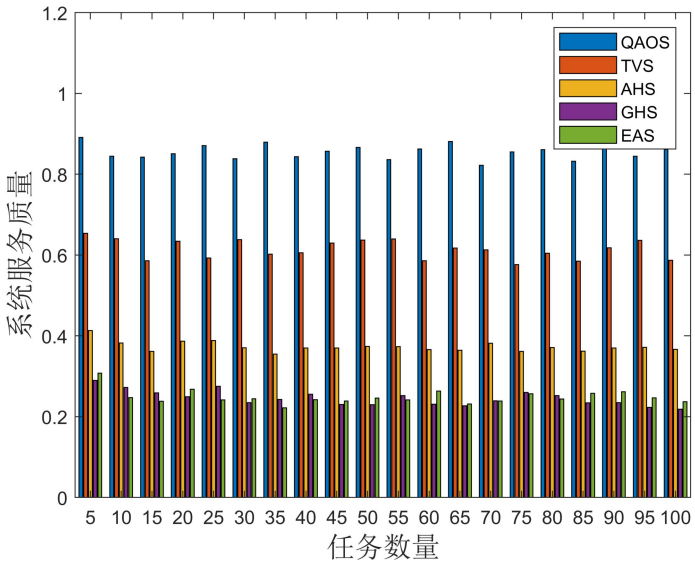


图 3-5 $\varphi=0.5$， $\beta=0.5$ 不同算法的服务质量对比

Fig.3-5 QoS comparison in different algorithms when $\varphi=0.5$ and $\beta=0.5$

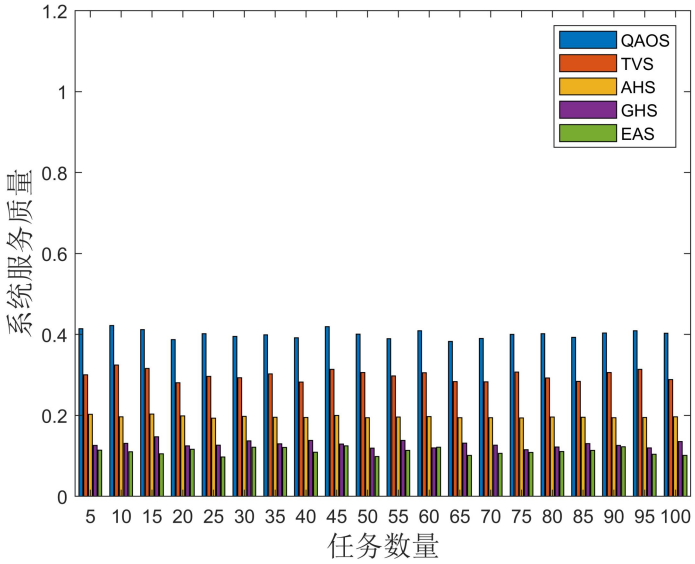
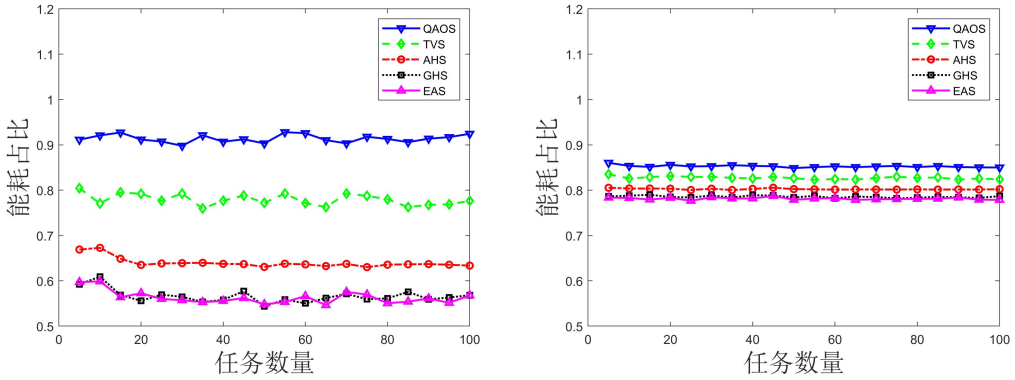


图 3-6 $\varphi = 0.2$，$\beta = 0.2$ 不同算法的服务质量对比

Fig.3-6 QoS comparison in different algorithms when $\varphi = 0.2$ and $\beta = 0.2$

图 3-5 及图 3-6 分别展示了不同算法在$\varphi = 0.5$，$\beta = 0.5$ 和$\varphi = 0.2$，$\beta = 0.2$ 条件下不同任务数量对系统服务质量性能的影响。实验结果表明，在任务数量变化的情况下，QAOS 算法的系统服务质量均优于 TVS-HT、AHS-HT、GHS-HT 及 EAS-HT 算法，这是由于 QAOS 算法在设计中引入了 DVFS 技术以及任务迁移策略，而 GHS-HT 和 EAS-HT 算法未充分考虑这些因素。此外，与 AHS-HT 等启发式算法相比，QAOS 算法求解属于最优算法，其求解结果在理论上更接近全局最优解；与 TVS-HT 算法相比，尽管两者均属于最优算法，但 QAOS 算法对于非精确计算任务的可选部分优化中采用连续线性调整策略，而 TVS-HT 算法采用离散化处理，从而 QAOS 算法的优化结果会更好。定量分析表明，QAOS 算法的平均系统服务质量值比 TVS-HT 算法高 39.09%，而 AHS-HT 算法可以达到 QAOS 算法的 43.35%，GHS-HT 算法可以达到 28.70%，EAS-HT 算法可以达到 29.08%。

图 3-7(a-b)分别展示了不同算法在$\varphi = 0.5$，$\beta = 0.5$ 和$\varphi = 0.2$，$\beta = 0.2$ 条件下的不同任务数量对能耗使用的影响，实验结果表明，QAOS 算法得到分配和调度方案使用的能耗均高于其他算法，这是因为相较于其他算法，QAOS 算法求得的解是采用连续线性调整策略的近似最优解，其能充分利用系统提供的能耗以实现更高的系统服务质量。定量分析表明，QAOS 算法的平均能耗占比值比 TVS-HT 算法高 14.81%，比 AHS-HT 算法高 35.29%，比 GHS-HT 算法高 55.93%，比 EAS-HT 算法高 52.14%。



(a) $\varphi=0.5$， $\beta=0.5$　　(b) $\varphi=0.2$， $\beta=0.2$

图 3-7 不同算法的使用能耗对比

Fig.3-7 Comparison of QoS in different algorithms

（3）实验三的仿真结果如图 3-8 及图 3-9 所示，本实验对比分析了本章提出的 QAOS 算法和 TVS-HT、AHS-HT、GHS-HT 及 EAS-HT 算法在时间影响因子φ固定为 0.5，能耗影响因子β固定为 0.5 条件下，任务集中不同精确计算任务数量对系统服务质量性能的影响。

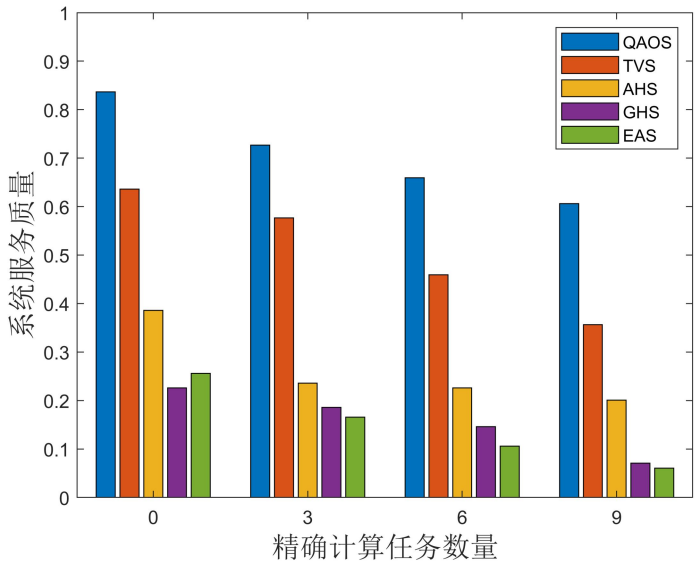


图 3-8 混合任务模型中不同算法的服务质量对比

Fig.3-8 QoS comparison in different algorithms in hybrid task models

图 3-8 展示了任务数量为 15 时，QAOS 算法的系统服务质量随任务集中精确计算任务数量的变化的趋势。实验结果表明，在任务集中精确计算任务数量变化的情况下，QAOS 算法的系统服务质量均优于 TVS-HT、AHS-HT、GHS-HT 及

EAS-HT 算法。这一优势主要归因于 WTV-HT、WDV-HT 及 WTA-HT 算法未对精确计算任务进行进一步优化，以及 TVS-HT 算法对于非精确计算任务的可选部分采用离散化处理。此外，在任务集中精确计算任务占比较大（精确计算任务数量为 9）时，QAOS 算法依然能够保持 62.35%的系统服务质量性能，而启发式算法 GHS-HT 和 EAS-HT 算法的系统服务质量性能显著下降并趋近于 0。这是因为 GHS-HT 和 EAS-HT 算法未充分考虑 DVFS 技术以及任务迁移策略，且在任务分配策略中未对精确计算任务进行优化。精确计算任务的分配会占用大量的系统资源（能耗和执行时间），从而导致系统为非精确计算任务的可选子任务执行所预留的剩余系统资源相对有限。定量分析表明，在任务集中存在精确计算任务的条件下，QAOS 算法的平均系统服务质量值比 TVS-HT 算法高 31.48%，而 AHS-HT 算法可以达到 QAOS 算法的 31.33%，GHS-HT 算法可以达到 18.78%，EAS-HT 算法可以达到 17.59%。

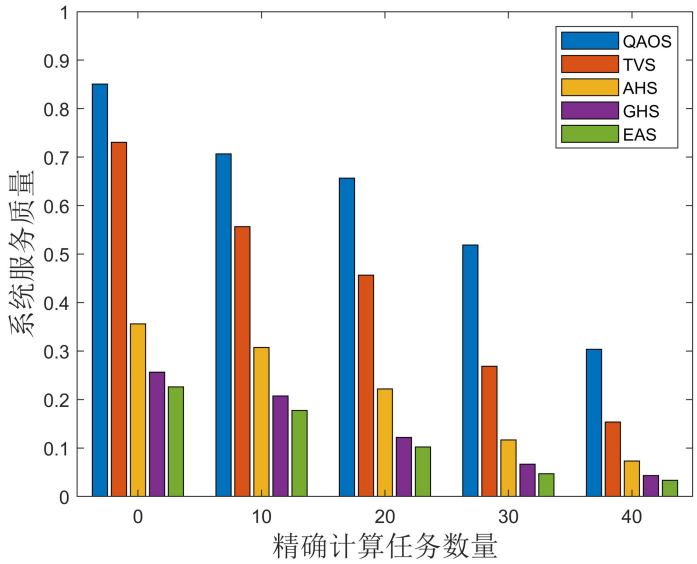


图 3-9 混合任务模型中不同算法的服务质量对比

Fig.3-9 QoS comparison in different algorithms in hybrid task models

图 3-9 展示了任务数量 50 时 QAOS 算法的系统服务质量随任务集中精确计算任务数量的变化的趋势。实验结果表明，与图 3-8 的结果一致，在任务集中精确计算任务数量变化的情况下，QAOS 算法的系统服务质量均优于 TVS-HT、AHS-HT、GHS-HT 及 EAS-HT 算法。同时，当任务集中精确计算任务占比较大（精确计算任务数量为 30 或 40）时，QAOS 算法依然能够保持 29.81%~51.68%的系统服务质量性能，而启发式算法 GHS-HT 和 EAS-HT 的系统服务质量性能则显著下降并趋近于 0，这些现象的原因与前述分析同理。定量分析表明，在任务集中存在精确计

算任务的条件下，QAOS 算法的平均系统服务质量值比 TVS-HT 算法高 23.82%，而 AHS-HT 算法可以达到 QAOS 算法的 25.29%，GHS-HT 算法可以达到其的 16.35%，EAS-HT 算法可以达到其的 13.77%。

（4）实验四的仿真结果如图 3-10 所示，本实验对比分析了本章提出的 QAOS 算法和 TVS-HT、AHS-HT、GHS-HT 及 EAS-HT 算法在时间影响因子 φ 固定为 0.5，能耗影响因子 β 固定为 0.5 条件下，任务集中不同任务数量对算法计算时间的影响。实验结果表明，在不同任务数量条件下，QAOS 算法的计算时间都大于启发式算法，并且在中大规模任务集中，其计算时间远远超过启发式算法。这是因为主问题的二进制变量 $\boldsymbol{x}$ 的组合数为 $2^{M\times N\times L}$，尽管可以通过最优性约束和可行性约束大幅减少符合要求的 $\boldsymbol{x}$ 的组合数量，但与启发式算法相比其循环次数依然是一个比较大的数值，从而导致计算时间增加。

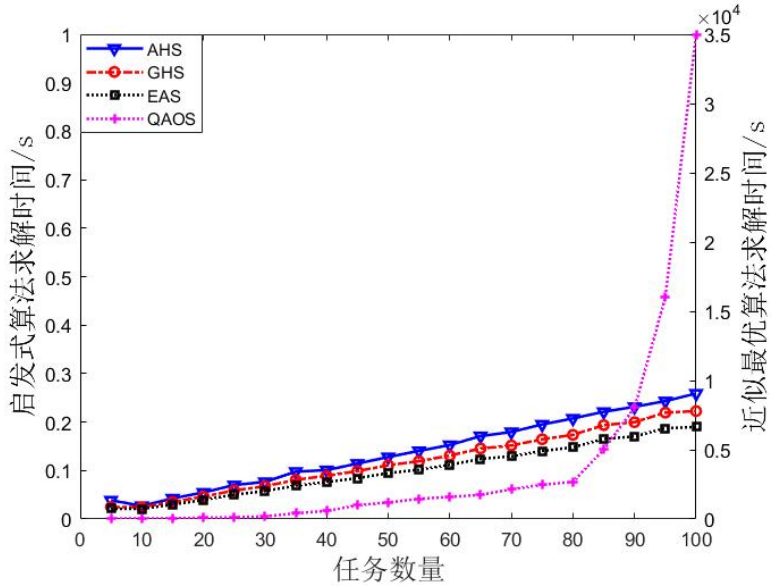


图 3-10 不同算法计算时间

Fig.3-10 Computation time in in different algorithms

3.7 本章小结

本章在异构多核平台下提出了一种基于服务质量感知的混合任务调度的近似最优算法，其目标是在满足能耗约束和实时性约束下，最大化系统服务质量。首先，本章从任务模型、处理器模型以及功耗模型三个方面对算法所采用的平台模型进行了介绍，并对所研究的问题进行了深入分析，明确了问题的相关约束条件。在此基础上，原问题被建模为一个 MINLP 问题。然后，通过线性化处理将原问题转化为 MILP 问题，并基于 Benders 分解通用方法将 MILP 问题进一步分解为一个

ILP 主问题和一个 LP 子问题。基于此分解框架，本章提出了一种近似最优算法（QAOS），使用 YALMIP 和 CPLEX 求解器进行对主问题和子问题进行迭代求解。最后，通过设计对比仿真实验验证了算法的有效性。实验结果表明，对于非精确计算任务模型，QAOS 算法相较于 TVS-HT 算法系统服务质量性能平均提高 39.09%；对于混合任务模型，QAOS 算法相较于 TVS-HT 算法系统服务质量性能平均提高 23.82%~31.48%。然而，该算法的能耗使用比其他算法高 14.81%~55.93%。同时，由于 QAOS 算法基于优化理论实现，该算法求解时间要高于一般的启发式算法，尤其是在任务集规模为中大型时。为解决 QAOS 算法求解时间长的问题，本文将在第 4 章提出相应的改进方法。

第 4 章　基于服务质量感知混合任务调度启发式算法

4.1 本章引言

在第 3 章中，本文提出近似最优算法使用 DVFS 技术，综合考虑任务分配、任务调度的任务迁移，在实时性约束和能耗约束下最大化系统服务质量。从仿真实验结果分析来看，该近似最优算法对系统服务质量的提升效果优于其他算法，但该算法本质是基于优化理论的近似算法，其只适用于小规模问题的求解，当用于中大规模问题时，计算时间将呈指数级增长，且算法本身也难以被实际工程实现。因此，针对以上问题，本章提出了一种基于服务质量感知混合任务调度多阶段启发式算法。此算法求解时间相较于近似最优算法具有更低的复杂度，更短的求解时间，并在系统服务质量提升效果上接近近似最优解。

本章结构如下：4.2 节从任务模型、处理器模型及功耗模型 3 个方面介绍了本章算法采用的平台模型；4.3 节根据提出的模型进行问题的定义；4.4 节根据问题的定义将问题形式化表达，整合约束条件和优化目标构建问题；4.5 节提出了基于服务质量感知混合任务调度多阶段启发式算法，且详细给出了每个阶段算法的设计思路和实现；4.6 节通过设置多组仿真实验对提出的算法性能进行验证分析；4.7 节对本章内容进行总结。

4.2 平台模型

本章研究的是适用于通用异构多核平台下的基于服务质量感知的混合任务调度启发式算法，算法基于的平台模型包括任务模型、处理器模型、功耗模型等。本节将对采用的模型进行逐一介绍。

4.2.1 任务模型

本章研究所采用的任务模型在 3.2.1 节基础上引入了任务分层机制，以帮助后续算法确定任务分配顺序。在 DAG 中，对于入口任务 τ_1，其任务层级 $level_1=1$；而对于非入口任务 τ_i，根据任务间依赖关系其任务层级 $level_i$ 可表示为：

$$level_i = \max_{\tau_j \in pred(\tau_i)} \{level_j\} + 1 \tag{4-1}$$

上式(4-1)表示任务 τ_i 的任务层级由其直接前驱任务集中最大任务层级决定。如图 4-1 所示，入口任务 τ_i 任务层级 $level_1=1$；任务 τ_2 任务层级 $level_2=2$；任务 τ_3 和任务 τ_4 任务层级 $level_3=level_4=3$。

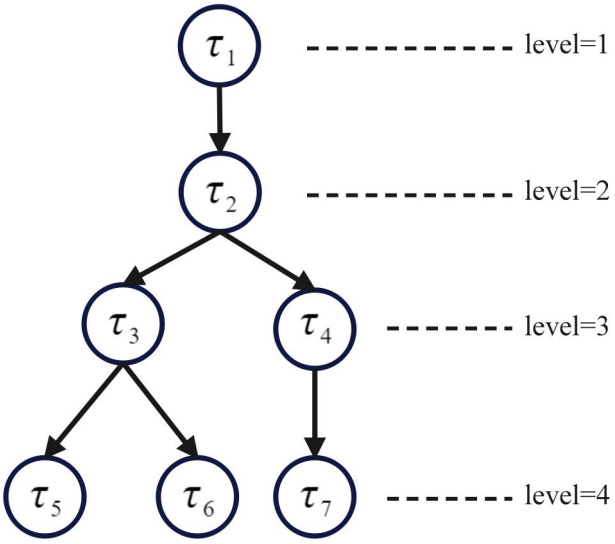


图 4-1 任务层示例

Fig.4-1 Example of task layer

4.2.2 处理器模型与功耗模型

本章中的处理器模型与功耗模型与 3.2 节完全相同。

4.3 问题定义

本章研究的系统由 M（$M \geq 2$）个异构处理器核 $\{\theta_1,\theta_2,...,\theta_M\}$ 组成，满足 3.2.1 节中处理器模型的全部要求，在该系统上执行的任务集 Γ 包含 N 个任务，即 $\Gamma=\{\tau_1,\tau_2,...,\tau_N\}$，其中处理器核数量小于任务数量 N。根据 3.2.1 节及 4.2.1 节中对平台模型的描述，本章采用的任务模型可通过五元组 $\tau_i=\{M_i,a_i,O_i,D_i,level_i\}$ 表示，其具体含义可见 3.2.1 节及 4.2.1 节。

问题的具体定义与 3.3 节完全一致，即任务调度问题在满足能耗约束、实时性约束及任务依赖约束的前提下，通过优化任务分配与调度策略，最大化系统服务质量。

4.4 问题分析

近似最优算法在求解过程中严格按照式(3-4)至式(3-6)所描述的约束条件，充分考虑了任务间的依赖关系以及任务的非抢占执行特性。相比之下，启发式算法为降低求解复杂度，需要对相应约束条件进行优化处理，从而在保证解的质量的同时也能提高算法的计算效率。

对于任务τ_i的开始执行时间，需要从两方面考虑：1）任务τ_i直接前驱任务集中所有任务最晚完成时间；2）若任务τ_i分配到处理器核θ_j上，处理器θ_j上最早可利用时间。基于上述条件，任务τ_i强制部分的开始执行时间$ts_{i,M}$及其完成时间$te_{i,M}$可以表示为：

$$ts_i = ts_{i,M} = \max\left\{\max_{\tau_h \in pred(\tau_i)}\{te_h\}, avail[\theta_j]\right\} \tag{4-2}$$

$$te_{i,M} = ts_{i,M} + t_{i,M} \tag{4-3}$$

上式(4-2)中，ts_i表示任务τ_i开始执行时间，$avail[\theta_j]$表示处理器θ_j上当前最早可用时间；上式(4-3)中$t_{i,M}$表示任务τ_i强制部分的执行时间。

由于任务τ_i存在相对截止期限D_i，其可选部分将在其强制部分完成后立即进行。因此，任务τ_i可选部分的开始执行时间$ts_{i,o}$及其完成时间$te_{i,o}$可以表示为：

$$ts_{i,o} = te_{i,M} \tag{4-4}$$

$$te_i = te_{i,o} = ts_{i,o} + t_{i,o} \tag{4-5}$$

上式(4-4)中，te_i表示任务τ_i完成时间；上式(4-5)中，$t_{i,o}$表示任务τ_i可选部分的执行时间。本章研究基于硬实时系统，即每个任务τ_i必须在截止期限D_i完成。因此，实时性约束可表示为：

$$te_i \le D_i \quad \forall i \in N' \tag{4-6}$$

上式(4-6)中，D_i由任务τ_i所需最小执行时间$t_{i,min}$、最大时间时间和$t_{i,max}$及任务τ_i直接前驱任务集中所有任务最晚完成时间决定，具体表达式见式(3-11)。通过上述方法，能够有效确保任务间的依赖关系以及非抢占执行特性得到满足，从而保证任务调度的正确性与实时性。

综上所述，本章研究算法是在通用异构多核平台上，以 DAG 模型为任务模型，在能耗约束和实时性约束下实现的，应当满足以下基本条件：

（1）任务分配到具体处理器核上；

（2）任务严格按照其依赖关系顺序执行；

（3）任务不允许抢占；

（4）确定任务的开始执行时间，所有任务结束时间不得超过截止期限；

（5）确定每个任务在核上的执行频率；

（6）确保任务执行所需能耗满足系统能耗约束。

基于优化问题形式，本章针对近似最优算法对中大规模任务集求解难度高且计算时间长的不足，提出一种新的多阶段启发式算法。该算法主要解决在通用异

构多核平台上如何高效地调度依赖任务集，并结合 DVFS 技术在满足实时性约束和能耗约束情况下最大化服务质量。

该算法采用分阶段递进求解策略，主要分为三个阶段：任务强制部分分配、空闲时间优化及任务可选部分分配。其核心思路如下：强制子任务和精确计算任务必须在任务截止期限前完成，以确保系统满足实时性约束；而可选子任务则在强制部分完成后执行，并根据系统中剩余可用资源（如处理器空闲时间与剩余能量）在截止期限前有选择地完成。通过这种分阶段优化策略，算法能够在满足实时性约束的同时，充分利用系统资源以提升系统服务质量。

4.5 启发式算法

4.5.1 算法概述

针对以上对近似最优算法存在问题的分析，本章提出一种基于服务质量感知混合任务调度多阶段启发式算法（QoS-Aware Hybrid Task Scheduling Heuristic Algorithm, QHTS）。该算法对近似最优算法的任务依赖性约束和任务非抢占调度约束做了优化，其基本流程如算法 4-1 所示。

算法 4-1：QHTS 算法

输入：处理器核 $\Theta=\{\theta_1,\theta_2,...,\theta_M\}$、任务集 $\Gamma=\{\tau_1,\tau_2,...,\tau_N\}$、任务依赖关系矩阵 $\boldsymbol{Q}_{N\times N}$、任务执行效率矩阵 $\boldsymbol{\alpha}_{N\times M}$

输出：最优任务调度方案，系统服务质量最大值

1.任务强制部分分配//算法 4-2

2.空闲时间优化//算法 4-3

3.任务可选部分分配//算法 4-4

根据上述算法概述表可知，本章提出的 QHTS 算法采用三阶段优化策略。第一阶段为任务强制部分频率选择及处理器核分配过程，其目的是在预设能耗范围内最小化强制部分和精确计算任务执行所需能耗；第二阶段为任务强制部分的调度以处理器空闲时间的优化，通过 DVFS 技术充分使用预设能耗以最大化处理器核上的空闲时间；第三阶段为任务可选部分的分配与调度，通过 DVFS 技术最大化非精确任务可选部分执行量，实现系统服务质量的最大化，算法的流程框图如图 4-2 所示。

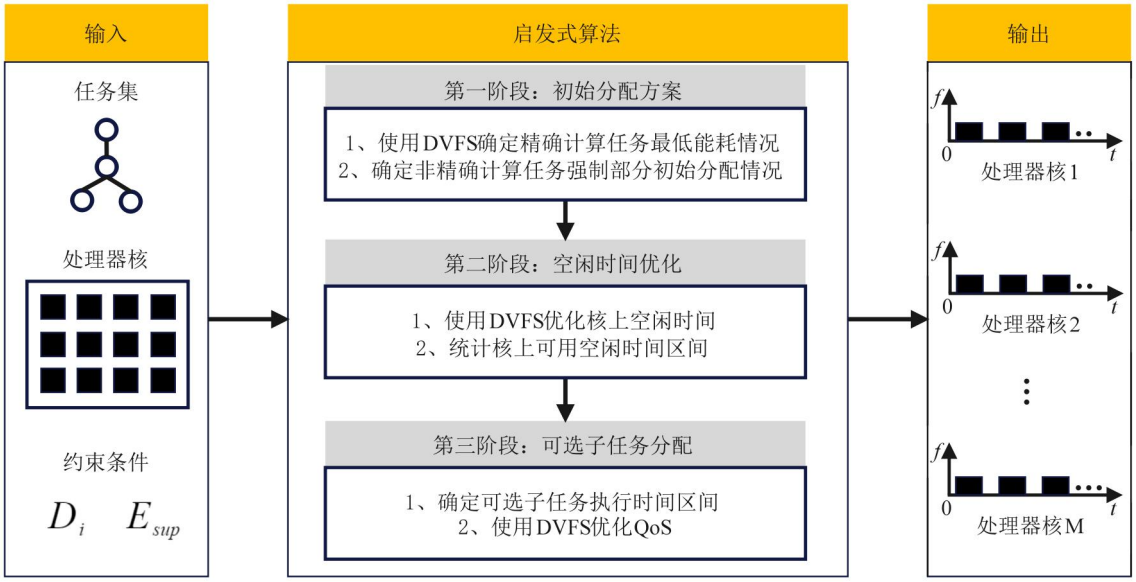


图 4-2 QHTS 算法框图

Fig.4-2 Diagram of QHTS algorithm

4.5.2 任务强制部分分配

本小节重点研究任务强制部分在处理器核间的分配问题，目的是在预设能耗范围内最小化强制部分和精确计算任务执行所需能耗，构建初始分配方案，为后续空闲时间和可选任务的分配优化奠定基础。

在详细介绍算法具体内容之前，本小节首先探讨算法为此阶段预设能耗的必要性。对于非精确计算任务 τ_i，假设其在时间 t 内以能耗 E 完成执行，其中强制部分以工作电压 v_M 和频率 f_M 执行，可选部分以工作电压 v_o 和频率 f_o 执行，则可建立如下关系式：

$$E=\frac{M}{f_M}f_M{v_M}^2+\frac{o}{f_o}f_o{v_o}^2=M{v_M}^2+o{v_o}^2 \tag{4-7}$$

$$t=\frac{M}{f_M}+\frac{o}{f_o} \tag{4-8}$$

通过联立式上式(4-7)和式(4-8)，可得任务的可选部分执行量 o 为：

$$o=\frac{E-tf_0f_Mv_M^2}{v_0^2-f_Mv_M^2} \tag{4-9}$$

上式(4-9)中，若保持可选部分工作电压 v_o 和频率 f_o 不变，同时降低强制部分的能耗（即降低 v_M 和 f_M），由于分子与分母同步变化，难以准确估计 o 的具体变化趋势。因此，本章算法通过预设能耗的方式，来充分考虑服务质量最大的情况，以确保任务调度的优化效果。

为确定此阶段的预设能耗，引入常量η，表示任务集中所有非精确计算任务强制部分和精确计算任务的周期之和与所有任务周期总和之比，其可表示为：

$$\eta = \frac{\Sigma_{i=1}^{n}(M_i + a_i O_i)}{\Sigma_{i=1}^{n}(M_i + O_i)} \tag{4-10}$$

基于此，此阶段的能耗预算$E_{total,M}$范围可表示为：

$$\eta E_{sup} \le E_{total,M} \le E_{sup} \tag{4-11}$$

上式(4-11)中，E_{sup}表示平台的能耗预算，其取值范围见式(3-16)，为进一步量化能耗预算的分配，引入变量μ（$\eta \le \mu \le 1$）表示$E_{total,M}$的占用比，即：

$$E_{total,M} \le \mu E_{sup} \tag{4-12}$$

通过上述方法，本小节任务强制部分分配算法伪代码如下算法 4-2 所示：

算法 4-2 任务强制部分分配

输入：$\Theta = \{\theta_1, \theta_2, ..., \theta_M\}$，$\Gamma = \{\tau_1, \tau_2, ..., \tau_N\}$，$\boldsymbol{Q}_{N\times N}$，$\boldsymbol{\alpha}_{N\times M}$

输出：任务强制部分初始分配方案

1:按照任务层级顺序和任务强制部分执行指令周期降序排列得到任务分配顺序队列Γ'

2:初始化$E_{total,M} = 0$，$E_{total,AC} = 0$，$avail[\theta_j] = 0$，$\mu = \eta$;

3: **for** $i \in \Gamma'$ **do**

4: $E_{i,M}^{min}$ = MAX_INT32 , *core_index* = 0, *level_index* = 0, *flag_finish* = 0;

5: **for** *j* = 1, …, *M* **do**

6: **for** *l* = 1, …, *L* **do**

7: 计算任务$\tau_{i,M}$的完成时间$te_{i,M}$;//式(3-9)

8: 计算任务$\tau_{i,M}$的所需能耗$E_{i,M}$;//式(3-16)

9: **if** $te_{i,M} \le D_i$ && $E_{total,M} + E_{i,M} \le \mu E_{sup}$ && $E_{i,M} \le E_{i,M}^{min}$ **then**

10: *flag_finish*=1;//$\tau_{i,M}$可分配

11: $E_{i,M}^{min} = E_{i,M}$;

12: *core_index=j*;//更新处理器核下标

13: *level_index=l*;//更新工作电压/频率等级下标

14: **end if**

15: **end for**

16: **end for**

17: **if** *flag_finish* == 0 **then**

18: 任务分配失败;

19: **else**

算法 4-2 任务强制部分分配（续）

20: $avail[\theta_{core_index}] = avail[\theta_{core_index}] + D_i$;
21: 记录处理器核 θ_{core_index} 时间区间使用情况;
22: **if** $a_i == 1$ **then**//任务 τ_i 为精确计算任务
23: **if** $te_{i,M} \neq D_i$ **then**
24: 找出小于 f_{level_index} 的最大可用电压 f_x;
25: 计算电压 f_x 占用比例 ω_1，见式(4-16);
26: 计算双频率组合执行所需能耗 $E_{i,M}^{min}$，见式(4-17);
27: **end if**
28: **end if**
29: $E_{total,M} = E_{total,M} + E_{i,M}^{min}$;
30: **end if**
31: **end for**

算法 4-2 步骤 1 确定任务分配优先级。根据任务的分层级别按升序排列，同一层级的任务再结合其强制部分（精确计算任务强制部分包含可选部分）的执行周期量按降序排列，生成任务分配顺序队列 Γ'。该排序策略优先分配层级较低且计算量较大的任务，以确保关键任务能够优先获得资源分配。

步骤 2 执行以下初始化操作：1）将能耗累加变量 $E_{total,M}$ 初始化为 0，该变量用于统计系统中所有任务强制部分在执行过程中所消耗的总能量；2）将能耗累加变量 $E_{total,AC}$ 初始化为 0，该变量用于统计系统中所有精确计算任务在执行过程中所消耗的总能量；3）将处理器核 θ_j 的可用空闲时间 $avail[\theta_j]$ 初始化为 0，该变量用于动态记录处理器核 θ_j 在当前及后续调度过程中可分配空闲时间资源，其值将根据任务分配结果实时更新；4）将此阶段预设能耗占用比 μ 初始化为 η，其值大小见式(4-10)。

在步骤 3-4 中，算法从任务集合 Γ' 中按预定义优先级顺序提取当前待分配任务。为确保能耗优化的初始条件，将任务的最小能耗值 $E_{i,M}^{min}$ 初始化为无符号整数的最大值，该值作为能耗计算的初始上界，用于后续动态更新任务的实际能耗。同时，将任务分配的处理器核编号 *core_index* 和工作电压/频率等级 *level_index* 均初始化为 0，分别表示任务初始未分配的处理器核编号和未选择的工作电压/频率等级，用于后续动态更新任务实际分配的处理器核及工作电压/频率等级。

步骤 6-31 详细描述了任务 $\tau_{i,M}$ 的分配优化流程，其具体过程如下：在步骤 6-16 中，首先计算任务 $\tau_{i,M}$ 在处理器核 θ_j 以第 l 级工作电压/频率等级执行时的完成时间 $te_{i,M}$ 及能耗 $E_{i,M}$。若任务 $\tau_{i,M}$ 满足以下条件：1）任务的完成时间 $te_{i,M}$ 不超过其相对

截止期限D_i；2）任务的能耗满足预设约束条件；3）当前分配方案产生的能耗低于已记录的最小能耗值；则更新任务分配参数，包括处理器核编号 *core_index* 及工作电压/频率等级 *level_index*。通过遍历所有处理器核及工作电压/频率等级组合，算法确定任务$\tau_{i,M}$的最小能耗分配方案。

步骤 22 中，若任务τ_i为精确计算任务，则需要通过双频率组合进一步降低其能耗，充分利用其相对截止期限。首先找出小于f_{level_index}的最大可用电压f_x，然后通过式(4-16)计算电压f_x占用比例ω_1，并计算任务双频率组合执行所需能耗。

在上述过程中，实时更新系统总能耗累加变量$E_{total,M}$和精确计算任务总能耗$E_{total,AC}$，并动态调整处理器核θ_j的可用空闲时间$avail[\theta_j]$，以反映任务分配后的资源占用情况。有关任务两个执行频率点的能耗低于单个执行频率点的能耗引理与证明如下。

引理 4：在离散工作电压/频率平台上，当系统在相同截止期限下运行时，若不存在与任务需求完全匹配的可用频率，则采用双频率组合执行策略能够实现比单频率执行策略更低的能耗。

证明：

设任务在单频率$(v_{i,cal}, f_{i,cal})$下的执行时间为t和能耗E_1为：

$$t = \frac{M_i}{\alpha_{i,j} f_{i,cal}} \tag{4-13}$$

$$E_1 = \frac{M_i}{\alpha_{i,j}} v_{i,cal}^{\ 2} \tag{4-14}$$

当目标电压$v_{i,cal}$不可用时，采用双电压f_x和f_{x+1}组合执行，其占比分别为ω_1和$1-\omega_1$，其中$0<\omega_1<1$。双电压组合执行的能耗关系可表示为：

$$t = t_x + t_{x+1} = \frac{\omega_1 M_i}{\alpha_{i,j} f_x} + \frac{(1-\omega_1) M_i}{\alpha_{i,j} f_{x+1}} \tag{4-15}$$

根据上式解得ω_1的值为：

$$\omega_1 = \frac{\alpha_{i,j} t f_x f_{x+1} - f_x M_i}{M_i (f_{x+1} - f_x)} \tag{4-16}$$

同时，双频率组合执行的总能耗E_2可表示为：

$$E_2 = \frac{\omega_1 M_i}{\alpha_{i,j}} v_x^{\ 2} + \frac{(1-\omega_1) M_i}{\alpha_{i,j}} v_{x+1}^{\ 2} = \frac{M_i}{\alpha_{i,j}} [\omega_1 v_x^{\ 2} + (1-\omega_1) v_{x+1}^{\ 2}] \tag{4-17}$$

因此，$E_1 - E_2$可表示为：

$$E_1 - E_2 = \frac{M_i}{\alpha_{i,j}} [v_{i,cal}^{\ 2} - \omega_1 v_x^{\ 2} - (1-\omega_1) v_{x+1}^{\ 2}] \tag{4-18}$$

上式中，由于目标电压$v_{i,cal}$不可用，若采用单电压执行，$v_{i,cal}$仅存在两种可能的取值情况：

1）若$v_{i,cal}=v_x$，则E_1-E_2的值为：

$$E_1-E_2=\frac{M_i}{\alpha_{i,j}}[(1-\omega_1)(v_x{}^2-v_{x+1}{}^2)]<0 \tag{4-19}$$

2）若$v_{i,cal}=v_{x+1}$，则E_1-E_2的值为：

$$E_1-E_2=\frac{\omega_1 M_i}{\alpha_{i,j}}(v_{x+1}{}^2-v_x{}^2)>0 \tag{4-20}$$

然而，当选择v_x作为单电压执行时，f_x无法满足任务相对截至期限，即$t_x>D_i$，所以只能选择v_{x+1}才能在相对截至期限前完成任务的执行。因此，综合上述分析可得$E_1-E_2>0$恒成立。这表明，双频率组合执行策略能够实现比单频率执行策略更低的能耗，证毕■。

以图 3-1(a)所示 DAG 为例，经过算法 4-2 的执行，非精确任务强制部分及精确计算任务的分配结果如图 4-3 所示。

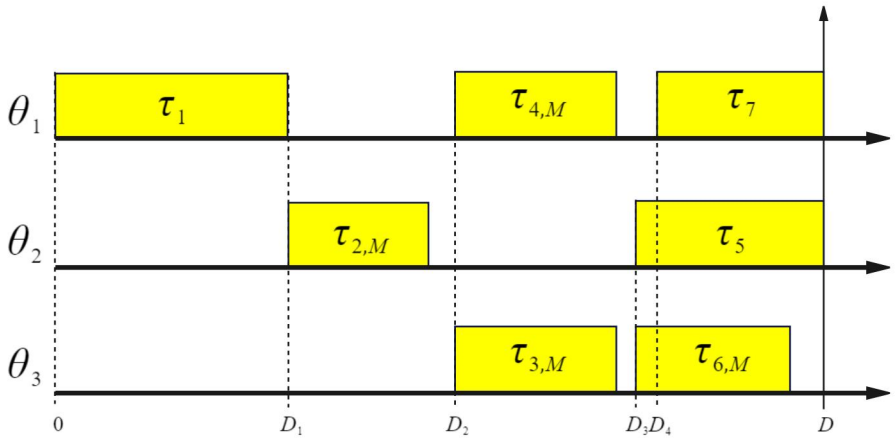


图 4-3 算法 4-2 结果示例

Fig.4-3 Example of algorithm 4-2 results

以处理器核θ_1为例，其执行过程中的频率变化情况如图 4-4 所示，核θ_1上共分配了 3 个任务，其频率变化曲线基于任务的执行时间区间及离散频率点的执行时间占比绘制而成。根据文献[63]的研究表明，处理器在频率切换过程中产生的时间开销范围为 30-150 微秒。由于该时间开销相对于任务执行的总时间可忽略不计，因此在任务调度建模中通常不予考虑。

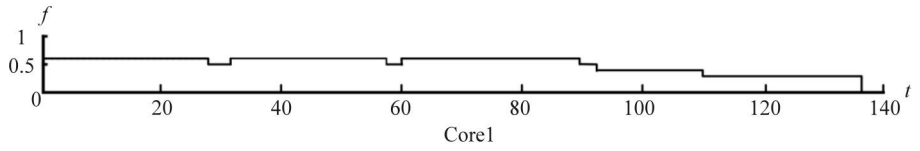


图 4-4 核上执行频率变化图

Fig.4-4 Change figure of execution frequency on the core

4.5.3 空闲时间优化

在算法 4-2 执行完成后，可以得到任务集中精确计算任务与非精确计算任务的强制子任务的初始分配方案。本小节算法将在该初始分配方案的基础上，进一步优化处理器核上空闲时间，旨在为后续可选子任务的执行预留更多的可用时间资源。本小节算法具体流程如算法 4-3 所示：

算法 4-3 空闲时间优化

输入：$\Theta=\{\theta_1,\theta_2,...,\theta_m\}$，$\Gamma'$，$\boldsymbol{Q}_{N\times N}$，$\boldsymbol{\alpha}_{N\times M}$，$\mu$

输出：$t_{i,op}$

1: $ESR_M=\frac{(\mu E_{sup}-E_{total,M})}{(E_{total,M}-E_{total,M})}$;

2: **for** j = 1, …, M **do**

3: 初始化 $core_avail(j)=[0,D]$;

4: **end for**

5: **for** $i\in\Gamma'$ **do**

6: $E_{i,ex}=ESR\times E_{i,M}^{min}$;

7: 计算满足 $E_{i,ex}$ 的目标电压 $v_{i,cal}$，见式(4-21);

8: **if** $v_{i,cal}\in\{v_1,v_2,...,v_L\}$

9: 计算目标电压执行时间 $t_{i,op}$;

10: **else**

11: 找出小于 $v_{i,cal}$ 的最大可用电压 v_x;

12: 找出大于 $v_{i,cal}$ 的最小可用电压 v_{x+1};

13: 计算电压 v_x 占用比例 ω_2，见式(4-25);

14: 计算双电压组合执行时间 $t_{i,op}$，见式(4-26);

15: **end if**

16: 更新 $core_avail(core_index)$，加入 $ts_{i,M}$ 和 $te_{i,M}$;

17: **end for**

在算法 4-3 的步骤 1 中，首先计算任务的能耗延展比例（Energy Stretch Ratio，ESR），该比例用于量化非精确计算任务的强制部分能耗扩展程度。步骤 2-4，初始化处理器核时间使用统计数组 $core_avail(core_index)$，用于记录各处理器核的时间占用情况，为后续处理器核空闲时间的统计与优化提供基础数据支持。步骤 5-17 是处理器核空闲时间优化阶段，步骤 6 计算任务 τ_i 延展后的可用能耗 $E_{i,ex}$，该值表示任务在满足实时性约束条件下可分配的最大能耗。在步骤 7 中，基于可用能耗 $E_{i,ex}$，计算满足该能耗约束的目标电压 $v_{i,cal}$：

$$v_{i,cal} = \sqrt{\frac{\alpha_{i,j} E_{i,ex}}{M_i}} \tag{4-21}$$

由于本章研究的异构处理器核仅支持离散频率执行点，因此由上式计算得到的目标电压 $v_{i,cal}$ 在多数情况下不可直接使用。这一离散化特性导致目标电压 $v_{i,cal}$ 通常无法与处理器核的可用电压点精确匹配，从而需要通过电压选择策略在相邻可用电压点之间进行优化分配，以实现任务执行时间与能耗的最优平衡。在步骤 5-12 中，算法根据目标电压 $v_{i,cal}$ 的可用性采取不同的执行策略：

1）目标电压可用情况：

若目标电压 $v_{i,cal}$ 可用，则直接采用 $v_{i,cal}$ 作为任务 τ_i 的执行电压，并计算任务在该工作电压/频率等级下的执行时间 $t_{i,op}$；

2）目标电压不可用情况：

若目标电压 $v_{i,cal}$ 不可用，则从处理器核的工作电压/频率等级中选择满足 $v_x \le v_{i,cal} \le v_{x+1}$ 的两个相邻电压 v_x 和 v_{x+1} 作为任务的执行电压。其中，v_x 为小于 $v_{i,cal}$ 的最大可用电压，v_{x+1} 为大于 $v_{i,cal}$ 的最小可用电压。随后，计算双电压组合下任务的执行时间 $t_{i,op}$。

以下引理及其证明阐述了在离散工作电压/频率平台上，任务采用双电压组合执行策略相较于单电压执行策略在时间性能上的优越性：

引理 5：在离散工作电压/频率平台上，当系统在相同能耗约束下运行时，若不存在与任务需求完全匹配的可用电压，则采用双电压组合执行策略能够实现比单电压执行策略更优的任务完成时间。

证明：

设任务在单电压 $(v_{i,cal}, f_{i,cal})$ 下的执行时间为 t_1 和能耗 E_{ex} 为：

$$t_1 = \frac{M_i}{\alpha_{i,j} f_{i,cal}} \tag{4-22}$$

$$E_{ex} = \frac{M_i}{\alpha_{i,j}} v_{i,cal}^{\ 2} \tag{4-23}$$

当目标电压$v_{i,cal}$不可用时，采用双电压v_x和v_{x+1}组合执行，其占比分别为ω_2和$1-\omega_2$，其中$0<\omega_2<1$。双电压组合执行的能耗关系可表示为：

$$E_{ex}=E_x+E_{x+1}=\frac{\omega_2 M_i}{\alpha_{i,j}}v_x^{\,2}+\frac{(1-\omega_2)M_i}{\alpha_{i,j}}v_{x+1}^{\,2} \tag{4-24}$$

根据上式解得ω_2的值为：

$$\omega_2=\frac{M_i v_{x+1}{}^2-\alpha_{i,j}E_{ex}}{M_i(v_{x+1}{}^2-v_x{}^2)}=\frac{v_{x+1}{}^2-v_{l,cal}{}^2}{v_{x+1}{}^2-v_x{}^2} \tag{4-25}$$

同时，双电压组合执行的总执行时间t_2可表示为：

$$t_2=\frac{\omega_2 M_i}{\alpha_{i,j}f_x}+\frac{(1-\omega_2)M_i}{\alpha_{i,j}f_{x+1}}=\frac{[(f_{x+1}-f_x)\omega_2+f_x]M_i}{\alpha_{i,j}f_x f_{x+1}} \tag{4-26}$$

因此，t_1-t_2可表示为：

$$t_1-t_2=\frac{(f_x-\omega_2 f_{i,cal})(f_{x+1}-f_{i,cal})}{\alpha_{i,j}f_x f_{x+1}f_{i,cal}} \tag{4-27}$$

上式中，由于目标频率$f_{i,cal}$不可用，若采用单电压执行，$f_{i,cal}$仅存在两种可能的取值情况：

1）若$f_{i,cal}=f_x$，则t_1-t_2的值为：

$$t_1-t_2=\frac{f_x(1-\omega_2)(f_{x+1}-f_x)}{\alpha_{i,j}f_x^{\,2}f_{x+1}}>0 \tag{4-28}$$

2）若$f_{i,cal}=f_{x+1}$，则t_1-t_2的值为：

$$t_1-t_2=\frac{(f_x-\omega_2 f_{x+1})(f_{x+1}-f_{x+1})}{\alpha_{i,j}f_x f_{x+1}{}^2}=0 \tag{4-29}$$

然而，当选择f_{x+1}作为单频率执行时，v_{x+1}无法满足能耗约束条件，即$E_{x+1}>E_{ex}$，所以只能选择f_x才能在能耗约束下完成任务的执行。因此，综合上述分析可得$t_1-t_2>0$恒成立。这表明，双电压组合执行策略能够实现比单电压执行策略更优的任务完成时间，证毕■。

最后在步骤 16 中，根据优化后的执行时间，统计处理器核的时间使用情况，即：

$$core_avail(core_index)=[0,ts_{i,M},te_{i,M},D] \tag{4-30}$$

上式(4-30)中，处理器核θ_{core_index}上的可用时间区间为$[0,ts_{i,M}]$和$[te_{i,M},D]$。

为了更好地说明这一情况，经过算法 4-3 的执行，任务强制部分的分配优化结果如图 4-5 所示。以子任务$\tau_{2,o}$为例，其可用时间区间包括$\Delta t_{1,1}$、$\Delta t_{2,2}$及$\Delta t_{3,1}$，

分别对应处理器核θ_1上第一段空闲时间区间、处理器核θ_2上第二段空闲时间区间以及处理器核θ_3上第一段空闲时间区间，$\tau_{2,o}$具体的处理器核选择由下一阶段确定。

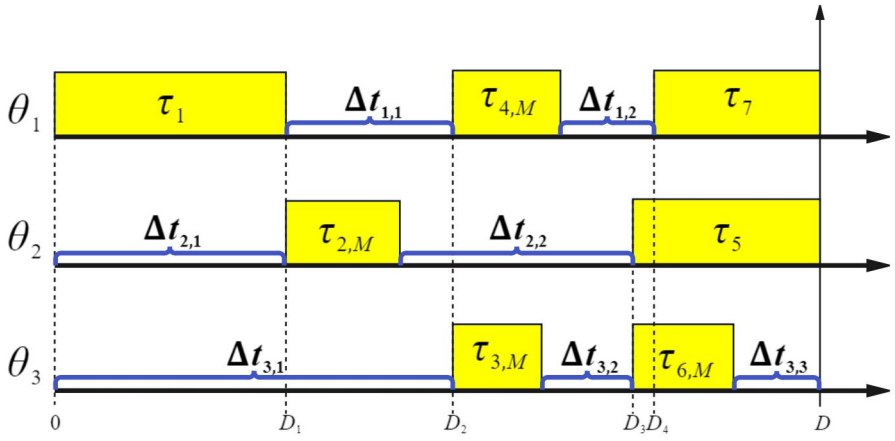


图 4-5 算法 4-3 结果示例

Fig.4-5 Example of algorithm 4-3 results

4.5.4 任务可选部分分配

根据算法 4-3 得出任务优化后的分配方案及处理器核可用空闲时间，本小结算法将为任务集中非精确任务的可选子任务选择合适的空闲时间区间及工作电压/频率执行。本小节算法流程如算法 4-4 所示：

算法 4-4 任务可选部分分配

输入：$\Theta=\{\theta_1,\theta_2,...,\theta_m\}$，$\Gamma'$，$\boldsymbol{Q}_{N\times N}$，$\boldsymbol{\alpha}_{N\times M}$，$\mu$，$t_{i,op}$

输出：分配和调度方案及系统服务质量

1:初始化 $E_{total,o}=0$;

2: **for** $i\in\Gamma'$ **do**

3: 初始化 $o_{\min}=0$, $core_index_option=0$, $level_index_option=0$;

4: 任务 $\tau_{i,o}$ 开始时间 $ts_{i,o}=te_i^M$;

5: 任务 $\tau_{i,o}$ 结束时间 $te_{i,o}=D_i$;

6: **for** $j=1,\ldots,M$ **do**

7: **for** $k=1{:}2{:}size(core_avail(core_index))$ **do**

8: **if** $core_avail(j)(k)\le ts_{i,o}$ && $core_avail(j)(k+1)\ge te_{i,o}$ **then**

9: 记录此处理器核下标 $core_index_option$;

10: **for** $l=0,1,\ldots,L$ **do**

算法 4-4 任务可选部分分配（续）

11: 计算可完成可选周期数 o_i;

12: **if** $o_i \le o_{min}$ **then**
13: $o_{min} = o_i$;
14: *core_index_option* = *j*;//更新处理器核下标
15: *level_index_option* = *l*;//更新工作电压/频率等级下标
16: **end if**
17: **end for**
18: **end if**
19: **end for**
20: 计算任务 $\tau_{i,o}$ 以 o_{min} 周期数执行所需能耗 $E_{i,o}$;
21: $E_{total,o}$=$E_{total,o} + E_{i,o}$;
22: **end for**
23: **end for**
24: $ESR_o = E_{total,o} / (1-\mu)E_{sup}$;
25: **for** $i \in \Gamma'$ **do**
26: $E_{i,ex} = ESR_o \times E_{i,o}$;
27: 计算 $o_{i,ex}$，见式(4-31);
28: **end for**
29: 输出分配和调度方案及系统服务质量;

算法 4-4 在步骤 1 中初始化任务可选部分能耗累加变量 $E_{total,o}$ 为 0，该变量用于统计系统中所有任务可选部分在执行过程中消耗的总能量，用于后续动态更新任务的实际能耗。同时，将任务分配的处理器核编号和工作电压/频率等级均初始化为 0，为后续任务分配提供初始参数。在步骤 2-3 中，算法从任务集合 Γ' 中按预定义优先级顺序提取当前待分配任务，并将任务 $\tau_{i,o}$ 的最小可选周期数 o_{min} 初始化为 0，该值作为可选周期数计算的初始下界。同时，将任务 $\tau_{i,o}$ 分配的处理器核编号 *core_index_option* 和工作电压/频率等级 *level_index_option* 均初始化为 0。

步骤 4-23 详细描述了任务 $\tau_{i,o}$ 的分配流程，其具体过程如下：首先确定任务 $\tau_{i,o}$ 的开始执行时间 $ts_{i,o}$ 和结束时间 $te_{i,o}$，为任务调度提供时间约束条件。遍历所有处理器核，寻找符合任务 $\tau_{i,o}$ 执行的可用空闲时间区间。计算满足条件的可选周期数 o_i，并确定在最小可选周期数 o_{min} 情况下的处理器核编号和工作电压/频率等级。进一步计算任务 $\tau_{i,o}$ 在此配置下的执行能耗 $E_{i,o}$，并实时更新任务可选部分系统总能耗 $E_{total,o}$。

步骤 24 计算任务的能耗延展比例 ESR_o，然后在步骤 25-28 中，计算延展后的能耗 $E_{i,ex}$，并计算在延展能耗下的 $o_{i,ex}$：

$$o_{i,ex}=\frac{\alpha_{i,j}E_{i,ex}}{v_l} \tag{4-31}$$

最后，输出分配和调度方案及系统服务质量。

为了更好地说明执行情况，经过算法 4-4 的执行，任务可选部分的分配优化结果如图 4-6 所示。

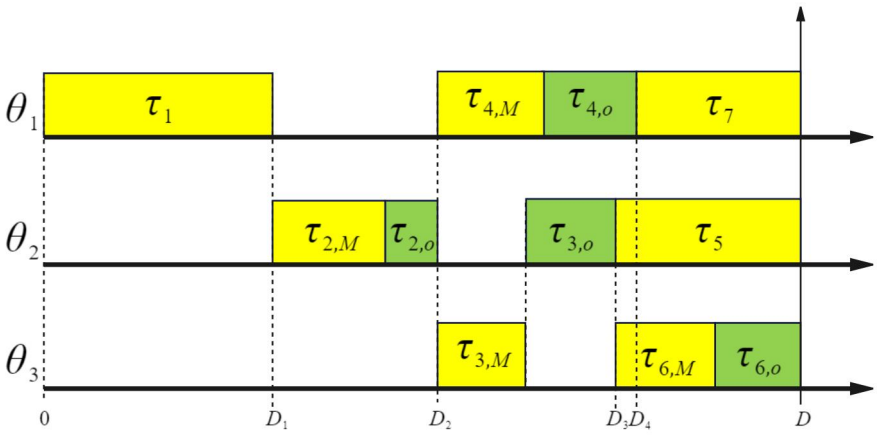


图 4-6 算法 4-4 结果示例

Fig.4-6 Example of algorithm 4-4 results

4.6 仿真结果与分析

为全面评估本章提出的 QHTS 算法的性能，本节设计并展开了多组对比实验。鉴于现有算法（如 AHS、GHS、EAS 等）主要针对非精确计算任务进行任务调度优化，而未考虑精确计算任务的存在，本节实验通过引入混合任务模型对上述算法进行扩展，分别得到 AHS-HT、GHS-HT 及 EAS-HT 算法，并将其与 QHTS 算法进行性能对比分析。

本节所有仿真实验均在 MATLAB R2023a 仿真平台上完成。实验环境配置如下：操作系统为 Windows10 专业版，硬件平台配备 16G 内存及 Intel Core i7-11800H@2.3GHz 处理器（八核十六线程）。

为量化评估 QHTS 算法的性能，本节实验采用标准化实现的服务质量指标作为评价标准，具体定义如下：

标准化实现的服务质量 NAQ：该指标表示各算法在实时性约束和能耗约束下能够实现的最大系统服务质量与系统所有非精确计算任务可选部分上限总和的比值，其数学表达式见式(3-36)。

4.6.1 仿真参数设置

本节仿真实验基于通用异构多核平台，旨在调度非固定数量的 DAG 任务。实验中的仿真变量参数取值范围详见表 4-1 所示。为模拟多样化任务场景，采用随机生成的方法为仿真实验生成不同规模的 DAG 任务集，其规模大小的取值范围参考文献[25]，[26]。假设所有处理器核支持 10 个离散频率执行点，其频率值分别为{0.1,0.2,…,1.0}。为确保实验数据的准确性并降低随机性影响，在仿真实验过程中，针对每组实验的每种情况均进行了 1000 次数据输入，汇总所有结果后剔除最高值与最低值，最终对剩余数据取平均值。

表 4-1 仿真参数范围设置

Table4-1 Simulation parameter range setting

参数名	参数取值范围
任务执行周期 M_i、O_i	$[4\times10^7,6\times10^8]$
执行效率 α	[0.4,1]
处理器核数量 M	8
小规模任务集	15-35
中等以上规模任务集	50-100

实验一：本实验旨在评估不同能耗影响因子 β 和时间影响因子 φ 对本章算法性能的影响。实验设计中，时间影响因子 φ 的取值范围设定为{0.2,0.4,0.6,0.8}，能耗影响因子 β 取值设定为{0.2,0.4,0.6,0.8}，任务数的取值范围为[5:5:100]。其他参数均参照表 4-1 中的变量范围进行设置。

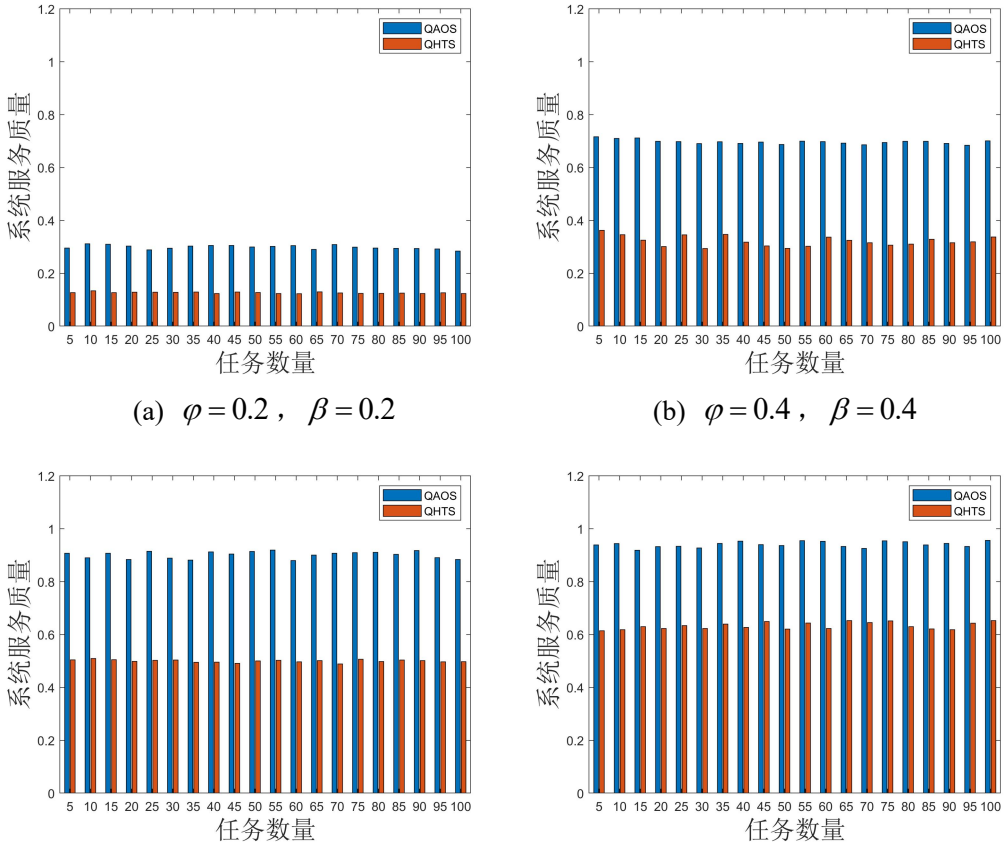
实验二：本实验旨在评估不同任务数量下各算法的服务质量性能表现以及能耗使用情况。实验设计中，时间影响因子 φ 固定为 0.5（或 0.2），能耗影响因子 β 固定为 0.5（或 0.2），任务数的取值范围设定为[5:5:100]。其他参数均参照表 4-1 中的变量范围进行设置。

实验三：本实验旨在评估任务模型中精确计算任务的不同占比对各算法的服务质量表现的影响。实验设计中，时间影响因子 φ 固定为 0.5，能耗影响因子 β 固定为 0.5，任务数分别设定为 15 和 50，精确计算任务数量的取值范围分别设定为[0:3:9]和[0:10:40]，其他参数均参照表 4-1 中的变量范围进行设置。

实验四：本实验旨在评估不同任务数量下的各算法的计算时间。在实验设计中，时间影响因子 φ 固定为 0.5，能耗影响因子 β 固定为 0.5，任务数的取值范围为[5:5:100]。其他参数均参照表 4-1 中的变量范围进行设置。

4.6.2 仿真结果分析

（1）实验一的仿真结果如图 4-7 及图 4-8 所示，本实验对比分析了本章提出的 QHTS 算法与 QAOS 算法在不同任务数量及不同的能耗影响因子β和时间影响因子φ条件下能够实现的系统服务质量。图 4-7(a-d)分别展示了不同β及φ下，QHTS 算法与 QAOS 算法的系统服务质量随任务数量变化的对比结果。实验结果表明，QHTS 算法平均能达到 QAOS 算法 52.83%的服务质量性能。这是由于 QHTS 算法与其他启发式算法类似，其求解得到的是次优解，难以接近近似最优算法获得的最优解。这一局限性主要源于启发式算法本身的设计特性，其在求解过程中倾向于通过局部搜索和启发式规则快速获得可行解，而非全局最优解。因此，QHTS 算法在解的质量上与理论上的最优解存在一定差距。定量分析表明，QHTS 算法的平均系统服务质量值在$\varphi=0.2$，$\beta=0.2$时平均可以达到 QAOS 算法的 42.41%，在$\varphi=0.4$，$\beta=0.4$时平均可以达到 QAOS 算法的 46.16%，在$\varphi=0.6$，$\beta=0.6$时平均可以达到 QAOS 算法的 55.45%，在$\varphi=0.8$，$\beta=0.8$时平均可以达到 QAOS 算法的 67.28%。



(a) $\varphi=0.2$，$\beta=0.2$　　(b) $\varphi=0.4$，$\beta=0.4$

(c) $\varphi=0.6$，$\beta=0.6$　　(d) $\varphi=0.8$，$\beta=0.8$

图 4-7 QHTS 算法与 QAOS 算法系统服务质量对比

Fig.4-7 System QoS comparison between QHTS and QAOS

图 4-8 展示了不同β及φ下，QHTS 算法的系统服务质量随任务数量变化的趋势。实验结果表明，在小型、中型及大型的任务规模范围内，系统服务质量均随β和φ的增加而显著提升。这一现象可归因于以下机制：如图 3-4(a)所示，能耗影响因子β与系统预设能耗呈正相关关系，系统能提供更多的能耗预算，可选子任务执行的周期数更多。同时如图 3-4(b)所示，时间影响因子φ与任务相对截止期限亦呈正相关关系，任务的截止期限越高，可选子任务执行的周期数更多。

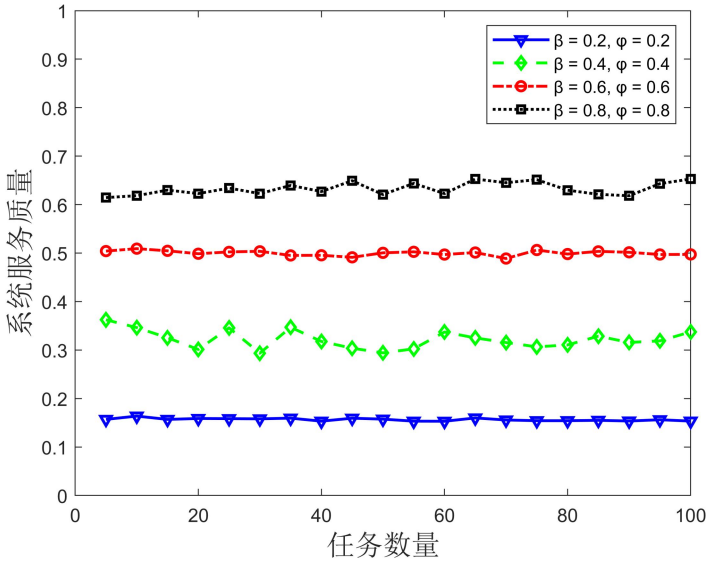


图 4-8 QHTS 算法在不同 N 、β及φ条件下的系统服务质量变化趋势

Fig.4-8 Trends in system QoS achieved by the QHTS with N, β and φ varying

（2）实验二的仿真结果如图 4-9、图 4-10 及图 4-11 所示，本实验对比分析了本章提出的 QHTS 算法和上一章节提出的 QAOS 算法以及 AHS-HT、GHS-HT、EAS-HT 等算法在时间影响因子φ固定为 0.5（或 0.2），能耗影响因子β固定为 0.5（或 0.2）条件下，任务集（不包含精确计算任务）中不同任务数量对系统服务质量性能和能耗使用的影响。

图 4-9 及图 4-10 分别展示了不同算法在$\varphi=0.5$，$\beta=0.5$和$\varphi=0.2$，$\beta=0.2$条件下不同任务数量对系统服务质量性能的影响。实验结果表明，在任务数量变化的情况下，QHTS 算法的系统服务质量均优于 AHS-HT、GHS-HT 及 EAS-HT 算法，这是由于 QHTS 算法在设计中引入了 DVFS 技术以及任务迁移策略，而 GHS-HT 及 EAS-HT 算法未充分考虑这些因素。此外，与 AHS-HT 算法相比，QHTS 算法在非精确计算任务强制部分分配阶段充分利用任务并行性。同时，在空闲时

间优化阶段，QHTS 算法充分利用 DVFS 技术进行工作电压的缩放，最大化处理器核的可用空闲时间，能比 AHS-HT 算法为非精确计算任务可选部分提供更多的执行时间。定量分析表明，QHTS 算法的平均系统服务质量值比 AHS-HT 算法高 7.22%，比 GHS-HT 算法高 49.18%，比 EAS-HT 算法高 48.78%，可以达到 QAOS 算法的 50.34%。

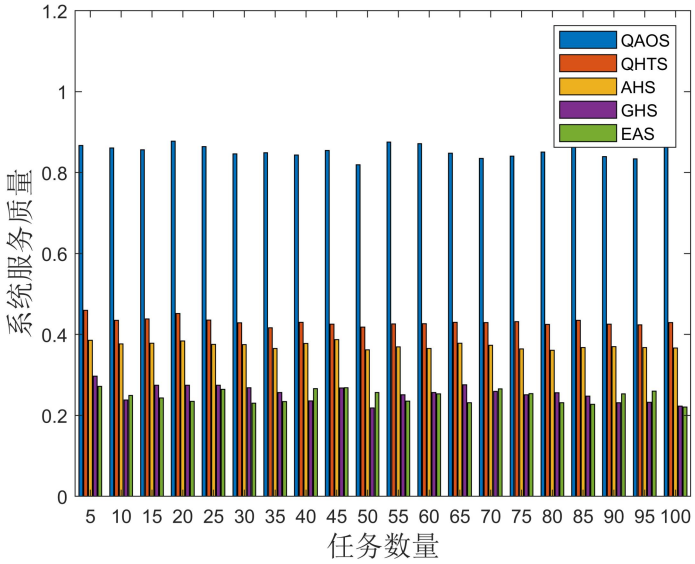


图 4-9 $\varphi = 0.5$，$\beta = 0.5$时不同算法的服务质量对比

Fig.4-9 QoS comparison in different algorithms when $\varphi = 0.5$ and $\beta = 0.5$

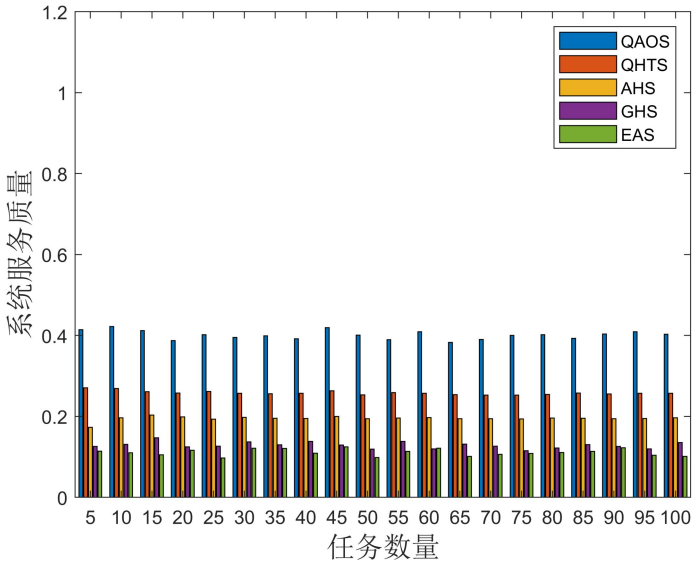
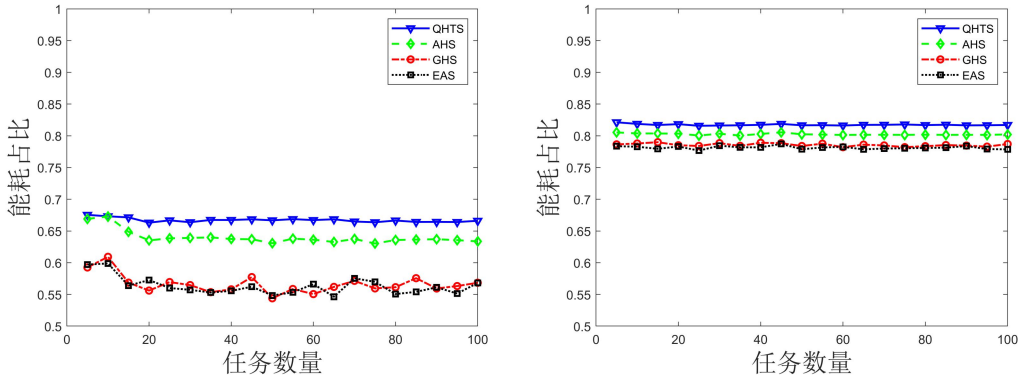


图 4-10 $\varphi = 0.2$，$\beta = 0.2$ 不同算法的服务质量对比

Fig.4-10 QoS comparison in different algorithms when $\varphi = 0.2$ and $\beta = 0.2$

图 4-11(a-b)分别展示了不同算法在$\varphi=0.5$，$\beta=0.5$和$\varphi=0.2$，$\beta=0.2$条件下的不同任务数量对能耗使用的影响，实验结果表明，QHTS 算法得到分配和调度方案使用的能耗均高于其他启发式算法，这是因为相较于其他算法，QHTS 算法在多阶段算法中充分利用 DVFS 技术对任务工作电压/频率进行调整，其能更多地利用系统提供的能耗以实现更高的系统服务质量。定量分析表明，QHTS 算法的平均能耗占比值比 AHS-HT 算法高 6.25%，比 GHS-HT 算法高 19.29%，EAS-HT 算法高 21.65%。



(a) $\varphi=0.5$，$\beta=0.5$　　(b) $\varphi=0.2$，$\beta=0.2$

图 4-11 不同算法的使用能耗对比

Fig.4-11 Energy comparison in different algorithms

（3）实验三的仿真结果如图 4-12 及图 4-13 所示，本实验对比分析了本章提出的 QHTS 算法和上一章节提出的 QAOS 算法以及 AHS-HT、GHS-HT、EAS-HT 等算法在时间影响因子φ固定为 0.5，能耗影响因子β固定为 0.5 条件下，任务集中不同精确计算任务数量对系统服务质量性能的影响。

图 4-12 展示了任务数量为 15 时，QHTS 算法的系统服务质量随任务集中精确计算任务数量的变化的趋势。实验结果表明，在任务集中精确计算任务数量变化的情况下，QHTS 算法的系统服务质量均优于 AHS-HT、GHS-HT 及 EAS-HT 算法。这一优势主要归因于 QHTS 算法在非精确计算任务强制部分分配阶段，通过充分利用 DVFS 技术实现了工作频率的选择和缩放，从而有效降低了精确计算任务使用的能耗；而 AHS-HT、GHS-HT 及 EAS-HT 算法未对精确计算任务进行进一步优化。此外，在任务集中精确计算任务占比较大（精确计算任务数量为 9）时，QHTS 算法依然能够保持 29.8%的系统服务质量性能，而 GHS-HT 及 EAS-HT 算法的系统服务质量性能显著下降并趋近于 0。这是因为 GHS-HT 及 EAS-HT 算法未充分考虑 DVFS 技术以及任务迁移策略，且在任务分配策略中未对精确计算任务进行

优化。精确计算任务的分配会占用大量的系统资源（能耗和执行时间），从而导致为非精确计算任务的可选子任务执行所预留的剩余资源相对有限。定量分析表明，在任务集中存在精确计算任务的条件下，QHTS 算法的平均系统服务质量值比 AHS-HT 算法高 31.05%，比 GHS-HT 算法高 59.47%，比 EAS-HT 算法高 61.72%，可以达到 QAOS 算法的 45.65%。

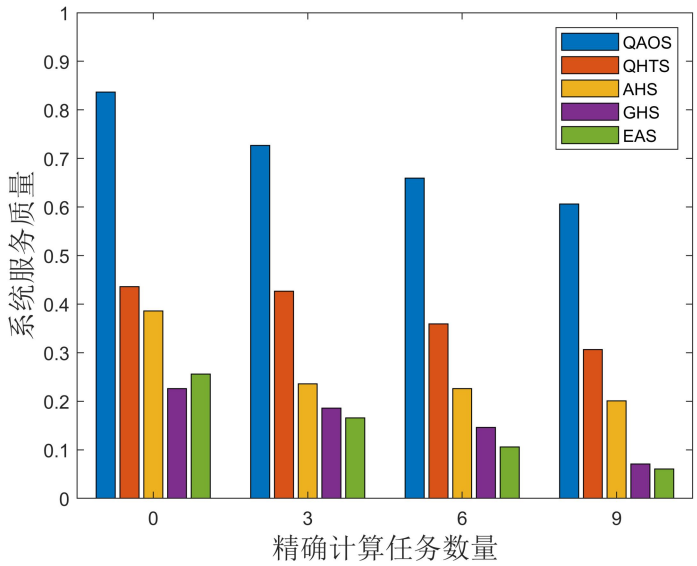


图 4-12 混合任务模型中不同算法的服务质量对比

Fig.4-12 QoS comparison in different algorithms in hybrid task models

图 4-13 展示了任务数量 50 时 QHTS 算法的系统服务质量随任务集中精确计算任务数量的变化的趋势。与图 4-12 一致，QHTS 算法的系统服务质量均优于其他启发式算法 AHS-HT、GHS-HT、EAS-HT。同时，当任务集中精确计算任务占比较大（精确计算任务数量为 30 或 40）时，QHTS 算法依然能够保持 27.53%~15.23%的系统服务质量性能，而启发式算法 GHS-HT 和 EAS-HT 的系统服务质量性能则显著下降并趋近于 0，这些现象的原因与前述分析同理。定量分析表明，在任务集中存在精确计算任务的条件下，QHTS 算法的平均系统服务质量值比 AHS-HT 算法高 32.05%，比 GHS-HT 算法高 64.22%，比 EAS-HT 算法高 63.79%，可以达到 QAOS 算法的 38.71%。

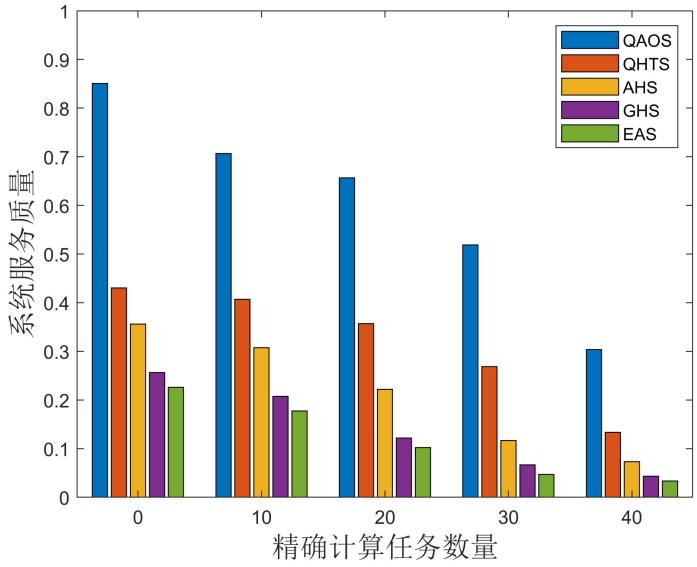


图 4-13 混合任务模型中不同算法的服务质量对比

Fig.4-13 QoS comparison of in different algorithms in hybrid task models

（4）实验四的仿真结果如图 4-14 所示，本实验对比分析了本章提出的 QHTS 算法和上一章节提出的 QAOS 算法以及 AHS-HT、GHS-HT、EAS-HT 等算法在时间影响因子 φ 固定为 0.5，能耗影响因子 β 固定为 0.5 条件下，任务集中不同任务数量对算法计算时间的影响。实验结果表明，QHTS 算法略高于其他启发式算法，这是因为相较于其他算法，QHTS 算法使用 DVFS 进行了更多的工作电压/频率缩放的操作；但 QHTS 算法的计算时间远远低于上一章节提出的 QAOS 算法，可以忽略不计。

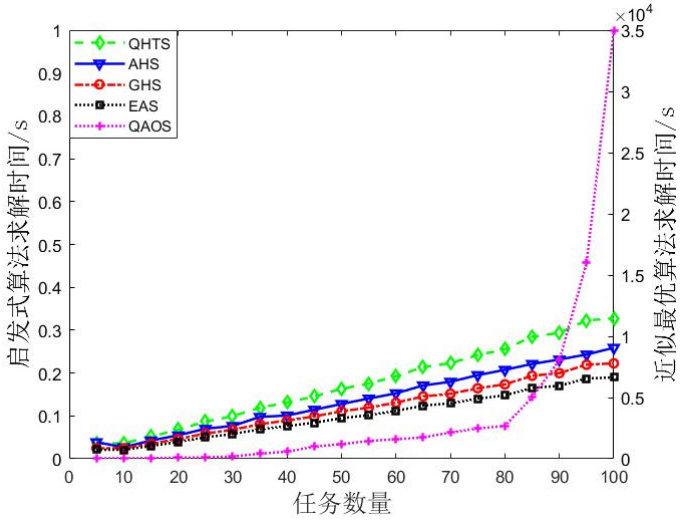


图 4-14 不同算法计算时间

Fig.4-14 Computation time in in different algorithms

4.7 本章小结

本章提出了一种异构多核平台下的服务质量感知的混合任务调度启发式算法。首先，本章对所提出启发式算法所涉及的实时性约束进行了补充定义；然后，对本章所研究问题进行形式化定义和分析，并提出了 QHTS 算法，该算法主要分为三个阶段完成：第一阶段在预设能耗范围内最小化强制部分和精确计算任务执行所需能耗，完成精确计算任务和非精确计算任务强制部分频率选择及处理器核分配；第二阶段通过 DVFS 技术充分使用预设能耗以最大化处理器核上的空闲时间，为任务强制部分的调度以处理器空闲时间的优化；第三阶段再次使用 DVFS 技术最大化非精确任务可选部分执行量，完成非精确计算任务可选部分的分配与调度，实现系统 服务质量的最大化。最后，通过设计仿真实验进行算法性能对比，实验结果表明，对于不包含精确计算任务的任务模型，QHTS 算法相较于 AHS-HT、GHS-HT、EAS-HT 等算法系统服务质量性能平均提高 13.40%~49.18%，且平均能达到 QAOS 算法的 50.34%；对于混合任务模型，QHTS 算法相较于 AHS-HT、GHS-HT、EAS-HT 等算法系统服务质量性能平均提高 31.05%~64.22%，但是该算法能耗使用提高了 6.25%~21.65%。此外，QHTS 算法平均能达到 QAOS 算法的 38.71%~45.65%，且其计算时间远远低于 QAOS 算法，在面对中大型任务集规模时仍能迅速得到结果。

第 5 章 总结与展望

5.1 主要工作与创新点

在信息和通信技术迅速进步的背景下，具备异构多核处理器的嵌入式系统更能满足现代应用场景的高性能需求。异构多核处理器通过集成多种功能各异的处理器核心，针对特定任务优化设计，实现了计算资源的灵活配置与性能提升，广泛应用于工业控制、自动驾驶、通信及图像处理等领域。服务质量是体现系统性能的重要指标，尤其是对于存在非精确计算任务的应用领域。现有的研究中，要么是针对精确计算任务的调度研究，要么是针对非精确计算任务的调度研究，但少有考虑同时包含精确计算与非精确计算任务的混合任务集。混合任务集的调度问题尤为复杂，如何在能耗约束与实时性要求下实现服务质量最大化，成为异构多核平台任务调度的核心问题。本文主要工作是在异构多核平台上设计满足能耗约束及实时性约束的服务质量感知的混合任务调度算法。本文的主要工作与创新点如下：

（1）当前研究主要关注精确计算任务或非精确计算任务的调度问题，而对混合任务模型的研究较少。为此，本文基于通用异构多核平台，针对混合任务模型，综合考虑能耗、实时性及任务依赖等多重约束，构建了以最大化系统服务质量为目标的任务调度模型。为降低计算复杂度，本文进一步采用线性化与 Benders 分解方法，将原问题分解为主问题和子问题。主问题优化整数变量，子问题则以主问题的解为参数优化连续变量。

（2）基于分解后的问题框架，本文提出了一种服务质量感知的混合任务调度近似最优算法。该算法通过主问题与子问题的交替迭代求解，利用 YALMIP 和 CPLEX 求解器逐步逼近最优解。该算法不仅降低了求解难度，还通过动态调整任务分配与资源调度策略，实现了系统服务质量的最大化，为后续启发式算法提供了理论参考。

（3）针对前文提出的近似最优算法在中大规模任务集下求解时间长、实现复杂度高的问题，本文提出了一种服务质量感知的混合任务调度启发式算法。该算法采用多阶段策略，分为强制任务分配、空闲时间优化及可选任务分配三个阶段。第一阶段根据任务分层结构及精确计算任务和非精确计算任务强制子任务计算量，确定分配优先级，在能耗约束下最小化此阶段的总能耗，生成初步分配方案。第二阶段通过 DVFS 技术优化处理器核的频率与电压，最大化空闲时间，为非精确计算任务可选子任务分配提供资源。第三阶段遍历非精确计算任务可选子任务的

可用处理器核，寻找最大化执行量的分配方案，以提升系统服务质量。该算法通过分阶段优化，显著降低了求解时间并提高了工程可实现性。

5.2 研究展望

在异构多核平台中，如何在满足实时性约束和能耗约束的前提下最大化系统服务质量仍然是当前学术界的一个难题。本文虽然提出了混合任务在通用异构多核平台上的服务质量优化问题，并提出了低复杂度的启发式算法，但在研究过程中仍存在一些局限性。基于对本文研究内容的总结，后续工作可以从以下几个方面进一步展开：

（1）本文提出的算法基于固定数量的处理器核及其固定的工作电压/频率范围进行任务调度优化。然而，实际系统中处理器核的数量及其性能参数（如工作电压/频率）通常是动态可调的。因此，后续的研究应进一步探索在不同处理器核数量及性能参数条件下的调度优化策略，以提升算法的适用性。

（2）本文提出的算法都是在给定任务集下进行的，即任务图的结构不会发生变化，任务的分配和调度方案在执行前就已确定。然而，在实际系统中任务图的结构可能会根据任务执行状态动态变化，这就导致预先根据调度算法得到的任务的分配和调度方案可能不再适用。因此，对于这种变化的动态调度问题也是后续研究的重要内容之一。

（3）本文提出的算法仅限于理论仿真实验，缺乏在实际平台以及真实应用场景中的验证。虽然本文提出的算法在理论层面能实现比较好的性能，但其在实际应用中的表现仍需进一步验证。因此，后续工作应将算法部署在实际异构多核平台上，通过调度真实任务集进一步验证算法的有效性，并根据实际性能表现进行优化改进。

参考文献

[1] 郭辉. 面向图计算的 GPU 体系结构优化关键技术研究[D]. 湖南：国防科技大学, 2018: 1-10.

[2] 陈刚, 关楠, 吕鸣松, 等. 实时多核嵌入式系统研究综述[J]. 软件学报, 2018, 29(7): 2152-2176.

[3] ROY S K, DEVARAJ R, SARKAR A, et al. SLAQA: Quality-level aware scheduling of task graphs on heterogeneous distributed systems[J]. ACM Transactions on Embedded Computing Systems (TECS), 2021, 20(5): 1-31.

[4] ZHOU J, SHEN Y, LI L, et al. Swarm intelligence-based task scheduling for enhancing security for IoT devices[J]. IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, 2022, 42(6): 1756-1769.

[5] 瞿伟, 余飞鸿. 基于多核处理器的非对称嵌入式系统研究综述[J]. 计算机科学, 2021, 48(S1): 538-542.

[6] SINGH A K, SHAFIQUE M, KUMAR A, et al. Mapping on multi/many-core systems: Survey of current and emerging trends[C]. Proceedings of the 50th annual design automation conference, New York, USA, 2013: 1-10.

[7] YOO S. An empirical validation of power-performance scaling: DVFS vs. multi-core scaling in big. LITTLE processor[J]. IEICE Electronics Express, 2015, 12(8): 20150236-20150236.

[8] SADEGHI A R, WACHSMANN C, WAIDNER M. Security and privacy challenges in industrial internet of things[C]. Proceedings of the 52nd annual design automation conference, New York, USA, 2015: 1-6.

[9] SAHA S, ZHAI X, EHSAN S, et al. RASA: Reliability-aware scheduling approach for FPGA-based resilient embedded systems in extreme environments[J]. IEEE transactions on systems, man, and cybernetics: systems, 2021, 52(6): 3885-3899.

[10] LIN K J, NATARAJAN S, LIU J W S. Imprecise results: Utilizing partial computations in real-time systems[R]. USA: National Aeronautics and Space Administration, 1987.

[11] 汪俊马. 自主获能系统中能量自适应的非精确计算任务分配和调度机制研究[D]. 上海: 华东师范大学, 2016: 1-6.

[12] MITTAL S. A survey of techniques for cache locking[J]. ACM Transactions on Design Automation of Electronic Systems (TODAES), 2016, 21(3): 1-24.

[13] YU H, HA Y, VEERAVALLI B. Quality-driven dynamic scheduling for real-time adaptive applications on multiprocessor systems[J]. IEEE Transactions on Computers, 2012, 62(10): 2026-2040.

[14] SAROJA S, REVATHI T, AULUCK N. Multi-criteria decision-making for heterogeneous multiprocessor scheduling[J]. International Journal of Information Technology & Decision Making, 2018, 17(05): 1399-1427.

[15] CHEN G, HUANG K, KNOLL A. Energy optimization for real-time multiprocessor system-on-chip with optimal DVFS and DPM combination[J]. ACM Transactions on Embedded Computing Systems (TECS), 2014, 13(3s): 1-21.

[16] ESMAILI A, NAZEMI M, PEDRAM M. Modeling processor idle times in MPSoC platforms to enable integrated DPM, DVFS, and task scheduling subject to a hard deadline[C]. Proceedings of the 24th Asia and South Pacific Design Automation Conference, New York, USA, 2019: 532-537.

[17] LI D, WU J. Minimizing energy consumption for frame-based tasks on heterogeneous multiprocessor platforms[J]. IEEE Transactions on Parallel and Distributed Systems, 2014, 26(3): 810-823.

[18] LI Y, NIU J, ATIQUZZAMAN M, et al. Energy-aware scheduling on heterogeneous multi-core systems with guaranteed probability[J]. Journal of Parallel and Distributed Computing, 2017, 103: 64-76.

[19] MO L, KRITIKAKOU A, SENTIEYS O. Energy-quality-time optimized task mapping on DVFS-enabled multicores[J]. IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, 2018, 37(11): 2428-2439.

[20] KIM K H. Reward-based allocation of cluster and grid resources for imprecise computation model-based applications[J]. International Journal of Web and Grid Services, 2013, 9(2): 146-171.

[21] RAVINDRAN R C, KRISHNA C M, KOREN I, et al. Scheduling imprecise task graphs for real-time applications[J]. International Journal of Embedded Systems, 2014, 6(1): 73-85.

[22] WEI T, ZHOU J, CAO K, et al. Cost-Constrained QoS Optimization for Approximate Computation Real-Time Tasks in Heterogeneous MPSoCs.[J]. IEEE Trans. on CAD of Integrated Circuits and Systems, 2018, 37(9): 1733-1746.

[23] ZHOU J, YAN J, WEI T, et al. Energy-adaptive scheduling of imprecise computation tasks for QoS optimization in real-time MPSoC systems[C]. Design, Automation & Test in Europe Conference & Exhibition (DATE), Leuven, BEL, 2017: 1402-1407

[24] YU H, HA Y, VEERAVALLI B. Quality-driven dynamic scheduling for real-time adaptive applications on multiprocessor systems[J]. IEEE Transactions on Computers, 2012, 62(10): 2026-2040.

[25] MO L, KRITIKAKOU A, SENTIEYS O. Approximation-aware task deployment on asymmetric multicore processors[C]. 2019 Design, Automation & Test in Europe Conference & Exhibition (DATE), Florence, Italy, 2019: 1513-1518.

[26] LI X, MO L, KRITIKAKOU A, et al. Approximation-aware task deployment on heterogeneous multicore platforms with DVFS[J]. IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, 2022, 42(7): 2108-2121.

[27] 彭超. 基于图的实时任务模型的可调度性分析与性能优化技术研究[D]. 湖南: 国防科技大学, 2019: 15-20.

[28] HUANG K, JIANG X, ZHANG X, et al. Energy-efficient fault-tolerant mapping and scheduling on heterogeneous multiprocessor real-time systems[J]. IEEE Access, 2018, 6: 57614-57630.

[29] AMINZADEH S, EJLALI A. A Comparative Study of System-Level Energy Management Methods for Fault-Tolerant Hard Real-Time Systems.[J]. IEEE Trans. Computers, 2011, 60(9): 1288-1299.

[30] HONG, SHENGYAN, CHANTEM, et al. Local-Deadline Assignment for Distributed Real-Time Systems[J]. IEEE Transactions on Computers, 2015, 64(7): 1983-1997.

[31] 彭林. 软件事务存储并行编程环境关键技术研究[D]. 湖南: 国防科学技术大学, 2010: 15-20.

[32] 陈磊. 异构多核架构下的高能效混合任务调度算法研究[D]. 重庆: 重庆邮电大学, 2022: 10-20.

[33] 张志超, 王剑, 章隆兵, 肖俊华. 基于软硬件协同加速的关系网络推理优化方法[J]. 高技术通讯, 2022, 32(04): 327-336.

[34] ACHARYA G P, RANI M A. FPGA Prototyping of Micro-Blaze soft-processor based Multi-core System on Chip[J]. International Journal of Engineering & Technology, 2018, 7(2.16): 57-60.

[35] 黄乐天, 别丽华. 同构与异构片上多核系统的演进过程[J]. 电子技术应用, 2017, 43(3): 6-11.

[36] BENATIA A, JI W, WANG Y, et al. Sparse Matrix Partitioning for Optimizing SpMV on CPU-GPU heterogeneous platforms[J]. The International Journal of High Performance Computing Applications, 2020, 34(1): 66-80.

[37] MITTAL S. A survey of techniques for architecting and managing asymmetric multicore processors[J]. ACM Computing Surveys (CSUR), 2016, 48(3): 1-38.

[38] KAHLE J A, DAY M N, HOFSTEE H P, et al. Introduction to the Cell multiprocessor[J]. IBM journal of Research and Development, 2005, 49(4.5): 589-604.

[39] 童辉辉. 面向异构多核处理器的混合式任务调度算法研究[D]. 广西: 桂林理工大学, 2021: 15-30.

[40] KOOHI S Z, HAMID N A W A, OTHMAN M, et al. MEMPHA: Model of Exascale Message-Passing Programs on Heterogeneous Architectures[J]. IEEE Transactions on Parallel and Distributed Systems, 2020, 31(11): 2570-2581.

[41] DE FALCO I, SCAFURI U, TARANTINO E. Two new fast heuristics for mapping parallel applications on cloud computing[J]. Future Generation Computer Systems, 2014, 37: 1-13.

[42] KOOHI S Z, HAMID A, OTHMAN M, et al. On modelling parallel programmers for static mapping: A comparative study[J]. Pertanika J. Sci. Technol, 2018, 26(2): 523-543.

[43] ZHANG P Y, ZHOU M C. Dynamic Cloud Task Scheduling Based on a Two-stage Strategy[J]. IEEE Transactions on Automation Science and Engineering, 2017, 15(2): 772~783.

[44] HAN J J, LI Q H. A novel static task scheduling algorithm in distributed computing environment[C]. 18th International Parallel and Distributed Processing Symposium, 2004. Proceedings, Santa Fe, NM, USA, 2004: 3-12.

[45] 陈晓明. 异构分布式嵌入式系统的高能效与安全性调度研究[D]. 湖南: 湖南大学, 2018: 15-30.

[46] CHENG Y, WU Z, LIU K, et al. Smart Dag Tasks Scheduling Between Trusted and Untrusted Entities Using the MCTS method[J]. Sustainability, 2019, 11(7): 1826-1842.

[47] CHUNG J Y, LIU J W S, LIN K J. Scheduling periodic jobs that allow imprecise results[J]. IEEE transactions on computers, 1990, 39(9): 1156-1174.

[48] AMIRIJOO M, HANSSON J, SON S H. Specification and management of QoS in real-time databases supporting imprecise computations[J]. IEEE Transactions on Computers, 2006, 55(3): 304-319.

[49] HUANG H, CHATURVEDI V, QUAN G, et al. Throughput maximization for periodic real-time systems under the maximal temperature constraint[J]. ACM Transactions on Embedded Computing Systems (TECS), 2014, 13(2s): 1-22.

[50] 夏军, 袁帅, 杨逸. 基于异构多核平台低能耗周期任务调度算法[J]. 计算机应用, 2019, 39(10): 2980-2984.

[51] CORTÉS L A, ELES P, PENG Z. Quasi-static assignment of voltages and optional cycles in imprecise-computation systems with energy considerations[J]. IEEE transactions on very large scale integration (VLSI) systems, 2006, 14(10): 1117-1129.

[52] 杨逸. 异构多核计算系统的节能任务调度算法研究[D]. 重庆: 重庆邮电大学, 2020: 10-15.

[53] MARTIN S M, FLAUTNER K, MUDGE T, et al. Combined dynamic voltage scaling and adaptive body biasing for lower power microprocessors under dynamic workloads[C]. Proceedings of the 2002 IEEE/ACM international conference on Computer-aided design, New York, USA, 2002: 721-725.

[54] KHAN A A. Developing an Energy-efficient Real-time System[M]. Missouri: Missouri University of Science and Technology, 2018: 150-153.

[55] JEJURIKAR R, PEREIRA C, GUPTA R. Leakage aware dynamic voltage scaling for real-time embedded systems[C]. Proceedings of the 41st annual Design Automation Conference, New York, USA, 2004: 275-280.

[56] POUWELSE J, LANGENDOEN K, SIPS H J. Application-directed voltage scaling[J]. IEEE Transactions on Very Large Scale Integration (VLSI) Systems, 2003, 11(5): 812-826.

[57] CHAKRABORTY S, SAHA S, SJÄLANDER M, et al. Prepare: P owe r-Awar e A p proximate Re a l-time Task Scheduling for Energy-Adaptive QoS Maximization[J]. ACM Transactions on Embedded Computing Systems (TECS), 2021, 20(5s): 1-25.

[58] MO L, KRITIKAKOU A, SENTIEYS O. Energy-quality-time optimized task mapping on DVFS-enabled multicores[J]. IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, 2018, 37(11): 2428-2439.

[59] HUANG H, CHATURVEDI V, QUAN G, et al. Throughput maximization for periodic real-time systems under the maximal temperature constraint[J]. ACM Transactions on Embedded Computing Systems (TECS), 2014, 13(2s): 1-22.

[60] CHWA H S, SEO J, YOO H, et al. Energy and feasibility optimal scheduling on big. LITTLE platforms[J]. Proc. IEEE RTSOPS, 2013: 770-777.

[61] QIN Y, ZENG G, KURACHI R, et al. Execution-variance-aware task allocation for energy minimization on the big. little architecture[J]. Sustainable Computing: Informatics and Systems, 2019, 22: 155-166.

[62] BENDERS J. Partitioning procedures for solving mixed-variables programming problems[J]. Computational Management Science, 2005, 2(1).

[63] ZHANG Y, WWANG Y, TANG X, et al. Energy-efficient task scheduling on heterogeneous computing systems by linear programming[J]. Concurrency and Computation: Practice and Experience, 2018, 30(19): e4731.

作者简介

1. 基本情况

2. 教育和工作经历

3. 攻读学位期间的研究成果

3.1 申请（授权）专利

[1] 夏军, **何爽**. 异构多核系统高能效任务分配方法和装置: CN202311189140.3[P]. 2023.9.15.

3.2 参与的科研项目及获奖

[1] 重庆邮电大学 2022 年度学业二等奖学金.
[2] 重庆邮电大学 2023 年度学业三等奖学金.
[3] 重庆邮电大学 2024 年度学业二等奖学金.

致　谢